

Production Web Systems Handbook

Next.js + React + FastAPI from Browser to Production

A first-principles reference for request journeys, rendering, side effects, real-time systems, performance, security, debugging, and production architecture

How to use this book. Do not memorize recipes. For every bug or design review, identify (1) the trigger, (2) where the code executes, (3) whether it is pure or a side effect, (4) which network boundaries are crossed, (5) which state changes, and (6) what evidence can prove the behavior.

Reference edition - framework APIs evolve; verify version-specific syntax against the official documentation used by your project.

Reading map

This handbook is intentionally broader than a React or Next.js course. It is designed for production work where a single feature can involve the browser runtime, React rendering, Next.js server execution, a reverse proxy, FastAPI, a database, and a persistent WebSocket at the same time.

The core mental model

Trigger → execution environment → pure calculation or side effect → network/process boundary → state change → render/response → observable evidence.

Use Parts I-IV until raw HTTP/browser behavior feels ordinary. Use Parts V-VII when reviewing application code. Use Parts VIII-XI for production readiness. Part XII is a troubleshooting index based on common failures in Next.js + FastAPI applications.

A request journey in one page

```
Browser click -> React event handler -> fetch('/api/users') -> browser HTTP stack -> load  
balancer/reverse proxy -> Next.js Route Handler -> server-side fetch(API_URL) -> FastAPI  
middleware -> dependency/auth -> route/service -> database -> JSON response -> Next.js response  
-> browser -> state update -> React render -> commit -> layout/paint
```

Each arrow may represent a new trust boundary, process, connection, request, cache, timeout, or failure mode. A reviewer who asks you to rewrite code is often reacting to one of those boundaries: the code runs in the wrong environment, a side effect is attached to the wrong lifecycle, authorization is enforced in the wrong layer, or a request makes an unnecessary hop.

Debugging card

1. Reproduce the issue with the smallest reliable sequence.
2. Open browser Network. Record URL, method, initiator, status, headers, payload, timing, redirects, and duplicate requests.
3. If React is involved, record a React Profiler interaction. Separate render from commit and browser paint.
4. Give the request a correlation ID and find the same ID in Next.js/FastAPI logs.
5. Draw the request path. Mark every place a new request is created.
6. Change one hypothesis at a time. A fix is not proven until the evidence changes in the predicted way.
7. Add a test, monitor, or assertion that would catch the regression.

Contents

Reading map	ii
A request journey in one page	iii
Debugging card	iv
1 I. First-Principles Web Thinking	1
1.1 The request-first debugging model	2
1.2 Client, server, process, thread, and network boundaries	4
1.3 Pure computation versus side effects	6
1.4 A production debugging workflow	8
2 II. URLs, Networking, and HTTP	10
2.1 URL, URI, origin, path, query, and fragment	11
2.2 DNS resolution	13
2.3 TCP, sockets, ports, and connection state	15
2.4 QUIC and HTTP/3	17
2.5 TLS and HTTPS	19
2.6 HTTP request anatomy	21
2.7 HTTP response anatomy	23
2.8 HTTP methods and idempotency	25
2.9 HTTP status codes	27
2.10 HTTP headers	29
2.11 Content-Type, Accept, encoding, and bodies	31
2.12 Cookies and browser-managed credentials	33
2.13 Sessions, access tokens, and refresh tokens	35
2.14 HTTP caching and validators	37
2.15 Same-origin policy and CORS	39
2.16 Browser storage	41
3 III. Browser Runtime and Rendering	43
3.1 Browser navigation lifecycle	44
3.2 HTML parsing and the DOM	46
3.3 CSS cascade, inheritance, and CSSOM	48
3.4 Style, layout, paint, and composite	50
3.5 JavaScript engine, call stack, and heap	52
3.6 The event loop	54
3.7 DOM events and propagation	56
3.8 HTML forms and native browser behavior	58
3.9 History, navigation, and URL state	60
3.10 Focus, keyboard behavior, and accessibility	62
3.11 Core Web Vitals and browser performance	64
3.12 Service workers and offline caches	66
4 IV. JavaScript Networking and Real-Time Communication	68
4.1 The Fetch API	69
4.2 Axios	71
4.3 Promises and async/await	73
4.4 Cancellation with AbortController	75

4.5	Race conditions and stale responses	77
4.6	Retries, backoff, and retry safety	79
4.7	Idempotency keys	81
4.8	API clients and Axios interceptors	83
4.9	Polling	85
4.10	Long polling	87
4.11	Server-Sent Events	89
4.12	WebSockets	91
4.13	Socket.IO	93
4.14	Choosing polling, SSE, WebSocket, or queues	95
5	V. React from First Principles	97
5.1	React component execution	98
5.2	Trigger, render, commit, and browser paint	100
5.3	Render purity	102
5.4	Props, state, and derived values	104
5.5	State updates, batching, and snapshots	106
5.6	Event handlers	108
5.7	useEffect as synchronization	110
5.8	Effect dependency arrays	112
5.9	Stale closures	114
5.10	Effect cleanup and resource lifetime	116
5.11	React Strict Mode	118
5.12	Refs	120
5.13	Context	122
5.14	Component identity and keys	124
5.15	Controlled and uncontrolled forms	126
5.16	State ownership and lifting state	128
5.17	React.memo, useMemo, and useCallback	130
5.18	Suspense and error boundaries	132
5.19	Transitions and concurrent rendering	134
5.20	React DevTools Profiler	136
5.21	MUI Dialog, Modal, portals, and focus	138
5.22	Responsive hooks and resize behavior	140
5.23	React performance patterns	142
6	VI. Next.js App Router and Server Boundaries	144
6.1	App Router file-system routing	145
6.2	Server Components versus Client Components	147
6.3	React Server Components payload	149
6.4	SSR and hydration	151
6.5	Server-side data fetching	153
6.6	Client-side data fetching	155
6.7	Route Handlers	157
6.8	Server Actions	159
6.9	proxy.ts	161
6.10	Reading headers and cookies on the server	163
6.11	Forwarding headers to FastAPI	165
6.12	Backend-for-Frontend architecture	167
6.13	Authentication in Next.js	169
6.14	Next.js caching and revalidation	171
6.15	Loading, error, and not-found boundaries	173
6.16	Streaming with Suspense	175
6.17	Link, navigation, and prefetching	177
6.18	Environment variables and server-only code	179
6.19	Images, fonts, scripts, and bundle size	181
6.20	Development versus production behavior	183
6.21	Next.js production performance	185
7	VII. FastAPI and Python Service Internals	187
7.1	ASGI and Uvicorn	188

7.2	FastAPI routing	190
7.3	Pydantic validation	192
7.4	Dependency injection	194
7.5	Middleware and middleware order	196
7.6	async def, def, and concurrency	198
7.7	Database session lifetime	200
7.8	Repository and service layers	202
7.9	FastAPI CORS	204
7.10	Authentication and Security dependencies	206
7.11	Exception handling and error responses	208
7.12	BackgroundTasks	210
7.13	FastAPI WebSockets	212
7.14	Uploads, downloads, and streaming	214
7.15	OpenAPI and API documentation	216
7.16	Testing FastAPI	218
7.17	FastAPI performance and workers	220
8	VIII. API and Data Architecture	222
8.1	REST resource modeling	223
8.2	Pagination	225
8.3	Search, filter, and sort	227
8.4	Database transactions	229
8.5	Locks and optimistic concurrency	231
8.6	Indexes and query plans	233
8.7	The N+1 query problem	235
8.8	API contracts and versioning	237
8.9	Validation and error envelopes	239
8.10	Rate limiting	241
8.11	Queues and background workers	243
8.12	Redis and application caching	245
8.13	Consistency and duplicate execution	247
9	IX. Production Infrastructure	249
9.1	Reverse proxies and Nginx	250
9.2	Load balancers	252
9.3	API gateways	254
9.4	Docker and containers	256
9.5	Environment configuration and secrets	258
9.6	Health checks and readiness	260
9.7	Deployments and rollbacks	262
9.8	CDNs and static assets	264
9.9	Horizontal scaling and statelessness	266
10	X. Security for Production Web Applications	268
10.1	Threat modeling	269
10.2	Broken access control	271
10.3	XSS	273
10.4	Content Security Policy	275
10.5	CSRF	277
10.6	SQL and NoSQL injection	279
10.7	Authentication failures	281
10.8	Multi-tenant isolation	283
10.9	Security headers and HTTPS	285
11	XI. Observability, Testing, and Engineering Quality	287
11.1	Structured logging	288
11.2	Correlation and request IDs	290
11.3	Metrics	292
11.4	Distributed tracing and OpenTelemetry	294
11.5	Frontend error monitoring	296
11.6	Unit, integration, component, and E2E tests	298
11.7	React Testing Library	300

11.8 Vitest	302
11.9 Playwright	304
11.10API contract and load testing	306
11.11Code review for web systems	308
12 XII. Failure Case Studies and Debugging Playbooks	310
12.1 Case study: Axios POST keeps firing	311
12.2 Case study: opening an MUI Dialog re-renders the page	313
12.3 Case study: screenshot or viewport change triggers work	315
12.4 Case study: Server Component request misses headers	317
12.5 Case study: client fetch works through proxy but server fetch does not	319
12.6 Case study: CORS fails only in the browser	321
12.7 Case study: cookie is not sent	323
12.8 Case study: 401 versus 403	325
12.9 Case study: 404, 405, and 422	327
12.10Case study: hydration mismatch	329
12.11Case study: stale data after mutation	331
12.12Case study: slow page caused by a request waterfall	333
12.13Case study: WebSocket disconnects in production	335
12.14Case study: form submits twice	337
12.15Case study: memory leak from listeners, timers, or subscriptions	339
12.16Case study: works in development, fails in production	341
12.17Case study: tracing one request from browser to database	343
A Quick decision tables	345
B Production review checklist	346
C Primary references	347
D Glossary	348
E Final mental model	349

Part 1

I. First-Principles Web Thinking

A production web feature is a chain of executions, not one program. Start by naming the trigger, the process that runs the code, the state it can see, and the next boundary it crosses. Each boundary creates new failure modes and often a new request. If you cannot draw the path, you are not ready to debug the symptom.

Part flow: Trigger -> executing code -> state/side effect -> boundary -> downstream work -> observable result

1.1 The request-first debugging model

Why this matters. Learn to reason from triggers, execution environment, network boundaries, state changes, and observable evidence instead of memorizing framework recipes.

Mental model. A production web feature is a chain of executions, not one program. Start by naming the trigger, the process that runs the code, the state it can see, and the next boundary it crosses. Each boundary creates new failure modes and often a new request. If you cannot draw the path, you are not ready to debug the symptom.

Core rules

- Name the trigger before reading code: navigation, click, submit, state update, timer, server request, message, or deployment event.
- Write down the execution environment: browser, Next.js server, FastAPI process, worker, database, or edge proxy.
- Separate pure calculations from side effects. Side effects need lifecycle, error handling, and often cleanup.
- Prefer observable proof from Network, Profiler, logs, traces, or tests over assumptions about framework behavior.

Request / execution flow

```
Trigger -> executing code -> state/side effect -> boundary -> downstream work -> observable result
```

Under the hood

For the request-first debugging model, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'The request-first debugging model' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: The request-first debugging model

Observe -> explain -> change one variable -> observe again

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Changing code before reproducing the symptom reliably.
- Assuming the request path instead of drawing it.
- Using console output as the only source of truth.

Production checklist

- Can I reproduce it?
- Can I draw every execution boundary?
- What evidence would falsify my current hypothesis?
- What regression check will remain after the fix?

Questions a strong reviewer may ask

- What exactly triggers this code?
- Which process owns this state?
- What happens if this runs twice?

Hands-on lab

Draw the full execution diagram for 'The request-first debugging model' in your current project. Mark each side effect and each place you can inspect evidence.

1.2 Client, server, process, thread, and network boundaries

Why this matters. Build a precise map of where code executes and what must cross a boundary explicitly.

Mental model. A production web feature is a chain of executions, not one program. Start by naming the trigger, the process that runs the code, the state it can see, and the next boundary it crosses. Each boundary creates new failure modes and often a new request. If you cannot draw the path, you are not ready to debug the symptom.

Core rules

- Name the trigger before reading code: navigation, click, submit, state update, timer, server request, message, or deployment event.
- Write down the execution environment: browser, Next.js server, FastAPI process, worker, database, or edge proxy.
- Separate pure calculations from side effects. Side effects need lifecycle, error handling, and often cleanup.
- Prefer observable proof from Network, Profiler, logs, traces, or tests over assumptions about framework behavior.

Request / execution flow

Trigger -> executing code -> state/side effect -> boundary -> downstream work -> observable result

Under the hood

For client, server, process, thread, and network boundaries, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Client, server, process, thread, and network boundaries' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Client, server, process, thread, and network boundaries

Observe -> explain -> change one variable -> observe again

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Changing code before reproducing the symptom reliably.
- Assuming the request path instead of drawing it.
- Using console output as the only source of truth.

Production checklist

- Can I reproduce it?
- Can I draw every execution boundary?
- What evidence would falsify my current hypothesis?
- What regression check will remain after the fix?

Questions a strong reviewer may ask

- What exactly triggers this code?
- Which process owns this state?
- What happens if this runs twice?

Hands-on lab

Draw the full execution diagram for 'Client, server, process, thread, and network boundaries' in your current project. Mark each side effect and each place you can inspect evidence.

1.3 Pure computation versus side effects

Why this matters. Separate calculations from operations that change the outside world: requests, storage, timers, subscriptions, logs, DOM mutation, and navigation.

Mental model. A production web feature is a chain of executions, not one program. Start by naming the trigger, the process that runs the code, the state it can see, and the next boundary it crosses. Each boundary creates new failure modes and often a new request. If you cannot draw the path, you are not ready to debug the symptom.

Core rules

- Name the trigger before reading code: navigation, click, submit, state update, timer, server request, message, or deployment event.
- Write down the execution environment: browser, Next.js server, FastAPI process, worker, database, or edge proxy.
- Separate pure calculations from side effects. Side effects need lifecycle, error handling, and often cleanup.
- Prefer observable proof from Network, Profiler, logs, traces, or tests over assumptions about framework behavior.

Request / execution flow

```
Trigger -> executing code -> state/side effect -> boundary -> downstream work -> observable result
```

Under the hood

For pure computation versus side effects, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Pure computation versus side effects' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Pure computation versus side effects

Observe -> explain -> change one variable -> observe again

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Changing code before reproducing the symptom reliably.
- Assuming the request path instead of drawing it.
- Using console output as the only source of truth.

Production checklist

- Can I reproduce it?
- Can I draw every execution boundary?
- What evidence would falsify my current hypothesis?
- What regression check will remain after the fix?

Questions a strong reviewer may ask

- What exactly triggers this code?
- Which process owns this state?
- What happens if this runs twice?

Hands-on lab

Draw the full execution diagram for 'Pure computation versus side effects' in your current project. Mark each side effect and each place you can inspect evidence.

1.4 A production debugging workflow

Why this matters. Use reproduction, observation, narrowing, instrumentation, and falsifiable hypotheses instead of random edits.

Mental model. A production web feature is a chain of executions, not one program. Start by naming the trigger, the process that runs the code, the state it can see, and the next boundary it crosses. Each boundary creates new failure modes and often a new request. If you cannot draw the path, you are not ready to debug the symptom.

Core rules

- Name the trigger before reading code: navigation, click, submit, state update, timer, server request, message, or deployment event.
- Write down the execution environment: browser, Next.js server, FastAPI process, worker, database, or edge proxy.
- Separate pure calculations from side effects. Side effects need lifecycle, error handling, and often cleanup.
- Prefer observable proof from Network, Profiler, logs, traces, or tests over assumptions about framework behavior.

Request / execution flow

```
Trigger -> executing code -> state/side effect -> boundary -> downstream work -> observable result
```

Under the hood

For a production debugging workflow, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'A production debugging workflow' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: A production debugging workflow

Observe -> explain -> change one variable -> observe again

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Changing code before reproducing the symptom reliably.
- Assuming the request path instead of drawing it.
- Using console output as the only source of truth.

Production checklist

- Can I reproduce it?
- Can I draw every execution boundary?
- What evidence would falsify my current hypothesis?
- What regression check will remain after the fix?

Questions a strong reviewer may ask

- What exactly triggers this code?
- Which process owns this state?
- What happens if this runs twice?

Hands-on lab

Draw the full execution diagram for 'A production debugging workflow' in your current project. Mark each side effect and each place you can inspect evidence.

Part 2

II. URLs, Networking, and HTTP

HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Part flow: Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client

2.1 URL, URI, origin, path, query, and fragment

Why this matters. Understand exactly what each part of an address means and which parts participate in routing, caching, and browser security.

Mental model. HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Core rules

- A new outbound call is a new request. Headers and cookies are not automatically inherited from a previous request unless the client or proxy forwards them.
- Method semantics matter for caching, retries, preflight behavior, and reviewer expectations.
- Use the Network panel or curl to inspect the actual bytes/metadata rather than assuming what a library sent.
- Keep authentication, authorization, caching, and content negotiation explicit at the boundary that owns them.
- Origin is scheme + host + port. Path/query do not change the origin.
- Fragments are handled client-side and are not sent as part of the HTTP request target in normal navigation.
- Query parameters are visible in URLs, logs, history, and often caches; do not put secrets there.

Request / execution flow

```
Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client
```

Under the hood

For url, uri, origin, path, query, and fragment, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'URL, URI, origin, path, query, and fragment' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: URL, URI, origin, path, query, and fragment

```
curl -i -v https://example.com/api/users \
-H 'Accept: application/json' \
-H 'X-Request-ID: demo-123'
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Treating a custom header as trusted identity.
- Ignoring method semantics and retry behavior.
- Debugging server code when DNS/TLS/CORS prevented the expected path.

Production checklist

- Method and URL correct?
- Actual request/response headers inspected?
- Credential behavior explicit?
- Timeout/retry/cache semantics understood?

Questions a strong reviewer may ask

- Who created this request?
- Which headers are trusted and why?
- What happens on retry, timeout, or cache hit?

Hands-on lab

Use `curl -v` and the browser Network panel to exercise 'URL, URI, origin, path, query, and fragment'. Capture request headers, response headers, and one deliberate failure.

2.2 DNS resolution

Why this matters. Follow a hostname from browser caches through resolvers to an IP address and understand TTLs and failure modes.

Mental model. HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Core rules

- A new outbound call is a new request. Headers and cookies are not automatically inherited from a previous request unless the client or proxy forwards them.
- Method semantics matter for caching, retries, preflight behavior, and reviewer expectations.
- Use the Network panel or curl to inspect the actual bytes/metadata rather than assuming what a library sent.
- Keep authentication, authorization, caching, and content negotiation explicit at the boundary that owns them.
- DNS maps names to records such as A/AAAA/CNAME; resolvers cache answers according to TTL.
- A DNS failure can occur before any TCP/TLS/HTTP activity, so application logs may show nothing.

Request / execution flow

```
Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client
```

Under the hood

For dns resolution, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'DNS resolution' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: DNS resolution

```
curl -i -v https://example.com/api/users \
-H 'Accept: application/json' \
-H 'X-Request-ID: demo-123'
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Treating a custom header as trusted identity.
- Ignoring method semantics and retry behavior.
- Debugging server code when DNS/TLS/CORS prevented the expected path.

Production checklist

- Method and URL correct?
- Actual request/response headers inspected?
- Credential behavior explicit?
- Timeout/retry/cache semantics understood?

Questions a strong reviewer may ask

- Who created this request?
- Which headers are trusted and why?
- What happens on retry, timeout, or cache hit?

Hands-on lab

Use `curl -v` and the browser Network panel to exercise 'DNS resolution'. Capture request headers, response headers, and one deliberate failure.

2.3 TCP, sockets, ports, and connection state

Why this matters. Understand reliable byte streams, ports, connection setup, and why an application socket is not the same thing as an HTTP request.

Mental model. HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Core rules

- A new outbound call is a new request. Headers and cookies are not automatically inherited from a previous request unless the client or proxy forwards them.
- Method semantics matter for caching, retries, preflight behavior, and reviewer expectations.
- Use the Network panel or curl to inspect the actual bytes/metadata rather than assuming what a library sent.
- Keep authentication, authorization, caching, and content negotiation explicit at the boundary that owns them.
- TCP provides an ordered reliable byte stream; your application must impose message boundaries such as HTTP framing.
- Ports identify transport endpoints on a host; NAT/proxies may rewrite connection endpoints.

Request / execution flow

```
Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client
```

Under the hood

For tcp, sockets, ports, and connection state, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'TCP, sockets, ports, and connection state' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: TCP, sockets, ports, and connection state

```
curl -i -v https://example.com/api/users \
-H 'Accept: application/json' \
-H 'X-Request-ID: demo-123'
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Treating a custom header as trusted identity.
- Ignoring method semantics and retry behavior.
- Debugging server code when DNS/TLS/CORS prevented the expected path.

Production checklist

- Method and URL correct?
- Actual request/response headers inspected?
- Credential behavior explicit?
- Timeout/retry/cache semantics understood?

Questions a strong reviewer may ask

- Who created this request?
- Which headers are trusted and why?
- What happens on retry, timeout, or cache hit?

Hands-on lab

Use `curl -v` and the browser Network panel to exercise 'TCP, sockets, ports, and connection state'. Capture request headers, response headers, and one deliberate failure.

2.4 QUIC and HTTP/3

Why this matters. Understand why modern browsers may use UDP-based QUIC and how transport changes without changing most application semantics.

Mental model. HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Core rules

- A new outbound call is a new request. Headers and cookies are not automatically inherited from a previous request unless the client or proxy forwards them.
- Method semantics matter for caching, retries, preflight behavior, and reviewer expectations.
- Use the Network panel or curl to inspect the actual bytes/metadata rather than assuming what a library sent.
- Keep authentication, authorization, caching, and content negotiation explicit at the boundary that owns them.

Request / execution flow

```
Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client
```

Under the hood

For quic and http/3, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'QUIC and HTTP/3' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: QUIC and HTTP/3

```
curl -i -v https://example.com/api/users \  
-H 'Accept: application/json' \  
-H 'X-Request-ID: demo-123'
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Treating a custom header as trusted identity.
- Ignoring method semantics and retry behavior.
- Debugging server code when DNS/TLS/CORS prevented the expected path.

Production checklist

- Method and URL correct?
- Actual request/response headers inspected?
- Credential behavior explicit?
- Timeout/retry/cache semantics understood?

Questions a strong reviewer may ask

- Who created this request?
- Which headers are trusted and why?
- What happens on retry, timeout, or cache hit?

Hands-on lab

Use `curl -v` and the browser Network panel to exercise 'QUIC and HTTP/3'. Capture request headers, response headers, and one deliberate failure.

2.5 TLS and HTTPS

Why this matters. Understand certificates, trust chains, encryption, TLS termination, and why HTTPS protects data in transit rather than fixing application authorization.

Mental model. HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Core rules

- A new outbound call is a new request. Headers and cookies are not automatically inherited from a previous request unless the client or proxy forwards them.
- Method semantics matter for caching, retries, preflight behavior, and reviewer expectations.
- Use the Network panel or curl to inspect the actual bytes/metadata rather than assuming what a library sent.
- Keep authentication, authorization, caching, and content negotiation explicit at the boundary that owns them.
- The server proves a certificate chain trusted for the requested hostname; key agreement establishes session secrets.
- TLS termination may happen at a load balancer, after which internal traffic may be re-encrypted or remain inside a trusted network.

Request / execution flow

```
Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client
```

Under the hood

For tls and https, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'TLS and HTTPS' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: TLS and HTTPS

```
curl -i -v https://example.com/api/users \  
-H 'Accept: application/json' \  
-H 'X-Request-ID: demo-123'
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Treating a custom header as trusted identity.
- Ignoring method semantics and retry behavior.
- Debugging server code when DNS/TLS/CORS prevented the expected path.

Production checklist

- Method and URL correct?
- Actual request/response headers inspected?
- Credential behavior explicit?
- Timeout/retry/cache semantics understood?

Questions a strong reviewer may ask

- Who created this request?
- Which headers are trusted and why?
- What happens on retry, timeout, or cache hit?

Hands-on lab

Use `curl -v` and the browser Network panel to exercise 'TLS and HTTPS'. Capture request headers, response headers, and one deliberate failure.

2.6 HTTP request anatomy

Why this matters. Read a request line, URL, headers, body, and connection metadata as concrete bytes and semantics.

Mental model. HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Core rules

- A new outbound call is a new request. Headers and cookies are not automatically inherited from a previous request unless the client or proxy forwards them.
- Method semantics matter for caching, retries, preflight behavior, and reviewer expectations.
- Use the Network panel or curl to inspect the actual bytes/metadata rather than assuming what a library sent.
- Keep authentication, authorization, caching, and content negotiation explicit at the boundary that owns them.
- Request headers are metadata, not trusted identity by themselves.
- Bodies can be streamed; Content-Length or chunked framing describes transfer details depending on protocol/version.

Request / execution flow

```
Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client
```

Under the hood

For http request anatomy, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'HTTP request anatomy' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: HTTP request anatomy

```
POST /api/users?invite=true HTTP/1.1
Host: app.example.com
Authorization: Bearer <token>
Content-Type: application/json
X-Request-ID: 8e3b...

{"name":"Ada","role":"admin"}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Treating a custom header as trusted identity.
- Ignoring method semantics and retry behavior.
- Debugging server code when DNS/TLS/CORS prevented the expected path.

Production checklist

- Method and URL correct?
- Actual request/response headers inspected?
- Credential behavior explicit?
- Timeout/retry/cache semantics understood?

Questions a strong reviewer may ask

- Who created this request?
- Which headers are trusted and why?
- What happens on retry, timeout, or cache hit?

Hands-on lab

Use `curl -v` and the browser Network panel to exercise 'HTTP request anatomy'. Capture request headers, response headers, and one deliberate failure.

2.7 HTTP response anatomy

Why this matters. Understand status, headers, body, caching instructions, content negotiation, and streaming responses.

Mental model. HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Core rules

- A new outbound call is a new request. Headers and cookies are not automatically inherited from a previous request unless the client or proxy forwards them.
- Method semantics matter for caching, retries, preflight behavior, and reviewer expectations.
- Use the Network panel or curl to inspect the actual bytes/metadata rather than assuming what a library sent.
- Keep authentication, authorization, caching, and content negotiation explicit at the boundary that owns them.

Request / execution flow

```
Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client
```

Under the hood

For http response anatomy, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'HTTP response anatomy' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: HTTP response anatomy

```
curl -i -v https://example.com/api/users \
-H 'Accept: application/json' \
-H 'X-Request-ID: demo-123'
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Treating a custom header as trusted identity.
- Ignoring method semantics and retry behavior.
- Debugging server code when DNS/TLS/CORS prevented the expected path.

Production checklist

- Method and URL correct?
- Actual request/response headers inspected?
- Credential behavior explicit?
- Timeout/retry/cache semantics understood?

Questions a strong reviewer may ask

- Who created this request?
- Which headers are trusted and why?
- What happens on retry, timeout, or cache hit?

Hands-on lab

Use `curl -v` and the browser Network panel to exercise 'HTTP response anatomy'. Capture request headers, response headers, and one deliberate failure.

2.8 HTTP methods and idempotency

Why this matters. Choose GET, POST, PUT, PATCH, DELETE, OPTIONS, and HEAD according to semantics and retry safety.

Mental model. HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Core rules

- A new outbound call is a new request. Headers and cookies are not automatically inherited from a previous request unless the client or proxy forwards them.
- Method semantics matter for caching, retries, preflight behavior, and reviewer expectations.
- Use the Network panel or curl to inspect the actual bytes/metadata rather than assuming what a library sent.
- Keep authentication, authorization, caching, and content negotiation explicit at the boundary that owns them.
- GET and HEAD should be safe reads; PUT is normally idempotent replacement; POST commonly creates or triggers non-idempotent work.
- Retry policy must consider whether duplicate execution changes state.

Request / execution flow

```
Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client
```

Under the hood

For http methods and idempotency, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'HTTP methods and idempotency' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: HTTP methods and idempotency

```
curl -i -v https://example.com/api/users \
-H 'Accept: application/json' \
-H 'X-Request-ID: demo-123'
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Treating a custom header as trusted identity.
- Ignoring method semantics and retry behavior.
- Debugging server code when DNS/TLS/CORS prevented the expected path.

Production checklist

- Method and URL correct?
- Actual request/response headers inspected?
- Credential behavior explicit?
- Timeout/retry/cache semantics understood?

Questions a strong reviewer may ask

- Who created this request?
- Which headers are trusted and why?
- What happens on retry, timeout, or cache hit?

Hands-on lab

Use `curl -v` and the browser Network panel to exercise 'HTTP methods and idempotency'. Capture request headers, response headers, and one deliberate failure.

2.9 HTTP status codes

Why this matters. Interpret 2xx, 3xx, 4xx, and 5xx codes and distinguish application errors from transport failures.

Mental model. HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Core rules

- A new outbound call is a new request. Headers and cookies are not automatically inherited from a previous request unless the client or proxy forwards them.
- Method semantics matter for caching, retries, preflight behavior, and reviewer expectations.
- Use the Network panel or curl to inspect the actual bytes/metadata rather than assuming what a library sent.
- Keep authentication, authorization, caching, and content negotiation explicit at the boundary that owns them.
- 401 means authentication is required/invalid; 403 means the server understood the identity but refuses the action.
- 404 can deliberately hide resource existence in security-sensitive APIs; 405 means the route exists but not for that method; 422 commonly signals validation failure in FastAPI.

Request / execution flow

```
Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client
```

Under the hood

For http status codes, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'HTTP status codes' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: HTTP status codes

```
curl -i -v https://example.com/api/users \
-H 'Accept: application/json' \
-H 'X-Request-ID: demo-123'
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Treating a custom header as trusted identity.
- Ignoring method semantics and retry behavior.
- Debugging server code when DNS/TLS/CORS prevented the expected path.

Production checklist

- Method and URL correct?
- Actual request/response headers inspected?
- Credential behavior explicit?
- Timeout/retry/cache semantics understood?

Questions a strong reviewer may ask

- Who created this request?
- Which headers are trusted and why?
- What happens on retry, timeout, or cache hit?

Hands-on lab

Use `curl -v` and the browser Network panel to exercise 'HTTP status codes'. Capture request headers, response headers, and one deliberate failure.

2.10 HTTP headers

Why this matters. Understand request headers, response headers, hop-by-hop versus end-to-end concepts, forwarded headers, and custom application context.

Mental model. HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Core rules

- A new outbound call is a new request. Headers and cookies are not automatically inherited from a previous request unless the client or proxy forwards them.
- Method semantics matter for caching, retries, preflight behavior, and reviewer expectations.
- Use the Network panel or curl to inspect the actual bytes/metadata rather than assuming what a library sent.
- Keep authentication, authorization, caching, and content negotiation explicit at the boundary that owns them.

Request / execution flow

```
Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client
```

Under the hood

For http headers, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'HTTP headers' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: HTTP headers

```
curl -i -v https://example.com/api/users \  
-H 'Accept: application/json' \  
-H 'X-Request-ID: demo-123'
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Treating a custom header as trusted identity.
- Ignoring method semantics and retry behavior.
- Debugging server code when DNS/TLS/CORS prevented the expected path.

Production checklist

- Method and URL correct?
- Actual request/response headers inspected?
- Credential behavior explicit?
- Timeout/retry/cache semantics understood?

Questions a strong reviewer may ask

- Who created this request?
- Which headers are trusted and why?
- What happens on retry, timeout, or cache hit?

Hands-on lab

Use `curl -v` and the browser Network panel to exercise 'HTTP headers'. Capture request headers, response headers, and one deliberate failure.

2.11 Content-Type, Accept, encoding, and bodies

Why this matters. Know how JSON, form data, multipart uploads, text, binary content, and compression are described and parsed.

Mental model. HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Core rules

- A new outbound call is a new request. Headers and cookies are not automatically inherited from a previous request unless the client or proxy forwards them.
- Method semantics matter for caching, retries, preflight behavior, and reviewer expectations.
- Use the Network panel or curl to inspect the actual bytes/metadata rather than assuming what a library sent.
- Keep authentication, authorization, caching, and content negotiation explicit at the boundary that owns them.

Request / execution flow

```
Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client
```

Under the hood

For content-type, accept, encoding, and bodies, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Content-Type, Accept, encoding, and bodies' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Content-Type, Accept, encoding, and bodies

```
curl -i -v https://example.com/api/users \
-H 'Accept: application/json' \
-H 'X-Request-ID: demo-123'
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Treating a custom header as trusted identity.
- Ignoring method semantics and retry behavior.
- Debugging server code when DNS/TLS/CORS prevented the expected path.

Production checklist

- Method and URL correct?
- Actual request/response headers inspected?
- Credential behavior explicit?
- Timeout/retry/cache semantics understood?

Questions a strong reviewer may ask

- Who created this request?
- Which headers are trusted and why?
- What happens on retry, timeout, or cache hit?

Hands-on lab

Use `curl -v` and the browser Network panel to exercise 'Content-Type, Accept, encoding, and bodies'. Capture request headers, response headers, and one deliberate failure.

2.12 Cookies and browser-managed credentials

Why this matters. Understand Set-Cookie, Cookie, Domain, Path, Expires, Max-Age, Secure, HttpOnly, and SameSite.

Mental model. HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Core rules

- A new outbound call is a new request. Headers and cookies are not automatically inherited from a previous request unless the client or proxy forwards them.
- Method semantics matter for caching, retries, preflight behavior, and reviewer expectations.
- Use the Network panel or curl to inspect the actual bytes/metadata rather than assuming what a library sent.
- Keep authentication, authorization, caching, and content negotiation explicit at the boundary that owns them.
- HttpOnly prevents JavaScript access but does not prevent the browser from attaching the cookie to eligible requests.
- SameSite changes whether cookies accompany cross-site requests; Secure requires HTTPS; Domain/Path scope eligibility.

Request / execution flow

```
Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client
```

Under the hood

For cookies and browser-managed credentials, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Cookies and browser-managed credentials' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Cookies and browser-managed credentials

```
HTTP/1.1 200 OK
Set-Cookie: session=abc123; Path=/; HttpOnly; Secure; SameSite=Lax
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Treating a custom header as trusted identity.
- Ignoring method semantics and retry behavior.
- Debugging server code when DNS/TLS/CORS prevented the expected path.

Production checklist

- Method and URL correct?
- Actual request/response headers inspected?
- Credential behavior explicit?
- Timeout/retry/cache semantics understood?

Questions a strong reviewer may ask

- Who created this request?
- Which headers are trusted and why?
- What happens on retry, timeout, or cache hit?

Hands-on lab

Use `curl -v` and the browser Network panel to exercise 'Cookies and browser-managed credentials'. Capture request headers, response headers, and one deliberate failure.

2.13 Sessions, access tokens, and refresh tokens

Why this matters. Understand stateful sessions versus bearer tokens, token lifetime, rotation, revocation, and where credentials should live.

Mental model. HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Core rules

- A new outbound call is a new request. Headers and cookies are not automatically inherited from a previous request unless the client or proxy forwards them.
- Method semantics matter for caching, retries, preflight behavior, and reviewer expectations.
- Use the Network panel or curl to inspect the actual bytes/metadata rather than assuming what a library sent.
- Keep authentication, authorization, caching, and content negotiation explicit at the boundary that owns them.

Request / execution flow

```
Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client
```

Under the hood

For sessions, access tokens, and refresh tokens, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Sessions, access tokens, and refresh tokens' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Sessions, access tokens, and refresh tokens

```
curl -i -v https://example.com/api/users \
-H 'Accept: application/json' \
-H 'X-Request-ID: demo-123'
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Treating a custom header as trusted identity.
- Ignoring method semantics and retry behavior.
- Debugging server code when DNS/TLS/CORS prevented the expected path.

Production checklist

- Method and URL correct?
- Actual request/response headers inspected?
- Credential behavior explicit?
- Timeout/retry/cache semantics understood?

Questions a strong reviewer may ask

- Who created this request?
- Which headers are trusted and why?
- What happens on retry, timeout, or cache hit?

Hands-on lab

Use `curl -v` and the browser Network panel to exercise 'Sessions, access tokens, and refresh tokens'. Capture request headers, response headers, and one deliberate failure.

2.14 HTTP caching and validators

Why this matters. Understand Cache-Control, freshness, ETag, If-None-Match, Last-Modified, and 304 responses.

Mental model. HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Core rules

- A new outbound call is a new request. Headers and cookies are not automatically inherited from a previous request unless the client or proxy forwards them.
- Method semantics matter for caching, retries, preflight behavior, and reviewer expectations.
- Use the Network panel or curl to inspect the actual bytes/metadata rather than assuming what a library sent.
- Keep authentication, authorization, caching, and content negotiation explicit at the boundary that owns them.

Request / execution flow

```
Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client
```

Under the hood

For http caching and validators, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'HTTP caching and validators' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: HTTP caching and validators

```
curl -i -v https://example.com/api/users \  
-H 'Accept: application/json' \  
-H 'X-Request-ID: demo-123'
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Treating a custom header as trusted identity.
- Ignoring method semantics and retry behavior.
- Debugging server code when DNS/TLS/CORS prevented the expected path.

Production checklist

- Method and URL correct?
- Actual request/response headers inspected?
- Credential behavior explicit?
- Timeout/retry/cache semantics understood?

Questions a strong reviewer may ask

- Who created this request?
- Which headers are trusted and why?
- What happens on retry, timeout, or cache hit?

Hands-on lab

Use `curl -v` and the browser Network panel to exercise 'HTTP caching and validators'. Capture request headers, response headers, and one deliberate failure.

2.15 Same-origin policy and CORS

Why this matters. Understand origin isolation, preflight, credentialed requests, allowed headers, and why server-to-server calls are different.

Mental model. HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Core rules

- A new outbound call is a new request. Headers and cookies are not automatically inherited from a previous request unless the client or proxy forwards them.
- Method semantics matter for caching, retries, preflight behavior, and reviewer expectations.
- Use the Network panel or curl to inspect the actual bytes/metadata rather than assuming what a library sent.
- Keep authentication, authorization, caching, and content negotiation explicit at the boundary that owns them.
- Origin is scheme + host + port. Path/query do not change the origin.
- Fragments are handled client-side and are not sent as part of the HTTP request target in normal navigation.
- Query parameters are visible in URLs, logs, history, and often caches; do not put secrets there.
- CORS does not stop a request from reaching the server in all cases; it primarily controls whether browser JavaScript may access the response.

Request / execution flow

```
Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client
```

Under the hood

For same-origin policy and cors, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Same-origin policy and CORS' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Same-origin policy and CORS

```
curl -i -v https://example.com/api/users \  
-H 'Accept: application/json' \  
-H 'X-Request-ID: demo-123'
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Treating a custom header as trusted identity.
- Ignoring method semantics and retry behavior.
- Debugging server code when DNS/TLS/CORS prevented the expected path.

Production checklist

- Method and URL correct?
- Actual request/response headers inspected?
- Credential behavior explicit?
- Timeout/retry/cache semantics understood?

Questions a strong reviewer may ask

- Who created this request?
- Which headers are trusted and why?
- What happens on retry, timeout, or cache hit?

Hands-on lab

Use `curl -v` and the browser Network panel to exercise 'Same-origin policy and CORS'. Capture request headers, response headers, and one deliberate failure.

2.16 Browser storage

Why this matters. Use localStorage, sessionStorage, IndexedDB, and cookies with the correct persistence and security expectations.

Mental model. HTTP is message passing between endpoints. A request has a method, target, headers, and possibly a body; a response has a status, headers, and possibly a body. Frameworks add conveniences, but they cannot remove network boundaries. Treat each request as an independent message and verify what actually crossed the wire.

Core rules

- A new outbound call is a new request. Headers and cookies are not automatically inherited from a previous request unless the client or proxy forwards them.
- Method semantics matter for caching, retries, preflight behavior, and reviewer expectations.
- Use the Network panel or curl to inspect the actual bytes/metadata rather than assuming what a library sent.
- Keep authentication, authorization, caching, and content negotiation explicit at the boundary that owns them.

Request / execution flow

```
Client -> DNS/TLS/connection -> HTTP request -> proxy/server -> HTTP response -> client
```

Under the hood

For browser storage, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Browser storage' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Browser storage

```
curl -i -v https://example.com/api/users \  
-H 'Accept: application/json' \  
-H 'X-Request-ID: demo-123'
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Treating a custom header as trusted identity.
- Ignoring method semantics and retry behavior.
- Debugging server code when DNS/TLS/CORS prevented the expected path.

Production checklist

- Method and URL correct?
- Actual request/response headers inspected?
- Credential behavior explicit?
- Timeout/retry/cache semantics understood?

Questions a strong reviewer may ask

- Who created this request?
- Which headers are trusted and why?
- What happens on retry, timeout, or cache hit?

Hands-on lab

Use `curl -v` and the browser Network panel to exercise 'Browser storage'. Capture request headers, response headers, and one deliberate failure.

Part 3

III. Browser Runtime and Rendering

The browser is both a network client and a rendering runtime. It parses documents, runs JavaScript, manages security policy, creates the DOM and CSSOM, computes layout, paints pixels, stores credentials, and schedules asynchronous work. Performance problems often come from confusing these subsystems with React itself.

Part flow: Network/DOM event -> JavaScript -> DOM/style changes -> style -> layout -> paint -> composite

3.1 Browser navigation lifecycle

Why this matters. Follow a navigation from URL entry through network loading, document creation, script execution, rendering, and history updates.

Mental model. The browser is both a network client and a rendering runtime. It parses documents, runs JavaScript, manages security policy, creates the DOM and CSSOM, computes layout, paints pixels, stores credentials, and schedules asynchronous work. Performance problems often come from confusing these subsystems with React itself.

Core rules

- React render, DOM commit, browser layout, browser paint, and compositing are different stages.
- A synchronous JavaScript task blocks the main thread until it returns; promises do not make CPU work parallel.
- Browser security policy applies to browser-initiated access, not arbitrary server-to-server traffic.
- Use native semantic elements and browser behavior before recreating them with JavaScript.

Request / execution flow

Network/DOM event -> JavaScript -> DOM/style changes -> style -> layout -> paint -> composite

Under the hood

For browser navigation lifecycle, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Browser navigation lifecycle' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Browser navigation lifecycle

```
performance.mark('start');  
// perform the operation  
performance.mark('end');  
performance.measure('operation', 'start', 'end');  
console.table(performance.getEntriesByName('operation'));
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Calling every visual update a React re-render.
- Blocking the main thread with expensive synchronous work.
- Recreating browser-native behavior with fragile custom JavaScript.

Production checklist

- Main-thread long tasks measured?
- Layout/paint distinguished from React rendering?
- Native semantics/accessibility preserved?
- Network and Performance panels inspected?

Questions a strong reviewer may ask

- Is this React work or browser rendering work?
- Does it block the main thread?
- What happens without JavaScript?

Hands-on lab

Create a minimal page demonstrating 'Browser navigation lifecycle', record a Performance trace, and label JavaScript, style, layout, paint, and network work.

3.2 HTML parsing and the DOM

Why this matters. Understand tokenization, DOM construction, scripts that block parsing, and DOM APIs.

Mental model. The browser is both a network client and a rendering runtime. It parses documents, runs JavaScript, manages security policy, creates the DOM and CSSOM, computes layout, paints pixels, stores credentials, and schedules asynchronous work. Performance problems often come from confusing these subsystems with React itself.

Core rules

- React render, DOM commit, browser layout, browser paint, and compositing are different stages.
- A synchronous JavaScript task blocks the main thread until it returns; promises do not make CPU work parallel.
- Browser security policy applies to browser-initiated access, not arbitrary server-to-server traffic.
- Use native semantic elements and browser behavior before recreating them with JavaScript.

Request / execution flow

Network/DOM event -> JavaScript -> DOM/style changes -> style -> layout -> paint -> composite

Under the hood

For html parsing and the dom, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'HTML parsing and the DOM' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: HTML parsing and the DOM

```
performance.mark('start');  
// perform the operation  
performance.mark('end');  
performance.measure('operation', 'start', 'end');  
console.table(performance.getEntriesByName('operation'));
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Calling every visual update a React re-render.
- Blocking the main thread with expensive synchronous work.
- Recreating browser-native behavior with fragile custom JavaScript.

Production checklist

- Main-thread long tasks measured?
- Layout/paint distinguished from React rendering?
- Native semantics/accessibility preserved?
- Network and Performance panels inspected?

Questions a strong reviewer may ask

- Is this React work or browser rendering work?
- Does it block the main thread?
- What happens without JavaScript?

Hands-on lab

Create a minimal page demonstrating 'HTML parsing and the DOM', record a Performance trace, and label JavaScript, style, layout, paint, and network work.

3.3 CSS cascade, inheritance, and CSSOM

Why this matters. Understand how the browser decides final styles before layout.

Mental model. The browser is both a network client and a rendering runtime. It parses documents, runs JavaScript, manages security policy, creates the DOM and CSSOM, computes layout, paints pixels, stores credentials, and schedules asynchronous work. Performance problems often come from confusing these subsystems with React itself.

Core rules

- React render, DOM commit, browser layout, browser paint, and compositing are different stages.
- A synchronous JavaScript task blocks the main thread until it returns; promises do not make CPU work parallel.
- Browser security policy applies to browser-initiated access, not arbitrary server-to-server traffic.
- Use native semantic elements and browser behavior before recreating them with JavaScript.

Request / execution flow

Network/DOM event -> JavaScript -> DOM/style changes -> style -> layout -> paint -> composite

Under the hood

For css cascade, inheritance, and cssom, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'CSS cascade, inheritance, and CSSOM' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: CSS cascade, inheritance, and CSSOM

```
performance.mark('start');  
// perform the operation  
performance.mark('end');  
performance.measure('operation', 'start', 'end');  
console.table(performance.getEntriesByName('operation'));
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Calling every visual update a React re-render.
- Blocking the main thread with expensive synchronous work.
- Recreating browser-native behavior with fragile custom JavaScript.

Production checklist

- Main-thread long tasks measured?
- Layout/paint distinguished from React rendering?
- Native semantics/accessibility preserved?
- Network and Performance panels inspected?

Questions a strong reviewer may ask

- Is this React work or browser rendering work?
- Does it block the main thread?
- What happens without JavaScript?

Hands-on lab

Create a minimal page demonstrating 'CSS cascade, inheritance, and CSSOM', record a Performance trace, and label JavaScript, style, layout, paint, and network work.

3.4 Style, layout, paint, and composite

Why this matters. Distinguish browser rendering stages and understand why not every visual update has the same performance cost.

Mental model. The browser is both a network client and a rendering runtime. It parses documents, runs JavaScript, manages security policy, creates the DOM and CSSOM, computes layout, paints pixels, stores credentials, and schedules asynchronous work. Performance problems often come from confusing these subsystems with React itself.

Core rules

- React render, DOM commit, browser layout, browser paint, and compositing are different stages.
- A synchronous JavaScript task blocks the main thread until it returns; promises do not make CPU work parallel.
- Browser security policy applies to browser-initiated access, not arbitrary server-to-server traffic.
- Use native semantic elements and browser behavior before recreating them with JavaScript.

Request / execution flow

Network/DOM event -> JavaScript -> DOM/style changes -> style -> layout -> paint -> composite

Under the hood

For style, layout, paint, and composite, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Style, layout, paint, and composite' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Style, layout, paint, and composite

```
performance.mark('start');  
// perform the operation  
performance.mark('end');  
performance.measure('operation', 'start', 'end');  
console.table(performance.getEntriesByName('operation'));
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Calling every visual update a React re-render.
- Blocking the main thread with expensive synchronous work.
- Recreating browser-native behavior with fragile custom JavaScript.

Production checklist

- Main-thread long tasks measured?
- Layout/paint distinguished from React rendering?
- Native semantics/accessibility preserved?
- Network and Performance panels inspected?

Questions a strong reviewer may ask

- Is this React work or browser rendering work?
- Does it block the main thread?
- What happens without JavaScript?

Hands-on lab

Create a minimal page demonstrating 'Style, layout, paint, and composite', record a Performance trace, and label JavaScript, style, layout, paint, and network work.

3.5 JavaScript engine, call stack, and heap

Why this matters. Understand synchronous execution, object allocation, garbage collection, and what a stack trace is telling you.

Mental model. The browser is both a network client and a rendering runtime. It parses documents, runs JavaScript, manages security policy, creates the DOM and CSSOM, computes layout, paints pixels, stores credentials, and schedules asynchronous work. Performance problems often come from confusing these subsystems with React itself.

Core rules

- React render, DOM commit, browser layout, browser paint, and compositing are different stages.
- A synchronous JavaScript task blocks the main thread until it returns; promises do not make CPU work parallel.
- Browser security policy applies to browser-initiated access, not arbitrary server-to-server traffic.
- Use native semantic elements and browser behavior before recreating them with JavaScript.

Request / execution flow

Network/DOM event -> JavaScript -> DOM/style changes -> style -> layout -> paint -> composite

Under the hood

For javascript engine, call stack, and heap, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'JavaScript engine, call stack, and heap' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: JavaScript engine, call stack, and heap

```
performance.mark('start');  
// perform the operation  
performance.mark('end');  
performance.measure('operation', 'start', 'end');  
console.table(performance.getEntriesByName('operation'));
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Calling every visual update a React re-render.
- Blocking the main thread with expensive synchronous work.
- Recreating browser-native behavior with fragile custom JavaScript.

Production checklist

- Main-thread long tasks measured?
- Layout/paint distinguished from React rendering?
- Native semantics/accessibility preserved?
- Network and Performance panels inspected?

Questions a strong reviewer may ask

- Is this React work or browser rendering work?
- Does it block the main thread?
- What happens without JavaScript?

Hands-on lab

Create a minimal page demonstrating 'JavaScript engine, call stack, and heap', record a Performance trace, and label JavaScript, style, layout, paint, and network work.

3.6 The event loop

Why this matters. Understand tasks, microtasks, rendering opportunities, timers, promises, and how long JavaScript blocks interaction.

Mental model. The browser is both a network client and a rendering runtime. It parses documents, runs JavaScript, manages security policy, creates the DOM and CSSOM, computes layout, paints pixels, stores credentials, and schedules asynchronous work. Performance problems often come from confusing these subsystems with React itself.

Core rules

- React render, DOM commit, browser layout, browser paint, and compositing are different stages.
- A synchronous JavaScript task blocks the main thread until it returns; promises do not make CPU work parallel.
- Browser security policy applies to browser-initiated access, not arbitrary server-to-server traffic.
- Use native semantic elements and browser behavior before recreating them with JavaScript.
- Promise callbacks run as microtasks before the browser moves to the next task, so a huge microtask chain can still starve rendering.
- `setTimeout(fn, 0)` schedules later work; it never means immediate execution.

Request / execution flow

```
Network/DOM event -> JavaScript -> DOM/style changes -> style -> layout -> paint -> composite
```

Under the hood

For the event loop, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'The event loop' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: The event loop

```
performance.mark('start');  
// perform the operation  
performance.mark('end');  
performance.measure('operation', 'start', 'end');  
console.table(performance.getEntriesByName('operation'));
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Calling every visual update a React re-render.
- Blocking the main thread with expensive synchronous work.
- Recreating browser-native behavior with fragile custom JavaScript.

Production checklist

- Main-thread long tasks measured?
- Layout/paint distinguished from React rendering?
- Native semantics/accessibility preserved?
- Network and Performance panels inspected?

Questions a strong reviewer may ask

- Is this React work or browser rendering work?
- Does it block the main thread?
- What happens without JavaScript?

Hands-on lab

Create a minimal page demonstrating 'The event loop', record a Performance trace, and label JavaScript, style, layout, paint, and network work.

3.7 DOM events and propagation

Why this matters. Understand capture, target, bubble, preventDefault, stopPropagation, passive listeners, and event delegation.

Mental model. The browser is both a network client and a rendering runtime. It parses documents, runs JavaScript, manages security policy, creates the DOM and CSSOM, computes layout, paints pixels, stores credentials, and schedules asynchronous work. Performance problems often come from confusing these subsystems with React itself.

Core rules

- React render, DOM commit, browser layout, browser paint, and compositing are different stages.
- A synchronous JavaScript task blocks the main thread until it returns; promises do not make CPU work parallel.
- Browser security policy applies to browser-initiated access, not arbitrary server-to-server traffic.
- Use native semantic elements and browser behavior before recreating them with JavaScript.

Request / execution flow

Network/DOM event -> JavaScript -> DOM/style changes -> style -> layout -> paint -> composite

Under the hood

For dom events and propagation, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'DOM events and propagation' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: DOM events and propagation

```
performance.mark('start');  
// perform the operation  
performance.mark('end');  
performance.measure('operation', 'start', 'end');  
console.table(performance.getEntriesByName('operation'));
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Calling every visual update a React re-render.
- Blocking the main thread with expensive synchronous work.
- Recreating browser-native behavior with fragile custom JavaScript.

Production checklist

- Main-thread long tasks measured?
- Layout/paint distinguished from React rendering?
- Native semantics/accessibility preserved?
- Network and Performance panels inspected?

Questions a strong reviewer may ask

- Is this React work or browser rendering work?
- Does it block the main thread?
- What happens without JavaScript?

Hands-on lab

Create a minimal page demonstrating 'DOM events and propagation', record a Performance trace, and label JavaScript, style, layout, paint, and network work.

3.8 HTML forms and native browser behavior

Why this matters. Understand submit events, button defaults, validation, FormData, Enter-key submission, and duplicate submission traps.

Mental model. The browser is both a network client and a rendering runtime. It parses documents, runs JavaScript, manages security policy, creates the DOM and CSSOM, computes layout, paints pixels, stores credentials, and schedules asynchronous work. Performance problems often come from confusing these subsystems with React itself.

Core rules

- React render, DOM commit, browser layout, browser paint, and compositing are different stages.
- A synchronous JavaScript task blocks the main thread until it returns; promises do not make CPU work parallel.
- Browser security policy applies to browser-initiated access, not arbitrary server-to-server traffic.
- Use native semantic elements and browser behavior before recreating them with JavaScript.
- A button inside a form defaults to type=submit unless specified otherwise in HTML.
- Calling preventDefault controls native navigation, but you still need duplicate-submission protection for network mutations.

Request / execution flow

```
Network/DOM event -> JavaScript -> DOM/style changes -> style -> layout -> paint -> composite
```

Under the hood

For html forms and native browser behavior, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'HTML forms and native browser behavior' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: HTML forms and native browser behavior

```
performance.mark('start');
// perform the operation
performance.mark('end');
performance.measure('operation', 'start', 'end');
console.table(performance.getEntriesByName('operation'));
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Calling every visual update a React re-render.
- Blocking the main thread with expensive synchronous work.
- Recreating browser-native behavior with fragile custom JavaScript.

Production checklist

- Main-thread long tasks measured?
- Layout/paint distinguished from React rendering?
- Native semantics/accessibility preserved?
- Network and Performance panels inspected?

Questions a strong reviewer may ask

- Is this React work or browser rendering work?
- Does it block the main thread?
- What happens without JavaScript?

Hands-on lab

Create a minimal page demonstrating 'HTML forms and native browser behavior', record a Performance trace, and label JavaScript, style, layout, paint, and network work.

3.9 History, navigation, and URL state

Why this matters. Understand `pushState`, `replaceState`, back/forward navigation, deep links, and why useful state often belongs in the URL.

Mental model. The browser is both a network client and a rendering runtime. It parses documents, runs JavaScript, manages security policy, creates the DOM and CSSOM, computes layout, paints pixels, stores credentials, and schedules asynchronous work. Performance problems often come from confusing these subsystems with React itself.

Core rules

- React render, DOM commit, browser layout, browser paint, and compositing are different stages.
- A synchronous JavaScript task blocks the main thread until it returns; promises do not make CPU work parallel.
- Browser security policy applies to browser-initiated access, not arbitrary server-to-server traffic.
- Use native semantic elements and browser behavior before recreating them with JavaScript.

Request / execution flow

Network/DOM event -> JavaScript -> DOM/style changes -> style -> layout -> paint -> composite

Under the hood

For history, navigation, and url state, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'History, navigation, and URL state' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: History, navigation, and URL state

```
performance.mark('start');  
// perform the operation  
performance.mark('end');  
performance.measure('operation', 'start', 'end');  
console.table(performance.getEntriesByName('operation'));
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Calling every visual update a React re-render.
- Blocking the main thread with expensive synchronous work.
- Recreating browser-native behavior with fragile custom JavaScript.

Production checklist

- Main-thread long tasks measured?
- Layout/paint distinguished from React rendering?
- Native semantics/accessibility preserved?
- Network and Performance panels inspected?

Questions a strong reviewer may ask

- Is this React work or browser rendering work?
- Does it block the main thread?
- What happens without JavaScript?

Hands-on lab

Create a minimal page demonstrating 'History, navigation, and URL state', record a Performance trace, and label JavaScript, style, layout, paint, and network work.

3.10 Focus, keyboard behavior, and accessibility

Why this matters. Understand focus management, semantic elements, labels, ARIA, tab order, and modal focus traps.

Mental model. The browser is both a network client and a rendering runtime. It parses documents, runs JavaScript, manages security policy, creates the DOM and CSSOM, computes layout, paints pixels, stores credentials, and schedules asynchronous work. Performance problems often come from confusing these subsystems with React itself.

Core rules

- React render, DOM commit, browser layout, browser paint, and compositing are different stages.
- A synchronous JavaScript task blocks the main thread until it returns; promises do not make CPU work parallel.
- Browser security policy applies to browser-initiated access, not arbitrary server-to-server traffic.
- Use native semantic elements and browser behavior before recreating them with JavaScript.

Request / execution flow

Network/DOM event -> JavaScript -> DOM/style changes -> style -> layout -> paint -> composite

Under the hood

For focus, keyboard behavior, and accessibility, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Focus, keyboard behavior, and accessibility' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Focus, keyboard behavior, and accessibility

```
performance.mark('start');  
// perform the operation  
performance.mark('end');  
performance.measure('operation', 'start', 'end');  
console.table(performance.getEntriesByName('operation'));
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Calling every visual update a React re-render.
- Blocking the main thread with expensive synchronous work.
- Recreating browser-native behavior with fragile custom JavaScript.

Production checklist

- Main-thread long tasks measured?
- Layout/paint distinguished from React rendering?
- Native semantics/accessibility preserved?
- Network and Performance panels inspected?

Questions a strong reviewer may ask

- Is this React work or browser rendering work?
- Does it block the main thread?
- What happens without JavaScript?

Hands-on lab

Create a minimal page demonstrating 'Focus, keyboard behavior, and accessibility', record a Performance trace, and label JavaScript, style, layout, paint, and network work.

3.11 Core Web Vitals and browser performance

Why this matters. Understand LCP, INP, CLS, network waterfalls, main-thread work, and how to measure rather than guess.

Mental model. The browser is both a network client and a rendering runtime. It parses documents, runs JavaScript, manages security policy, creates the DOM and CSSOM, computes layout, paints pixels, stores credentials, and schedules asynchronous work. Performance problems often come from confusing these subsystems with React itself.

Core rules

- React render, DOM commit, browser layout, browser paint, and compositing are different stages.
- A synchronous JavaScript task blocks the main thread until it returns; promises do not make CPU work parallel.
- Browser security policy applies to browser-initiated access, not arbitrary server-to-server traffic.
- Use native semantic elements and browser behavior before recreating them with JavaScript.

Request / execution flow

Network/DOM event -> JavaScript -> DOM/style changes -> style -> layout -> paint -> composite

Under the hood

For core web vitals and browser performance, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Core Web Vitals and browser performance' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Core Web Vitals and browser performance

```
performance.mark('start');  
// perform the operation  
performance.mark('end');  
performance.measure('operation', 'start', 'end');  
console.table(performance.getEntriesByName('operation'));
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Calling every visual update a React re-render.
- Blocking the main thread with expensive synchronous work.
- Recreating browser-native behavior with fragile custom JavaScript.

Production checklist

- Main-thread long tasks measured?
- Layout/paint distinguished from React rendering?
- Native semantics/accessibility preserved?
- Network and Performance panels inspected?

Questions a strong reviewer may ask

- Is this React work or browser rendering work?
- Does it block the main thread?
- What happens without JavaScript?

Hands-on lab

Create a minimal page demonstrating 'Core Web Vitals and browser performance', record a Performance trace, and label JavaScript, style, layout, paint, and network work.

3.12 Service workers and offline caches

Why this matters. Understand the extra programmable network layer that a service worker adds and why stale caches can become surprising.

Mental model. The browser is both a network client and a rendering runtime. It parses documents, runs JavaScript, manages security policy, creates the DOM and CSSOM, computes layout, paints pixels, stores credentials, and schedules asynchronous work. Performance problems often come from confusing these subsystems with React itself.

Core rules

- React render, DOM commit, browser layout, browser paint, and compositing are different stages.
- A synchronous JavaScript task blocks the main thread until it returns; promises do not make CPU work parallel.
- Browser security policy applies to browser-initiated access, not arbitrary server-to-server traffic.
- Use native semantic elements and browser behavior before recreating them with JavaScript.

Request / execution flow

Network/DOM event -> JavaScript -> DOM/style changes -> style -> layout -> paint -> composite

Under the hood

For service workers and offline caches, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Service workers and offline caches' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Service workers and offline caches

```
performance.mark('start');  
// perform the operation  
performance.mark('end');  
performance.measure('operation', 'start', 'end');  
console.table(performance.getEntriesByName('operation'));
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Calling every visual update a React re-render.
- Blocking the main thread with expensive synchronous work.
- Recreating browser-native behavior with fragile custom JavaScript.

Production checklist

- Main-thread long tasks measured?
- Layout/paint distinguished from React rendering?
- Native semantics/accessibility preserved?
- Network and Performance panels inspected?

Questions a strong reviewer may ask

- Is this React work or browser rendering work?
- Does it block the main thread?
- What happens without JavaScript?

Hands-on lab

Create a minimal page demonstrating 'Service workers and offline caches', record a Performance trace, and label JavaScript, style, layout, paint, and network work.

Part 4

IV. JavaScript Networking and Real-Time Communication

Network libraries do not decide your application flow. They execute when your code asks them to execute. Repeated requests are therefore a control-flow problem unless a retry layer is explicitly configured. Real-time transports add persistent connection state, which introduces lifecycle, reconnection, and backpressure concerns.

Part flow: Trigger -> API client -> request -> remote server -> response/message -> state update -> UI

4.1 The Fetch API

Why this matters. Understand Request, Response, headers, body streams, credentials, aborting, errors, and the fact that HTTP errors do not automatically reject fetch promises.

Mental model. Network libraries do not decide your application flow. They execute when your code asks them to execute. Repeated requests are therefore a control-flow problem unless a retry layer is explicitly configured. Real-time transports add persistent connection state, which introduces lifecycle, reconnection, and backpressure concerns.

Core rules

- A network call executes because your control flow reached it; libraries only retry when you configure or wrap retries.
- Cancel or ignore obsolete async work when the initiating state changes.
- Design reconnect and retry paths with idempotency and bounded backoff.
- Measure latency by phase: queueing, connection, server processing, download, and client processing.
- fetch resolves for HTTP 404/500 responses; check response.ok or status yourself.
- credentials controls whether browser credentials are included in cross-origin contexts; the server must also allow them.

Request / execution flow

```
Trigger -> API client -> request -> remote server -> response/message -> state update -> UI
```

Under the hood

For the fetch api, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'The Fetch API' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: The Fetch API

```
const controller = new AbortController();
const res = await fetch('/api/users', {
  signal: controller.signal,
  credentials: 'include',
});
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const users = await res.json();
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Starting overlapping requests without cancellation or stale-response protection.
- Retrying non-idempotent mutations blindly.
- Leaving sockets/timers alive after the owning screen unmounts.

Production checklist

- Request has cancellation?
- Stale response cannot overwrite fresh state?
- Retry is bounded and safe?
- Errors and timeouts are observable?

Questions a strong reviewer may ask

- Can this request be cancelled?
- What prevents duplicate or stale updates?
- What is the reconnect/retry policy?

Hands-on lab

Implement a tiny request/message flow for 'The Fetch API', then add delay, cancellation, failure, and duplicate triggers. Observe the result rather than guessing.

4.2 Axios

Why this matters. Understand Axios instances, defaults, transforms, interceptors, cancellation, errors, and why repeated calls normally come from your control flow rather than Axios itself.

Mental model. Network libraries do not decide your application flow. They execute when your code asks them to execute. Repeated requests are therefore a control-flow problem unless a retry layer is explicitly configured. Real-time transports add persistent connection state, which introduces lifecycle, reconnection, and backpressure concerns.

Core rules

- A network call executes because your control flow reached it; libraries only retry when you configure or wrap retries.
- Cancel or ignore obsolete async work when the initiating state changes.
- Design reconnect and retry paths with idempotency and bounded backoff.
- Measure latency by phase: queueing, connection, server processing, download, and client processing.
- Axios rejects promises for many non-2xx statuses by default, unlike fetch, but repeated calls still originate from application control flow or configured retry logic.
- Use an Axios instance for baseURL/common headers and interceptors sparingly so request behavior remains inspectable.

Request / execution flow

```
Trigger -> API client -> request -> remote server -> response/message -> state update -> UI
```

Under the hood

For axios, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Axios' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Axios

```
const api = axios.create({ baseURL: '/api', timeout: 8000 });
api.interceptors.request.use((config) => {
  config.headers['X-Request-ID'] = crypto.randomUUID();
  return config;
});

const { data } = await api.get('/users');
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Starting overlapping requests without cancellation or stale-response protection.
- Retrying non-idempotent mutations blindly.
- Leaving sockets/timers alive after the owning screen unmounts.

Production checklist

- Request has cancellation?
- Stale response cannot overwrite fresh state?
- Retry is bounded and safe?
- Errors and timeouts are observable?

Questions a strong reviewer may ask

- Can this request be cancelled?
- What prevents duplicate or stale updates?
- What is the reconnect/retry policy?

Hands-on lab

Implement a tiny request/message flow for 'Axios', then add delay, cancellation, failure, and duplicate triggers. Observe the result rather than guessing.

4.3 Promises and async/await

Why this matters. Understand promise states, chaining, await suspension, error propagation, and parallel versus sequential async work.

Mental model. Network libraries do not decide your application flow. They execute when your code asks them to execute. Repeated requests are therefore a control-flow problem unless a retry layer is explicitly configured. Real-time transports add persistent connection state, which introduces lifecycle, reconnection, and backpressure concerns.

Core rules

- A network call executes because your control flow reached it; libraries only retry when you configure or wrap retries.
- Cancel or ignore obsolete async work when the initiating state changes.
- Design reconnect and retry paths with idempotency and bounded backoff.
- Measure latency by phase: queueing, connection, server processing, download, and client processing.

Request / execution flow

```
Trigger -> API client -> request -> remote server -> response/message -> state update -> UI
```

Under the hood

For promises and async/await, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Promises and async/await' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Promises and `async/await`

```
try {
  const response = await apiCall();
  updateState(response);
} catch (error) {
  reportError(error);
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Starting overlapping requests without cancellation or stale-response protection.
- Retrying non-idempotent mutations blindly.
- Leaving sockets/timers alive after the owning screen unmounts.

Production checklist

- Request has cancellation?
- Stale response cannot overwrite fresh state?
- Retry is bounded and safe?
- Errors and timeouts are observable?

Questions a strong reviewer may ask

- Can this request be cancelled?
- What prevents duplicate or stale updates?
- What is the reconnect/retry policy?

Hands-on lab

Implement a tiny request/message flow for 'Promises and `async/await`', then add delay, cancellation, failure, and duplicate triggers. Observe the result rather than guessing.

4.4 Cancellation with AbortController

Why this matters. Cancel obsolete network work when a component unmounts, a query changes, or a new request supersedes an old one.

Mental model. Network libraries do not decide your application flow. They execute when your code asks them to execute. Repeated requests are therefore a control-flow problem unless a retry layer is explicitly configured. Real-time transports add persistent connection state, which introduces lifecycle, reconnection, and backpressure concerns.

Core rules

- A network call executes because your control flow reached it; libraries only retry when you configure or wrap retries.
- Cancel or ignore obsolete async work when the initiating state changes.
- Design reconnect and retry paths with idempotency and bounded backoff.
- Measure latency by phase: queueing, connection, server processing, download, and client processing.
- AbortController signals cooperative cancellation to fetch and compatible libraries.
- Aborting does not undo server work that has already executed; it stops waiting/reading on the client side.

Request / execution flow

```
Trigger -> API client -> request -> remote server -> response/message -> state update -> UI
```

Under the hood

For cancellation with abortcontroller, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Cancellation with AbortController' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Cancellation with AbortController

```
useEffect(() => {
  const c = new AbortController();
  fetch(`/api/users?q=${query}`, { signal: c.signal })
    .then(r => r.json())
    .then(setUsers)
    .catch(e => { if (e.name !== 'AbortError') throw e; });
  return () => c.abort();
}, [query]);
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Starting overlapping requests without cancellation or stale-response protection.
- Retrying non-idempotent mutations blindly.
- Leaving sockets/timers alive after the owning screen unmounts.

Production checklist

- Request has cancellation?
- Stale response cannot overwrite fresh state?
- Retry is bounded and safe?
- Errors and timeouts are observable?

Questions a strong reviewer may ask

- Can this request be cancelled?
- What prevents duplicate or stale updates?
- What is the reconnect/retry policy?

Hands-on lab

Implement a tiny request/message flow for 'Cancellation with AbortController', then add delay, cancellation, failure, and duplicate triggers. Observe the result rather than guessing.

4.5 Race conditions and stale responses

Why this matters. Understand how an older request can finish after a newer request and overwrite correct state.

Mental model. Network libraries do not decide your application flow. They execute when your code asks them to execute. Repeated requests are therefore a control-flow problem unless a retry layer is explicitly configured. Real-time transports add persistent connection state, which introduces lifecycle, reconnection, and backpressure concerns.

Core rules

- A network call executes because your control flow reached it; libraries only retry when you configure or wrap retries.
- Cancel or ignore obsolete async work when the initiating state changes.
- Design reconnect and retry paths with idempotency and bounded backoff.
- Measure latency by phase: queueing, connection, server processing, download, and client processing.

Request / execution flow

```
Trigger -> API client -> request -> remote server -> response/message -> state update -> UI
```

Under the hood

For race conditions and stale responses, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Race conditions and stale responses' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Race conditions and stale responses

```
try {  
  const response = await apiCall();  
  updateState(response);  
} catch (error) {  
  reportError(error);  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Starting overlapping requests without cancellation or stale-response protection.
- Retrying non-idempotent mutations blindly.
- Leaving sockets/timers alive after the owning screen unmounts.

Production checklist

- Request has cancellation?
- Stale response cannot overwrite fresh state?
- Retry is bounded and safe?
- Errors and timeouts are observable?

Questions a strong reviewer may ask

- Can this request be cancelled?
- What prevents duplicate or stale updates?
- What is the reconnect/retry policy?

Hands-on lab

Implement a tiny request/message flow for 'Race conditions and stale responses', then add delay, cancellation, failure, and duplicate triggers. Observe the result rather than guessing.

4.6 Retries, backoff, and retry safety

Why this matters. Retry only when the operation and failure mode are compatible with retry, using bounded exponential backoff and jitter.

Mental model. Network libraries do not decide your application flow. They execute when your code asks them to execute. Repeated requests are therefore a control-flow problem unless a retry layer is explicitly configured. Real-time transports add persistent connection state, which introduces lifecycle, reconnection, and backpressure concerns.

Core rules

- A network call executes because your control flow reached it; libraries only retry when you configure or wrap retries.
- Cancel or ignore obsolete async work when the initiating state changes.
- Design reconnect and retry paths with idempotency and bounded backoff.
- Measure latency by phase: queueing, connection, server processing, download, and client processing.

Request / execution flow

```
Trigger -> API client -> request -> remote server -> response/message -> state update -> UI
```

Under the hood

For retries, backoff, and retry safety, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Retries, backoff, and retry safety' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Retries, backoff, and retry safety

```
try {
  const response = await apiCall();
  updateState(response);
} catch (error) {
  reportError(error);
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Starting overlapping requests without cancellation or stale-response protection.
- Retrying non-idempotent mutations blindly.
- Leaving sockets/timers alive after the owning screen unmounts.

Production checklist

- Request has cancellation?
- Stale response cannot overwrite fresh state?
- Retry is bounded and safe?
- Errors and timeouts are observable?

Questions a strong reviewer may ask

- Can this request be cancelled?
- What prevents duplicate or stale updates?
- What is the reconnect/retry policy?

Hands-on lab

Implement a tiny request/message flow for 'Retries, backoff, and retry safety', then add delay, cancellation, failure, and duplicate triggers. Observe the result rather than guessing.

4.7 Idempotency keys

Why this matters. Protect payment-like or create-like operations from duplicate execution across retries and double submissions.

Mental model. Network libraries do not decide your application flow. They execute when your code asks them to execute. Repeated requests are therefore a control-flow problem unless a retry layer is explicitly configured. Real-time transports add persistent connection state, which introduces lifecycle, reconnection, and backpressure concerns.

Core rules

- A network call executes because your control flow reached it; libraries only retry when you configure or wrap retries.
- Cancel or ignore obsolete async work when the initiating state changes.
- Design reconnect and retry paths with idempotency and bounded backoff.
- Measure latency by phase: queueing, connection, server processing, download, and client processing.
- The server must store the key and associated operation/result long enough to recognize repeats.
- A client-generated UUID per logical submission is common; do not generate a new key for each retry of the same logical action.

Request / execution flow

```
Trigger -> API client -> request -> remote server -> response/message -> state update -> UI
```

Under the hood

For idempotency keys, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Idempotency keys' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Idempotency keys

```
try {
  const response = await apiCall();
  updateState(response);
} catch (error) {
  reportError(error);
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Starting overlapping requests without cancellation or stale-response protection.
- Retrying non-idempotent mutations blindly.
- Leaving sockets/timers alive after the owning screen unmounts.

Production checklist

- Request has cancellation?
- Stale response cannot overwrite fresh state?
- Retry is bounded and safe?
- Errors and timeouts are observable?

Questions a strong reviewer may ask

- Can this request be cancelled?
- What prevents duplicate or stale updates?
- What is the reconnect/retry policy?

Hands-on lab

Implement a tiny request/message flow for 'Idempotency keys', then add delay, cancellation, failure, and duplicate triggers. Observe the result rather than guessing.

4.8 API clients and Axios interceptors

Why this matters. Centralize base URLs, authentication, correlation IDs, and common error handling without hiding important request behavior.

Mental model. Network libraries do not decide your application flow. They execute when your code asks them to execute. Repeated requests are therefore a control-flow problem unless a retry layer is explicitly configured. Real-time transports add persistent connection state, which introduces lifecycle, reconnection, and backpressure concerns.

Core rules

- A network call executes because your control flow reached it; libraries only retry when you configure or wrap retries.
- Cancel or ignore obsolete async work when the initiating state changes.
- Design reconnect and retry paths with idempotency and bounded backoff.
- Measure latency by phase: queueing, connection, server processing, download, and client processing.
- Axios rejects promises for many non-2xx statuses by default, unlike fetch, but repeated calls still originate from application control flow or configured retry logic.
- Use an Axios instance for baseURL/common headers and interceptors sparingly so request behavior remains inspectable.

Request / execution flow

```
Trigger -> API client -> request -> remote server -> response/message -> state update -> UI
```

Under the hood

For api clients and axios interceptors, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'API clients and Axios interceptors' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: API clients and Axios interceptors

```
const api = axios.create({ baseURL: '/api', timeout: 8000 });
api.interceptors.request.use((config) => {
  config.headers['X-Request-ID'] = crypto.randomUUID();
  return config;
});

const { data } = await api.get('/users');
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Starting overlapping requests without cancellation or stale-response protection.
- Retrying non-idempotent mutations blindly.
- Leaving sockets/timers alive after the owning screen unmounts.

Production checklist

- Request has cancellation?
- Stale response cannot overwrite fresh state?
- Retry is bounded and safe?
- Errors and timeouts are observable?

Questions a strong reviewer may ask

- Can this request be cancelled?
- What prevents duplicate or stale updates?
- What is the reconnect/retry policy?

Hands-on lab

Implement a tiny request/message flow for 'API clients and Axios interceptors', then add delay, cancellation, failure, and duplicate triggers. Observe the result rather than guessing.

4.9 Polling

Why this matters. Understand periodic refresh, overlap prevention, visibility handling, cleanup, and when polling is simpler than a persistent channel.

Mental model. Network libraries do not decide your application flow. They execute when your code asks them to execute. Repeated requests are therefore a control-flow problem unless a retry layer is explicitly configured. Real-time transports add persistent connection state, which introduces lifecycle, reconnection, and backpressure concerns.

Core rules

- A network call executes because your control flow reached it; libraries only retry when you configure or wrap retries.
- Cancel or ignore obsolete async work when the initiating state changes.
- Design reconnect and retry paths with idempotency and bounded backoff.
- Measure latency by phase: queueing, connection, server processing, download, and client processing.

Request / execution flow

```
Trigger -> API client -> request -> remote server -> response/message -> state update -> UI
```

Under the hood

For polling, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Polling' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Polling

```
try {
  const response = await apiCall();
  updateState(response);
} catch (error) {
  reportError(error);
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Starting overlapping requests without cancellation or stale-response protection.
- Retrying non-idempotent mutations blindly.
- Leaving sockets/timers alive after the owning screen unmounts.

Production checklist

- Request has cancellation?
- Stale response cannot overwrite fresh state?
- Retry is bounded and safe?
- Errors and timeouts are observable?

Questions a strong reviewer may ask

- Can this request be cancelled?
- What prevents duplicate or stale updates?
- What is the reconnect/retry policy?

Hands-on lab

Implement a tiny request/message flow for 'Polling', then add delay, cancellation, failure, and duplicate triggers. Observe the result rather than guessing.

4.10 Long polling

Why this matters. Understand holding a request open until data changes and how it differs from ordinary polling.

Mental model. Network libraries do not decide your application flow. They execute when your code asks them to execute. Repeated requests are therefore a control-flow problem unless a retry layer is explicitly configured. Real-time transports add persistent connection state, which introduces lifecycle, reconnection, and backpressure concerns.

Core rules

- A network call executes because your control flow reached it; libraries only retry when you configure or wrap retries.
- Cancel or ignore obsolete async work when the initiating state changes.
- Design reconnect and retry paths with idempotency and bounded backoff.
- Measure latency by phase: queueing, connection, server processing, download, and client processing.

Request / execution flow

```
Trigger -> API client -> request -> remote server -> response/message -> state update -> UI
```

Under the hood

For long polling, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Long polling' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Long polling

```
try {
  const response = await apiCall();
  updateState(response);
} catch (error) {
  reportError(error);
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Starting overlapping requests without cancellation or stale-response protection.
- Retrying non-idempotent mutations blindly.
- Leaving sockets/timers alive after the owning screen unmounts.

Production checklist

- Request has cancellation?
- Stale response cannot overwrite fresh state?
- Retry is bounded and safe?
- Errors and timeouts are observable?

Questions a strong reviewer may ask

- Can this request be cancelled?
- What prevents duplicate or stale updates?
- What is the reconnect/retry policy?

Hands-on lab

Implement a tiny request/message flow for 'Long polling', then add delay, cancellation, failure, and duplicate triggers. Observe the result rather than guessing.

4.11 Server-Sent Events

Why this matters. Use a one-way server-to-browser stream for event feeds and automatic reconnection semantics.

Mental model. Network libraries do not decide your application flow. They execute when your code asks them to execute. Repeated requests are therefore a control-flow problem unless a retry layer is explicitly configured. Real-time transports add persistent connection state, which introduces lifecycle, reconnection, and backpressure concerns.

Core rules

- A network call executes because your control flow reached it; libraries only retry when you configure or wrap retries.
- Cancel or ignore obsolete async work when the initiating state changes.
- Design reconnect and retry paths with idempotency and bounded backoff.
- Measure latency by phase: queueing, connection, server processing, download, and client processing.

Request / execution flow

```
Trigger -> API client -> request -> remote server -> response/message -> state update -> UI
```

Under the hood

For server-sent events, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Server-Sent Events' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Server-Sent Events

```
try {
  const response = await apiCall();
  updateState(response);
} catch (error) {
  reportError(error);
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Starting overlapping requests without cancellation or stale-response protection.
- Retrying non-idempotent mutations blindly.
- Leaving sockets/timers alive after the owning screen unmounts.

Production checklist

- Request has cancellation?
- Stale response cannot overwrite fresh state?
- Retry is bounded and safe?
- Errors and timeouts are observable?

Questions a strong reviewer may ask

- Can this request be cancelled?
- What prevents duplicate or stale updates?
- What is the reconnect/retry policy?

Hands-on lab

Implement a tiny request/message flow for 'Server-Sent Events', then add delay, cancellation, failure, and duplicate triggers. Observe the result rather than guessing.

4.12 WebSockets

Why this matters. Understand the HTTP upgrade handshake, persistent bidirectional messages, connection state, heartbeats, backpressure, and reconnection.

Mental model. Network libraries do not decide your application flow. They execute when your code asks them to execute. Repeated requests are therefore a control-flow problem unless a retry layer is explicitly configured. Real-time transports add persistent connection state, which introduces lifecycle, reconnection, and backpressure concerns.

Core rules

- A network call executes because your control flow reached it; libraries only retry when you configure or wrap retries.
- Cancel or ignore obsolete async work when the initiating state changes.
- Design reconnect and retry paths with idempotency and bounded backoff.
- Measure latency by phase: queueing, connection, server processing, download, and client processing.
- A WebSocket starts as an HTTP handshake and then upgrades to a persistent full-duplex connection.
- Proxies need upgrade support and sensible idle timeouts; applications need heartbeat/reconnect and authorization lifecycle decisions.

Request / execution flow

```
Trigger -> API client -> request -> remote server -> response/message -> state update -> UI
```

Under the hood

For websockets, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'WebSockets' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: WebSockets

```
const ws = new WebSocket('wss://example.com/ws');
ws.addEventListener('open', () => ws.send(JSON.stringify({type: 'subscribe'})));
ws.addEventListener('message', (event) => {
  const msg = JSON.parse(event.data);
  console.log(msg);
});
ws.addEventListener('close', scheduleReconnect);
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Starting overlapping requests without cancellation or stale-response protection.
- Retrying non-idempotent mutations blindly.
- Leaving sockets/timers alive after the owning screen unmounts.
- Proxy upgrade headers missing.
- Idle timeout shorter than heartbeat interval.
- Reconnect loop has no backoff.

Production checklist

- Request has cancellation?
- Stale response cannot overwrite fresh state?
- Retry is bounded and safe?
- Errors and timeouts are observable?

Questions a strong reviewer may ask

- Can this request be cancelled?
- What prevents duplicate or stale updates?
- What is the reconnect/retry policy?

Hands-on lab

Implement a tiny request/message flow for 'WebSockets', then add delay, cancellation, failure, and duplicate triggers. Observe the result rather than guessing.

4.13 Socket.IO

Why this matters. Understand how Socket.IO adds events, reconnection, acknowledgements, rooms, and fallbacks on top of transport concepts; it is not the WebSocket protocol itself.

Mental model. Network libraries do not decide your application flow. They execute when your code asks them to execute. Repeated requests are therefore a control-flow problem unless a retry layer is explicitly configured. Real-time transports add persistent connection state, which introduces lifecycle, reconnection, and backpressure concerns.

Core rules

- A network call executes because your control flow reached it; libraries only retry when you configure or wrap retries.
- Cancel or ignore obsolete async work when the initiating state changes.
- Design reconnect and retry paths with idempotency and bounded backoff.
- Measure latency by phase: queueing, connection, server processing, download, and client processing.
- Socket.IO adds its own framing and semantics such as rooms and acknowledgements; a raw WebSocket client cannot simply speak Socket.IO.
- When scaling Socket.IO across instances, shared adapters or routing decisions may be required for room/event fan-out.

Request / execution flow

```
Trigger -> API client -> request -> remote server -> response/message -> state update -> UI
```

Under the hood

For socket.io, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Socket.IO' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Socket.IO

```
try {
  const response = await apiCall();
  updateState(response);
} catch (error) {
  reportError(error);
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Starting overlapping requests without cancellation or stale-response protection.
- Retrying non-idempotent mutations blindly.
- Leaving sockets/timers alive after the owning screen unmounts.

Production checklist

- Request has cancellation?
- Stale response cannot overwrite fresh state?
- Retry is bounded and safe?
- Errors and timeouts are observable?

Questions a strong reviewer may ask

- Can this request be cancelled?
- What prevents duplicate or stale updates?
- What is the reconnect/retry policy?

Hands-on lab

Implement a tiny request/message flow for 'Socket.IO', then add delay, cancellation, failure, and duplicate triggers. Observe the result rather than guessing.

4.14 Choosing polling, SSE, WebSocket, or queues

Why this matters. Choose the simplest communication model that matches directionality, latency, scale, delivery guarantees, and infrastructure.

Mental model. Network libraries do not decide your application flow. They execute when your code asks them to execute. Repeated requests are therefore a control-flow problem unless a retry layer is explicitly configured. Real-time transports add persistent connection state, which introduces lifecycle, reconnection, and backpressure concerns.

Core rules

- A network call executes because your control flow reached it; libraries only retry when you configure or wrap retries.
- Cancel or ignore obsolete async work when the initiating state changes.
- Design reconnect and retry paths with idempotency and bounded backoff.
- Measure latency by phase: queueing, connection, server processing, download, and client processing.

Request / execution flow

```
Trigger -> API client -> request -> remote server -> response/message -> state update -> UI
```

Under the hood

For choosing polling, sse, websocket, or queues, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Choosing polling, SSE, WebSocket, or queues' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Choosing polling, SSE, WebSocket, or queues

```
try {
  const response = await apiCall();
  updateState(response);
} catch (error) {
  reportError(error);
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Starting overlapping requests without cancellation or stale-response protection.
- Retrying non-idempotent mutations blindly.
- Leaving sockets/timers alive after the owning screen unmounts.
- Proxy upgrade headers missing.
- Idle timeout shorter than heartbeat interval.
- Reconnect loop has no backoff.

Production checklist

- Request has cancellation?
- Stale response cannot overwrite fresh state?
- Retry is bounded and safe?
- Errors and timeouts are observable?

Questions a strong reviewer may ask

- Can this request be cancelled?
- What prevents duplicate or stale updates?
- What is the reconnect/retry policy?

Hands-on lab

Implement a tiny request/message flow for 'Choosing polling, SSE, WebSocket, or queues', then add delay, cancellation, failure, and duplicate triggers. Observe the result rather than guessing.

Part 5

V. React from First Principles

React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Part flow: Event/state change -> render calculation -> commit -> browser layout/paint

5.1 React component execution

Why this matters. Treat a function component as code that React may call again; never assume it runs exactly once.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.
- Rendering calls component functions to calculate JSX. Commit applies necessary DOM changes. Browser paint happens afterward.
- A re-render can produce zero DOM mutations when the calculated result is unchanged.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For react component execution, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'React component execution' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: React component execution

```
function UsersPage({ users }: { users: User[] }) {  
  // Pure calculation only. Do not POST here.  
  const active = users.filter((u) => u.active);  
  return <UsersTable users={active} />;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'React component execution'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.2 Trigger, render, commit, and browser paint

Why this matters. Separate React calculation from DOM mutation and browser painting so that 'render' stops meaning four different things.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.
- Rendering calls component functions to calculate JSX. Commit applies necessary DOM changes. Browser paint happens afterward.
- A re-render can produce zero DOM mutations when the calculated result is unchanged.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For trigger, render, commit, and browser paint, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Trigger, render, commit, and browser paint' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Trigger, render, commit, and browser paint

```
function Component() {  
  const [value, setValue] = useState(0);  
  console.log('render', { value });  
  return <button onClick={() => setValue(v => v + 1)}>{value}</button>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'Trigger, render, commit, and browser paint'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.3 Render purity

Why this matters. Keep component rendering deterministic and free of side effects so retries and extra development renders are safe.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.
- Never call `axios.post`, mutate module globals, write `localStorage`, or subscribe during render.
- Strict Mode extra calls reveal impure rendering; disabling the check hides the bug rather than fixing it.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For render purity, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Render purity' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Render purity

```
function UsersPage({ users }: { users: User[] }) {  
  // Pure calculation only. Do not POST here.  
  const active = users.filter((u) => u.active);  
  return <UsersTable users={active} />;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'Render purity'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.4 Props, state, and derived values

Why this matters. Know what data should be passed, stored, calculated, or removed from state.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For props, state, and derived values, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Props, state, and derived values' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Props, state, and derived values

```
function Component() {  
  const [value, setValue] = useState(0);  
  console.log('render', { value });  
  return <button onClick={() => setValue(v => v + 1)}>{value}</button>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'Props, state, and derived values'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.5 State updates, batching, and snapshots

Why this matters. Understand why `setState` schedules work, why reads see a render snapshot, and how multiple updates are batched.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For state updates, batching, and snapshots, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'State updates, batching, and snapshots' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: State updates, batching, and snapshots

```
function Component() {  
  const [value, setValue] = useState(0);  
  console.log('render', { value });  
  return <button onClick={() => setValue(v => v + 1)}>{value}</button>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'State updates, batching, and snapshots'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.6 Event handlers

Why this matters. Put interaction-specific work in handlers and avoid accidentally invoking a handler while constructing JSX.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.
- `onClick={save}` passes a function; `onClick={save()}` executes immediately during render.
- If the effect is caused by a particular user gesture, keep it in that event path rather than encoding the gesture as an Effect dependency.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For event handlers, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Event handlers' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Event handlers

```
async function handleSubmit(e: React.FormEvent) {  
  e.preventDefault();  
  await api.post('/users', form);  
}  
  
return <form onSubmit={handleSubmit}>  
  <button type="submit">Create</button>  
</form>;
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'Event handlers'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.7 useEffect as synchronization

Why this matters. Use Effects for synchronizing with external systems rather than as a generic 'after render' callback.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.
- Effects run after a commit and are for synchronization with systems outside React.
- If you are deriving one piece of state from another, calculate it during render instead of using an Effect.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For useeffect as synchronization, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'useEffect as synchronization' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: `useEffect` as synchronization

```
useEffect(() => {  
  const socket = connect(roomId);  
  socket.subscribe(onMessage);  
  return () => {  
    socket.unsubscribe(onMessage);  
    socket.close();  
  };  
}, [roomId]);
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'useEffect as synchronization'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.8 Effect dependency arrays

Why this matters. Understand reactive dependencies, Object.is comparisons, unstable objects/functions, and infinite Effect loops.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.
- Each dependency is compared with Object.is. New object/function literals are different identities on every render.
- An Effect that sets a dependency can create an infinite loop.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For effect dependency arrays, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Effect dependency arrays' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Effect dependency arrays

```
useEffect(() => {  
  const socket = connect(roomId);  
  socket.subscribe(onMessage);  
  return () => {  
    socket.unsubscribe(onMessage);  
    socket.close();  
  };  
}, [roomId]);
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'Effect dependency arrays'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.9 Stale closures

Why this matters. Understand why callbacks capture values from the render that created them and how stale state appears in timers, subscriptions, and async code.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.
- A callback sees the state values from the render that created that callback.
- Use functional state updates, refs, or correctly updated dependencies when long-lived callbacks need current data.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For stale closures, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Stale closures' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Stale closures

```
function Component() {  
  const [value, setValue] = useState(0);  
  console.log('render', { value });  
  return <button onClick={() => setValue(v => v + 1)}>{value}</button>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'Stale closures'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.10 Effect cleanup and resource lifetime

Why this matters. Correctly remove listeners, timers, subscriptions, observers, and connections so remounts do not leak work.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.
- Return cleanup from `useEffect` for listeners, timers, sockets, subscriptions, and observers.
- Cleanup should make setup-cleanup-setup behavior indistinguishable to the user.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For effect cleanup and resource lifetime, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Effect cleanup and resource lifetime' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Effect cleanup and resource lifetime

```
useEffect(() => {  
  const socket = connect(roomId);  
  socket.subscribe(onMessage);  
  return () => {  
    socket.unsubscribe(onMessage);  
    socket.close();  
  };  
}, [roomId]);
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'Effect cleanup and resource lifetime'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.11 React Strict Mode

Why this matters. Understand development-only extra rendering and Effect checks and use them as bug detectors rather than disabling them.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.
- Strict Mode development checks intentionally re-render and re-run Effect setup/cleanup to expose unsafe lifecycle code.
- Production does not perform the same development-only stress checks, so compare performance in a production build.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For react strict mode, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'React Strict Mode' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: React Strict Mode

```
<StrictMode>
  <App />
</StrictMode>

// Development may run render and Effect checks extra times.
// Code must remain correct under setup -> cleanup -> setup.
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'React Strict Mode'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.12 Refs

Why this matters. Use refs for mutable values and imperative handles that do not need to trigger rendering.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.

Request / execution flow

Event/state change -> render calculation -> commit -> browser layout/paint

Under the hood

For refs, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Refs' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Refs

```
function Component() {  
  const [value, setValue] = useState(0);  
  console.log('render', { value });  
  return <button onClick={() => setValue(v => v + 1)}>{value}</button>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'Refs'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.13 Context

Why this matters. Use context for broadly shared dependencies while understanding that provider value changes can re-render consumers.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For context, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Context' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Context

```
function Component() {
  const [value, setValue] = useState(0);
  console.log('render', { value });
  return <button onClick={() => setValue(v => v + 1)}>{value}</button>;
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'Context'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.14 Component identity and keys

Why this matters. Understand preservation and resetting of state based on tree position and keys.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.
- React preserves state when the same component type remains at the same tree position.
- Changing a key forces React to treat the element as a different identity and reset local state.

Request / execution flow

Event/state change -> render calculation -> commit -> browser layout/paint

Under the hood

For component identity and keys, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Component identity and keys' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Component identity and keys

```
function Component() {  
  const [value, setValue] = useState(0);  
  console.log('render', { value });  
  return <button onClick={() => setValue(v => v + 1)}>{value}</button>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'Component identity and keys'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.15 Controlled and uncontrolled forms

Why this matters. Understand value ownership, `defaultValue`, form libraries, validation, and performance tradeoffs.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.
- A button inside a form defaults to `type=submit` unless specified otherwise in HTML.
- Calling `preventDefault` controls native navigation, but you still need duplicate-submission protection for network mutations.

Request / execution flow

Event/state change -> render calculation -> commit -> browser layout/paint

Under the hood

For controlled and uncontrolled forms, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Controlled and uncontrolled forms' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Controlled and uncontrolled forms

```
function Component() {  
  const [value, setValue] = useState(0);  
  console.log('render', { value });  
  return <button onClick={() => setValue(v => v + 1)}>{value}</button>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'Controlled and uncontrolled forms'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.16 State ownership and lifting state

Why this matters. Place state at the lowest common owner and avoid making a small interaction re-render a huge tree unnecessarily.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For state ownership and lifting state, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'State ownership and lifting state' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: State ownership and lifting state

```
function Component() {  
  const [value, setValue] = useState(0);  
  console.log('render', { value });  
  return <button onClick={() => setValue(v => v + 1)}>{value}</button>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'State ownership and lifting state'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.17 React.memo, useMemo, and useCallback

Why this matters. Use memoization to avoid measured work, not as decoration; understand referential identity.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.

Request / execution flow

Event/state change -> render calculation -> commit -> browser layout/paint

Under the hood

For react.memo, usememo, and usecallback, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'React.memo, useMemo, and useCallback' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: React.memo, useMemo, and useCallback

```
function Component() {  
  const [value, setValue] = useState(0);  
  console.log('render', { value });  
  return <button onClick={() => setValue(v => v + 1)}>{value}</button>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'React.memo, useMemo, and useCallback'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.18 Suspense and error boundaries

Why this matters. Understand how loading and failure can be modeled as boundaries in the component tree.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.

Request / execution flow

Event/state change -> render calculation -> commit -> browser layout/paint

Under the hood

For suspense and error boundaries, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Suspense and error boundaries' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Suspense and error boundaries

```
function Component() {  
  const [value, setValue] = useState(0);  
  console.log('render', { value });  
  return <button onClick={() => setValue(v => v + 1)}>{value}</button>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'Suspense and error boundaries'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.19 Transitions and concurrent rendering

Why this matters. Mark non-urgent updates, preserve responsiveness, and understand that render work can be interrupted or restarted.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For transitions and concurrent rendering, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Transitions and concurrent rendering' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Transitions and concurrent rendering

```
function Component() {  
  const [value, setValue] = useState(0);  
  console.log('render', { value });  
  return <button onClick={() => setValue(v => v + 1)}>{value}</button>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'Transitions and concurrent rendering'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.20 React DevTools Profiler

Why this matters. Measure which components render, how long they take, and what interaction caused a commit.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For react devtools profiler, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'React DevTools Profiler' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: React DevTools Profiler

```
function Component() {  
  const [value, setValue] = useState(0);  
  console.log('render', { value });  
  return <button onClick={() => setValue(v => v + 1)}>{value}</button>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'React DevTools Profiler'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.21 MUI Dialog, Modal, portals, and focus

Why this matters. Understand why opening a dialog changes state, how portals place DOM outside the local subtree, and how mounting and focus management affect performance.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.
- Opening a controlled Dialog usually calls `setOpen(true)`, which re-renders the owner; that alone is not a bug.
- MUI modal content may mount/unmount and uses portal/focus management; expensive content can be profiled and optionally kept mounted when justified.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For mui dialog, modal, portals, and focus, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Mui Dialog, Modal, portals, and focus' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: MUI Dialog, Modal, portals, and focus

```
function DialogController() {
  const [open, setOpen] = useState(false);
  return <>
    <Button onClick={() => setOpen(true)}>New user</Button>
    <Dialog open={open} onClose={() => setOpen(false)}>
      <UserForm />
    </Dialog>
  </>;
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.
- Treating an expected owner re-render as a bug without profiling.
- Keeping expensive table state in the same owner as dialog state.
- Measuring dev Strict Mode logs instead of production commit cost.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'MUI Dialog, Modal, portals, and focus'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.22 Responsive hooks and resize behavior

Why this matters. Understand media queries, resize observers, viewport changes, and why screenshots or DevTools resizing can trigger layout or state changes.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For responsive hooks and resize behavior, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Responsive hooks and resize behavior' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Responsive hooks and resize behavior

```
function Component() {  
  const [value, setValue] = useState(0);  
  console.log('render', { value });  
  return <button onClick={() => setValue(v => v + 1)}>{value}</button>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'Responsive hooks and resize behavior'. Add render counters and profile one interaction before and after a deliberate bug/fix.

5.23 React performance patterns

Why this matters. Localize state, remove Effect chains, split expensive subtrees, virtualize large lists, and keep urgent interactions small.

Mental model. React repeatedly evaluates component trees to compute UI. A render is a calculation; a commit changes the DOM; the browser then lays out and paints. State updates, parent renders, context changes, and external stores can trigger new calculations. Keep rendering pure so React can safely retry work.

Core rules

- Component bodies must be pure: do not POST, subscribe, mutate shared state, or register listeners during render.
- Interaction-specific side effects belong in event handlers; synchronization with external systems belongs in Effects.
- Every Effect that acquires a resource should have a cleanup story.
- Optimize measured bottlenecks; localizing state and removing unnecessary Effects often beats blanket memoization.

Request / execution flow

```
Event/state change -> render calculation -> commit -> browser layout/paint
```

Under the hood

For react performance patterns, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'React performance patterns' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: React performance patterns

```
function Component() {  
  const [value, setValue] = useState(0);  
  console.log('render', { value });  
  return <button onClick={() => setValue(v => v + 1)}>{value}</button>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Performing side effects during render.
- Using Effects to derive state that could be calculated directly.
- Optimizing with memoization before measuring the actual expensive subtree.

Production checklist

- Render is pure?
- State lives low enough?
- Effect really synchronizes an external system?
- Cleanup exists?
- Profiler shows a real bottleneck?

Questions a strong reviewer may ask

- Why is this state here?
- Why is this an Effect instead of an event or derived value?
- What causes this subtree to render?

Hands-on lab

Build a component that demonstrates 'React performance patterns'. Add render counters and profile one interaction before and after a deliberate bug/fix.

Part 6

VI. Next.js App Router and Server Boundaries

Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Part flow: Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI request -> response/RSC/HTML -> browser

6.1 App Router file-system routing

Why this matters. Understand `page.tsx`, `layout.tsx`, `route.ts`, `loading.tsx`, `error.tsx`, `not-found.tsx`, dynamic segments, and route groups.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through `proxy.ts`; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI
request -> response/RSC/HTML -> browser
```

Under the hood

For app router file-system routing, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'App Router file-system routing' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: App Router file-system routing

```
export default async function Page() {  
  // First question: does this line run in the browser or on the Next.js server?  
  return <main>Inspect the execution boundary before adding networking.</main>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'App Router file-system routing' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.2 Server Components versus Client Components

Why this matters. Choose an execution environment deliberately and understand the network boundary introduced by 'use client'.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.
- Server Components can access server resources and do not ship their component code to the browser; Client Components enable browser interactivity.
- The 'use client' directive establishes a client boundary for that module and its imported client graph.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI  
request -> response/RSC/HTML -> browser
```

Under the hood

For server components versus client components, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Server Components versus Client Components' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Server Components versus Client Components

```
// Server Component (no 'use client')
export default async function Page() {
  const res = await fetch(`${process.env.API_URL}/users`, { cache: 'no-store' });
  const users = await res.json();
  return <UsersTable users={users} />;
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Server Components versus Client Components' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.3 React Server Components payload

Why this matters. Understand that Server Components are serialized as an RSC payload rather than shipped as client JavaScript.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI  
request -> response/RSC/HTML -> browser
```

Under the hood

For react server components payload, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'React Server Components payload' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: React Server Components payload

```
export default async function Page() {  
  // First question: does this line run in the browser or on the Next.js server?  
  return <main>Inspect the execution boundary before adding networking.</main>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'React Server Components payload' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.4 SSR and hydration

Why this matters. Understand initial HTML generation, hydration of interactive client islands, and why hydration mismatches occur.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.
- Hydration attaches client behavior to server-generated HTML; server and client initial output must match.
- Random values, timestamps, browser-only conditionals, locale differences, or invalid HTML nesting can cause mismatches.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI  
request -> response/RSC/HTML -> browser
```

Under the hood

For SSR and hydration, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'SSR and hydration' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: SSR and hydration

```
export default async function Page() {  
  // First question: does this line run in the browser or on the Next.js server?  
  return <main>Inspect the execution boundary before adding networking.</main>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'SSR and hydration' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.5 Server-side data fetching

Why this matters. Fetch close to the data source when possible and understand request context, caching, and backend credentials.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI  
request -> response/RSC/HTML -> browser
```

Under the hood

For server-side data fetching, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Server-side data fetching' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Server-side data fetching

```
// Server Component (no 'use client')
export default async function Page() {
  const res = await fetch(`${process.env.API_URL}/users`, { cache: 'no-store' });
  const users = await res.json();
  return <UsersTable users={users} />;
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Server-side data fetching' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.6 Client-side data fetching

Why this matters. Fetch after hydration for interactive or user-driven data and understand loading, cancellation, caching, and browser security.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI  
request -> response/RSC/HTML -> browser
```

Under the hood

For client-side data fetching, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Client-side data fetching' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Client-side data fetching

```
const controller = new AbortController();
const res = await fetch('/api/users', {
  signal: controller.signal,
  credentials: 'include',
});
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const users = await res.json();
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Client-side data fetching' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.7 Route Handlers

Why this matters. Build HTTP endpoints inside Next.js and understand Request/Response, methods, validation, and when a BFF route is useful.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.
- route.ts defines HTTP handlers such as GET/POST; it is appropriate when the browser needs an HTTP endpoint owned by Next.js.
- Do not automatically route Server Component data through your own Route Handler; that can add an unnecessary hop.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI
request -> response/RSC/HTML -> browser
```

Under the hood

For route handlers, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Route Handlers' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Route Handlers

```
// app/api/users/route.ts
export async function GET(request: Request) {
  const res = await fetch(`${process.env.API_URL}/users`, {
    headers: { 'X-Internal-Source': 'next-bff' },
  });
  return new Response(res.body, { status: res.status, headers: res.headers });
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Route Handlers' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.8 Server Actions

Why this matters. Understand browser-to-server function invocation, mutation workflows, authorization requirements, and downstream backend calls.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.
- Server Actions execute on the server but can be invoked from UI interactions/forms.
- Treat every action as an externally reachable mutation boundary: validate input and re-check authorization inside the action or downstream trusted backend.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI  
request -> response/RSC/HTML -> browser
```

Under the hood

For server actions, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Server Actions' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Server Actions

```
'use server';

export async function createUser(formData: FormData) {
  const name = String(formData.get('name') ?? '');
  if (!name) throw new Error('name required');
  await backendFetch('/users', {
    method: 'POST',
    body: JSON.stringify({ name }),
  });
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Server Actions' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.9 proxy.ts

Why this matters. Understand that Proxy handles incoming requests to Next.js; it does not automatically intercept every outbound fetch from server code.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.
- proxy.ts sees incoming requests that match its configuration. A server-side fetch from Next.js to FastAPI is an outbound request and does not automatically pass through proxy.ts.
- Use Proxy for routing/rewrites/lightweight request handling, not as the only authorization enforcement for sensitive operations.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI
request -> response/RSC/HTML -> browser
```

Under the hood

For proxy.ts, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'proxy.ts' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: proxy.ts

```
// proxy.ts
export function proxy(request: NextRequest) {
  const headers = new Headers(request.headers);
  headers.set('x-request-id', request.headers.get('x-request-id') ?? crypto.randomUUID());
  return NextResponse.next({ request: { headers } });
}
// This sees incoming Next.js requests, not arbitrary outbound server fetches.
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'proxy.ts' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.10 Reading headers and cookies on the server

Why this matters. Read incoming request context safely and understand dynamic rendering implications.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.
- HttpOnly prevents JavaScript access but does not prevent the browser from attaching the cookie to eligible requests.
- SameSite changes whether cookies accompany cross-site requests; Secure requires HTTPS; Domain/Path scope eligibility.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI  
request -> response/RSC/HTML -> browser
```

Under the hood

For reading headers and cookies on the server, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Reading headers and cookies on the server' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Reading headers and cookies on the server

```
import { headers } from 'next/headers';

export async function backendFetch(path: string, init: RequestInit = {}) {
  const incoming = await headers();
  const tenant = incoming.get('x-trusted-tenant');
  return fetch(`${process.env.API_URL}${path}`, {
    ...init,
    headers: { ...init.headers, ...(tenant ? {'x-tenant-id': tenant} : {}) },
  });
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Reading headers and cookies on the server' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.11 Forwarding headers to FastAPI

Why this matters. Treat a downstream fetch as a new HTTP request and forward only the identity and context that the backend genuinely needs.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.
- Whitelist the headers to forward. Forwarding every browser header can leak cookies, hop-by-hop metadata, or spoofed values.
- If tenant/user identity is derived by trusted server code, prefer forwarding a trusted internal representation over blindly trusting a client-provided X-Tenant-ID.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI
request -> response/RSC/HTML -> browser
```

Under the hood

For forwarding headers to fastapi, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Forwarding headers to FastAPI' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Forwarding headers to FastAPI

```
import { headers } from 'next/headers';

export async function backendFetch(path: string, init: RequestInit = {}) {
  const incoming = await headers();
  const tenant = incoming.get('x-trusted-tenant');
  return fetch(`${process.env.API_URL}${path}`, {
    ...init,
    headers: { ...init.headers, ...(tenant ? {'x-tenant-id': tenant} : {}) },
  });
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Forwarding headers to FastAPI' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.12 Backend-for-Frontend architecture

Why this matters. Use Next.js as a client-specific server boundary without turning every Server Component fetch into an unnecessary extra hop.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI  
request -> response/RSC/HTML -> browser
```

Under the hood

For backend-for-frontend architecture, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Backend-for-Frontend architecture' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Backend-for-Frontend architecture

```
export default async function Page() {  
  // First question: does this line run in the browser or on the Next.js server?  
  return <main>Inspect the execution boundary before adding networking.</main>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Backend-for-Frontend architecture' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.13 Authentication in Next.js

Why this matters. Understand session verification, authorization at mutation boundaries, cookie handling, and trusted server-only code.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI  
request -> response/RSC/HTML -> browser
```

Under the hood

For authentication in next.js, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Authentication in Next.js' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Authentication in Next.js

```
export default async function Page() {  
  // First question: does this line run in the browser or on the Next.js server?  
  return <main>Inspect the execution boundary before adding networking.</main>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Authentication in Next.js' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.14 Next.js caching and revalidation

Why this matters. Understand request memoization, data caching, revalidation tags/paths, and stale UI failure modes.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.
- Stale data can come from multiple caches: browser, CDN, Next data cache, client query cache, Redis, or database replicas.
- Every mutation needs an explicit answer to: which cached views become stale and how are they invalidated or revalidated?

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI
request -> response/RSC/HTML -> browser
```

Under the hood

For next.js caching and revalidation, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Next.js caching and revalidation' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Next.js caching and revalidation

```
export default async function Page() {  
  // First question: does this line run in the browser or on the Next.js server?  
  return <main>Inspect the execution boundary before adding networking.</main>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Next.js caching and revalidation' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.15 Loading, error, and not-found boundaries

Why this matters. Model route-level pending, failure, and absence states without mixing them into every component.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI  
request -> response/RSC/HTML -> browser
```

Under the hood

For loading, error, and not-found boundaries, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Loading, error, and not-found boundaries' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Loading, error, and not-found boundaries

```
export default async function Page() {  
  // First question: does this line run in the browser or on the Next.js server?  
  return <main>Inspect the execution boundary before adding networking.</main>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Loading, error, and not-found boundaries' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.16 Streaming with Suspense

Why this matters. Send useful UI earlier while slower server work continues, and understand how this changes waterfalls.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI  
request -> response/RSC/HTML -> browser
```

Under the hood

For streaming with suspense, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Streaming with Suspense' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Streaming with Suspense

```
export default async function Page() {  
  // First question: does this line run in the browser or on the Next.js server?  
  return <main>Inspect the execution boundary before adding networking.</main>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Streaming with Suspense' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.17 Link, navigation, and prefetching

Why this matters. Understand client navigation, route prefetch, history integration, and when navigation causes fresh server work.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI  
request -> response/RSC/HTML -> browser
```

Under the hood

For link, navigation, and prefetching, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Link, navigation, and prefetching' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Link, navigation, and prefetching

```
export default async function Page() {  
  // First question: does this line run in the browser or on the Next.js server?  
  return <main>Inspect the execution boundary before adding networking.</main>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Link, navigation, and prefetching' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.18 Environment variables and server-only code

Why this matters. Protect secrets, distinguish build-time and runtime configuration, and avoid exposing private variables to client bundles.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.
- Client-exposed environment variables become part of browser-delivered code and must not contain secrets.
- Some frameworks resolve values at build time; production runtime configuration must match the deployment model.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI
request -> response/RSC/HTML -> browser
```

Under the hood

For environment variables and server-only code, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Environment variables and server-only code' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Environment variables and server-only code

```
export default async function Page() {  
  // First question: does this line run in the browser or on the Next.js server?  
  return <main>Inspect the execution boundary before adding networking.</main>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Environment variables and server-only code' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.19 Images, fonts, scripts, and bundle size

Why this matters. Use framework optimizations while understanding the cost of client JavaScript and third-party scripts.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI  
request -> response/RSC/HTML -> browser
```

Under the hood

For images, fonts, scripts, and bundle size, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Images, fonts, scripts, and bundle size' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Images, fonts, scripts, and bundle size

```
export default async function Page() {  
  // First question: does this line run in the browser or on the Next.js server?  
  return <main>Inspect the execution boundary before adding networking.</main>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Images, fonts, scripts, and bundle size' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.20 Development versus production behavior

Why this matters. Understand HMR, source maps, Strict Mode checks, minification, production caching, proxies, and infrastructure differences.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI  
request -> response/RSC/HTML -> browser
```

Under the hood

For development versus production behavior, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Development versus production behavior' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Development versus production behavior

```
export default async function Page() {  
  // First question: does this line run in the browser or on the Next.js server?  
  return <main>Inspect the execution boundary before adding networking.</main>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Development versus production behavior' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

6.21 Next.js production performance

Why this matters. Profile server time, client bundles, streaming, data waterfalls, caching, and static/dynamic tradeoffs.

Mental model. Next.js spans two execution environments separated by a network boundary: the browser and the server. The same project contains code that runs in different places. Always identify whether a line runs in a Client Component, Server Component, Route Handler, Server Action, or Proxy before reasoning about headers, cookies, secrets, or fetch behavior.

Core rules

- Ask where the code runs before asking what API to use.
- Incoming requests may pass through proxy.ts; an outbound server fetch is a separate request and does not magically re-enter that proxy.
- Keep secrets and privileged credentials in server-only code.
- Prefer direct server data access when appropriate; add a BFF hop when it provides a real boundary such as browser-safe auth, aggregation, or contract adaptation.

Request / execution flow

```
Browser request -> proxy.ts (if matched) -> Next route/component/action -> optional FastAPI  
request -> response/RSC/HTML -> browser
```

Under the hood

For next.js production performance, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Next.js production performance' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Next.js production performance

```
export default async function Page() {  
  // First question: does this line run in the browser or on the Next.js server?  
  return <main>Inspect the execution boundary before adding networking.</main>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Assuming proxy.ts intercepts outbound server fetches.
- Exposing a server secret through client code or public environment variables.
- Adding a Route Handler hop without a clear architectural reason.

Production checklist

- Execution environment known?
- Browser vs server request path drawn?
- Secrets stay server-side?
- Authorization checked at trusted mutation/resource boundary?
- Caching/revalidation intentional?

Questions a strong reviewer may ask

- Why server or client?
- Does this request pass through proxy.ts?
- Why is a BFF hop needed here?
- Where is authorization re-checked?

Hands-on lab

Implement 'Next.js production performance' in a toy Next.js route. Write down which lines run in browser versus server and capture every HTTP hop.

Part 7

VII. FastAPI and Python Service Internals

FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Part flow: Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database -> response

7.1 ASGI and Uvicorn

Why this matters. Understand the protocol boundary between an async Python application and the server process accepting connections.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For asgi and uvicorn, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'ASGI and Uvicorn' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: ASGI and Uvicorn

```
@app.get('/health')
async def health():
    return {'status': 'ok'}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'ASGI and Uvicorn', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

7.2 FastAPI routing

Why this matters. Understand path matching, path parameters, query parameters, request bodies, and method dispatch.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For fastapi routing, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'FastAPI routing' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: FastAPI routing

```
@app.get('/users/{user_id}')
async def get_user(user_id: int, include_roles: bool = False):
    user = await service.get_user(user_id, include_roles=include_roles)
    if user is None:
        raise HTTPException(status_code=404, detail='user not found')
    return user
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'FastAPI routing', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

7.3 Pydantic validation

Why this matters. Understand parsing, typed models, validation failures, serialization, and why 422 errors happen.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.
- FastAPI validates request data against typed models before entering your endpoint.
- A 422 usually means the request reached the route but failed schema/path/query validation, so compare the actual request payload to the model.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For pydantic validation, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Pydantic validation' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Pydantic validation

```
class UserCreate(BaseModel):
    name: str = Field(min_length=1, max_length=100)
    email: EmailStr

@app.post('/users', status_code=201)
async def create_user(payload: UserCreate):
    return await service.create_user(payload)
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'Pydantic validation', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

7.4 Dependency injection

Why this matters. Use dependencies for authentication, database sessions, tenant context, and reusable request requirements.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For dependency injection, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Dependency injection' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Dependency injection

```
async def current_user(token: str = Depends(oauth2_scheme)) -> User:
    claims = verify_token(token)
    user = await users.get(claims.sub)
    if not user:
        raise HTTPException(status_code=401)
    return user

@app.get('/me')
async def me(user: User = Depends(current_user)):
    return user
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'Dependency injection', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

7.5 Middleware and middleware order

Why this matters. Understand request/response wrapping and why ordering changes authentication, logging, CORS, and error behavior.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.
- Middleware forms a stack: outer layers see the request first and the response last.
- Ordering affects CORS, tracing, exception handling, compression, and authentication observations.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For middleware and middleware order, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Middleware and middleware order' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Middleware and middleware order

```
@app.middleware('http')
async def request_context(request: Request, call_next):
    request_id = request.headers.get('x-request-id') or str(uuid4())
    start = time.perf_counter()
    response = await call_next(request)
    response.headers['x-request-id'] = request_id
    response.headers['server-timing'] = f'app;dur={time.perf_counter()-start}*1000:.1f}'
    return response
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'Middleware and middleware order', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

7.6 async def, def, and concurrency

Why this matters. Understand the event loop, blocking work, thread pools, I/O concurrency, and why async does not make CPU work faster.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.
- Await releases the event loop only when awaiting non-blocking operations; CPU-bound work still monopolizes the process.
- Blocking database/client libraries may need threadpool execution or async-native alternatives.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For async def, def, and concurrency, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'async def, def, and concurrency' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: async def, def, and concurrency

```
@app.get('/health')
async def health():
    return {'status': 'ok'}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'async def, def, and concurrency', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

7.7 Database session lifetime

Why this matters. Scope transactions and connections to requests and avoid leaks or sharing unsafe sessions across concurrent requests.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.
- A request-scoped session/transaction should be closed even on failure.
- Do not share non-thread-safe sessions globally across concurrent requests.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For database session lifetime, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Database session lifetime' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Database session lifetime

```
@app.get('/health')
async def health():
    return {'status': 'ok'}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'Database session lifetime', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

7.8 Repository and service layers

Why this matters. Separate HTTP concerns, business logic, and persistence without creating ceremony for its own sake.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For repository and service layers, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Repository and service layers' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Repository and service layers

```
@app.get('/health')
async def health():
    return {'status': 'ok'}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'Repository and service layers', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

7.9 FastAPI CORS

Why this matters. Configure explicit origins and credentials correctly and understand what the middleware does with preflight and normal requests.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.
- CORS does not stop a request from reaching the server in all cases; it primarily controls whether browser JavaScript may access the response.
- A preflight is an OPTIONS request that asks whether the actual cross-origin request is allowed.
- Credentials require explicit compatible origin configuration; wildcard behavior is restricted.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For fastapi cors, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'FastAPI CORS' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: FastAPI CORS

```
@app.get('/health')
async def health():
    return {'status': 'ok'}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'FastAPI CORS', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

7.10 Authentication and Security dependencies

Why this matters. Extract credentials, verify identities, load users/tenants, and enforce permissions at the backend boundary.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For authentication and security dependencies, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Authentication and Security dependencies' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Authentication and Security dependencies

```
async def current_user(token: str = Depends(oauth2_scheme)) -> User:
    claims = verify_token(token)
    user = await users.get(claims.sub)
    if not user:
        raise HTTPException(status_code=401)
    return user

@app.get('/me')
async def me(user: User = Depends(current_user)):
    return user
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'Authentication and Security dependencies', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

7.11 Exception handling and error responses

Why this matters. Map expected domain failures to stable HTTP responses while letting unexpected failures remain observable.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For exception handling and error responses, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Exception handling and error responses' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Exception handling and error responses

```
@app.get('/health')
async def health():
    return {'status': 'ok'}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'Exception handling and error responses', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

7.12 BackgroundTasks

Why this matters. Understand in-process post-response work and when durability requirements demand a real queue instead.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.
- FastAPI BackgroundTasks run in the application process after the response path; they are not a durable job queue.
- If the process dies, in-process background work can be lost.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For backgroundtasks, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'BackgroundTasks' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: BackgroundTasks

```
@app.get('/health')
async def health():
    return {'status': 'ok'}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'BackgroundTasks', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

7.13 FastAPI WebSockets

Why this matters. Accept, authenticate, read, write, disconnect, and manage many persistent connections safely.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.
- A WebSocket starts as an HTTP handshake and then upgrades to a persistent full-duplex connection.
- Proxies need upgrade support and sensible idle timeouts; applications need heartbeat/reconnect and authorization lifecycle decisions.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For fastapi websockets, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'FastAPI WebSockets' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: FastAPI WebSockets

```
@app.websocket('/ws')
async def websocket_endpoint(ws: WebSocket):
    await ws.accept()
    try:
        while True:
            message = await ws.receive_json()
            await ws.send_json({'echo': message})
    except WebSocketDisconnect:
        pass
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.
- Proxy upgrade headers missing.
- Idle timeout shorter than heartbeat interval.
- Reconnect loop has no backoff.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'FastAPI WebSockets', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

7.14 Uploads, downloads, and streaming

Why this matters. Handle large bodies without loading everything into memory and understand content length and backpressure.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For uploads, downloads, and streaming, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Uploads, downloads, and streaming' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Uploads, downloads, and streaming

```
@app.get('/health')
async def health():
    return {'status': 'ok'}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'Uploads, downloads, and streaming', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

7.15 OpenAPI and API documentation

Why this matters. Use generated schemas as a contract aid while understanding that runtime behavior and authorization still need tests.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For openapi and api documentation, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'OpenAPI and API documentation' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: OpenAPI and API documentation

```
@app.get('/health')
async def health():
    return {'status': 'ok'}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'OpenAPI and API documentation', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

7.16 Testing FastAPI

Why this matters. Test validation, dependencies, auth, database behavior, and HTTP contracts with isolated and integration tests.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For testing fastapi, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Testing FastAPI' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Testing FastAPI

```
@app.get('/health')
async def health():
    return {'status': 'ok'}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'Testing FastAPI', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

7.17 FastAPI performance and workers

Why this matters. Understand process workers, connection pools, blocking dependencies, serialization cost, and bottleneck measurement.

Mental model. FastAPI turns an incoming ASGI request into validated Python inputs, executes dependencies and endpoint logic, and serializes a response. Middleware wraps the request/response path. Async code improves I/O concurrency only when the operations underneath cooperate with the event loop.

Core rules

- Treat validation, authentication, authorization, business logic, and persistence as separate responsibilities.
- Never block the event loop with long synchronous work inside async endpoints.
- Scope database connections and transactions deliberately.
- Return stable error semantics and log unexpected failures with correlation context.

Request / execution flow

```
Socket/ASGI -> middleware -> router -> dependencies -> validation -> endpoint/service -> database  
-> response
```

Under the hood

For fastapi performance and workers, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'FastAPI performance and workers' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: FastAPI performance and workers

```
@app.get('/health')
async def health():
    return {'status': 'ok'}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Blocking async endpoints with synchronous I/O or CPU-heavy work.
- Using a global mutable database session across requests.
- Returning inconsistent error shapes that force clients to guess.

Production checklist

- Input validated?
- Auth and tenant permissions enforced?
- Async path non-blocking?
- DB transaction/session lifetime clear?
- Error and request ID logged?

Questions a strong reviewer may ask

- Where is validation/auth/tenant enforcement?
- Can this block the event loop?
- What owns transaction cleanup?

Hands-on lab

Implement a small FastAPI endpoint around 'FastAPI performance and workers', add request-ID logging, then test valid, invalid, unauthorized, and concurrent requests.

Part 8

VIII. API and Data Architecture

Data architecture is about correctness under scale and concurrency. Query shape, ordering, transactions, indexes, retries, and duplicate execution matter as much as endpoint syntax. Design contracts that stay deterministic when multiple users and workers act at once.

Part flow: API input -> validation -> transaction/query plan -> storage -> deterministic result -> API response

8.1 REST resource modeling

Why this matters. Design resources and actions so URLs and methods communicate intent instead of becoming RPC-shaped accidents.

Mental model. Data architecture is about correctness under scale and concurrency. Query shape, ordering, transactions, indexes, retries, and duplicate execution matter as much as endpoint syntax. Design contracts that stay deterministic when multiple users and workers act at once.

Core rules

- Always define deterministic ordering for pagination.
- Correctness under concurrent updates must be designed, not assumed.
- Indexes should match real query patterns and be validated with query plans.
- Durable asynchronous work needs idempotency, retry policy, and dead-letter or failure handling.

Request / execution flow

```
API input -> validation -> transaction/query plan -> storage -> deterministic result -> API response
```

Under the hood

For rest resource modeling, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'REST resource modeling' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: REST resource modeling

```
EXPLAIN ANALYZE
SELECT id, name
FROM users
WHERE tenant_id = $1
ORDER BY id
LIMIT 50;
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Paginating without deterministic ordering.
- Ignoring transaction/concurrency behavior.
- Adding indexes without validating real query plans.

Production checklist

- Ordering deterministic?
- Index/query plan verified?
- Concurrency semantics documented?
- Limits prevent unbounded work?
- Retries/duplicates safe?

Questions a strong reviewer may ask

- How does this behave with 1M rows and concurrent writes?
- What is the stable ordering?
- Which index serves the query?

Hands-on lab

Create a small dataset for 'REST resource modeling', inspect the query/transaction behavior, then scale the dataset or add concurrent writers to expose the edge case.

8.2 Pagination

Why this matters. Choose offset or cursor pagination, return stable ordering, and avoid loading huge result sets into memory or the DOM.

Mental model. Data architecture is about correctness under scale and concurrency. Query shape, ordering, transactions, indexes, retries, and duplicate execution matter as much as endpoint syntax. Design contracts that stay deterministic when multiple users and workers act at once.

Core rules

- Always define deterministic ordering for pagination.
- Correctness under concurrent updates must be designed, not assumed.
- Indexes should match real query patterns and be validated with query plans.
- Durable asynchronous work needs idempotency, retry policy, and dead-letter or failure handling.
- Offset pagination is simple but can become slow or inconsistent on frequently changing datasets.
- Cursor pagination needs a stable indexed ordering key and an opaque continuation token or well-defined cursor fields.

Request / execution flow

```
API input -> validation -> transaction/query plan -> storage -> deterministic result -> API response
```

Under the hood

For pagination, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Pagination' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Pagination

```
@router.get('/users')
async def list_users(limit: int = Query(50, le=100), after: int | None = None):
    rows = await repo.list_after(after=after, limit=limit + 1)
    has_more = len(rows) > limit
    items = rows[:limit]
    return {'items': items, 'next': items[-1].id if has_more else None}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Paginating without deterministic ordering.
- Ignoring transaction/concurrency behavior.
- Adding indexes without validating real query plans.

Production checklist

- Ordering deterministic?
- Index/query plan verified?
- Concurrency semantics documented?
- Limits prevent unbounded work?
- Retries/duplicates safe?

Questions a strong reviewer may ask

- How does this behave with 1M rows and concurrent writes?
- What is the stable ordering?
- Which index serves the query?

Hands-on lab

Create a small dataset for 'Pagination', inspect the query/transaction behavior, then scale the dataset or add concurrent writers to expose the edge case.

8.3 Search, filter, and sort

Why this matters. Define validated query syntax, indexes, limits, and deterministic ordering for production list endpoints.

Mental model. Data architecture is about correctness under scale and concurrency. Query shape, ordering, transactions, indexes, retries, and duplicate execution matter as much as endpoint syntax. Design contracts that stay deterministic when multiple users and workers act at once.

Core rules

- Always define deterministic ordering for pagination.
- Correctness under concurrent updates must be designed, not assumed.
- Indexes should match real query patterns and be validated with query plans.
- Durable asynchronous work needs idempotency, retry policy, and dead-letter or failure handling.

Request / execution flow

```
API input -> validation -> transaction/query plan -> storage -> deterministic result -> API response
```

Under the hood

For search, filter, and sort, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Search, filter, and sort' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Search, filter, and sort

```
EXPLAIN ANALYZE
SELECT id, name
FROM users
WHERE tenant_id = $1
ORDER BY id
LIMIT 50;
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Paginating without deterministic ordering.
- Ignoring transaction/concurrency behavior.
- Adding indexes without validating real query plans.

Production checklist

- Ordering deterministic?
- Index/query plan verified?
- Concurrency semantics documented?
- Limits prevent unbounded work?
- Retries/duplicates safe?

Questions a strong reviewer may ask

- How does this behave with 1M rows and concurrent writes?
- What is the stable ordering?
- Which index serves the query?

Hands-on lab

Create a small dataset for 'Search, filter, and sort', inspect the query/transaction behavior, then scale the dataset or add concurrent writers to expose the edge case.

8.4 Database transactions

Why this matters. Understand atomicity, isolation, rollback, and what the application may observe under concurrent changes.

Mental model. Data architecture is about correctness under scale and concurrency. Query shape, ordering, transactions, indexes, retries, and duplicate execution matter as much as endpoint syntax. Design contracts that stay deterministic when multiple users and workers act at once.

Core rules

- Always define deterministic ordering for pagination.
- Correctness under concurrent updates must be designed, not assumed.
- Indexes should match real query patterns and be validated with query plans.
- Durable asynchronous work needs idempotency, retry policy, and dead-letter or failure handling.
- A transaction groups changes atomically, but isolation level determines what concurrent work can observe.
- Keep transactions short and avoid holding locks while waiting on slow external services.

Request / execution flow

```
API input -> validation -> transaction/query plan -> storage -> deterministic result -> API response
```

Under the hood

For database transactions, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Database transactions' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Database transactions

```
EXPLAIN ANALYZE
SELECT id, name
FROM users
WHERE tenant_id = $1
ORDER BY id
LIMIT 50;
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Paginating without deterministic ordering.
- Ignoring transaction/concurrency behavior.
- Adding indexes without validating real query plans.

Production checklist

- Ordering deterministic?
- Index/query plan verified?
- Concurrency semantics documented?
- Limits prevent unbounded work?
- Retries/duplicates safe?

Questions a strong reviewer may ask

- How does this behave with 1M rows and concurrent writes?
- What is the stable ordering?
- Which index serves the query?

Hands-on lab

Create a small dataset for 'Database transactions', inspect the query/transaction behavior, then scale the dataset or add concurrent writers to expose the edge case.

8.5 Locks and optimistic concurrency

Why this matters. Prevent lost updates using row locks, versions, compare-and-swap patterns, or carefully chosen isolation.

Mental model. Data architecture is about correctness under scale and concurrency. Query shape, ordering, transactions, indexes, retries, and duplicate execution matter as much as endpoint syntax. Design contracts that stay deterministic when multiple users and workers act at once.

Core rules

- Always define deterministic ordering for pagination.
- Correctness under concurrent updates must be designed, not assumed.
- Indexes should match real query patterns and be validated with query plans.
- Durable asynchronous work needs idempotency, retry policy, and dead-letter or failure handling.

Request / execution flow

```
API input -> validation -> transaction/query plan -> storage -> deterministic result -> API response
```

Under the hood

For locks and optimistic concurrency, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Locks and optimistic concurrency' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Locks and optimistic concurrency

```
EXPLAIN ANALYZE
SELECT id, name
FROM users
WHERE tenant_id = $1
ORDER BY id
LIMIT 50;
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Paginating without deterministic ordering.
- Ignoring transaction/concurrency behavior.
- Adding indexes without validating real query plans.

Production checklist

- Ordering deterministic?
- Index/query plan verified?
- Concurrency semantics documented?
- Limits prevent unbounded work?
- Retries/duplicates safe?

Questions a strong reviewer may ask

- How does this behave with 1M rows and concurrent writes?
- What is the stable ordering?
- Which index serves the query?

Hands-on lab

Create a small dataset for 'Locks and optimistic concurrency', inspect the query/transaction behavior, then scale the dataset or add concurrent writers to expose the edge case.

8.6 Indexes and query plans

Why this matters. Understand why an index changes access cost, when it cannot be used, and how to inspect plans rather than guessing.

Mental model. Data architecture is about correctness under scale and concurrency. Query shape, ordering, transactions, indexes, retries, and duplicate execution matter as much as endpoint syntax. Design contracts that stay deterministic when multiple users and workers act at once.

Core rules

- Always define deterministic ordering for pagination.
- Correctness under concurrent updates must be designed, not assumed.
- Indexes should match real query patterns and be validated with query plans.
- Durable asynchronous work needs idempotency, retry policy, and dead-letter or failure handling.

Request / execution flow

```
API input -> validation -> transaction/query plan -> storage -> deterministic result -> API response
```

Under the hood

For indexes and query plans, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Indexes and query plans' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Indexes and query plans

```
EXPLAIN ANALYZE
SELECT id, name
FROM users
WHERE tenant_id = $1
ORDER BY id
LIMIT 50;
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Paginating without deterministic ordering.
- Ignoring transaction/concurrency behavior.
- Adding indexes without validating real query plans.

Production checklist

- Ordering deterministic?
- Index/query plan verified?
- Concurrency semantics documented?
- Limits prevent unbounded work?
- Retries/duplicates safe?

Questions a strong reviewer may ask

- How does this behave with 1M rows and concurrent writes?
- What is the stable ordering?
- Which index serves the query?

Hands-on lab

Create a small dataset for 'Indexes and query plans', inspect the query/transaction behavior, then scale the dataset or add concurrent writers to expose the edge case.

8.7 The N+1 query problem

Why this matters. Recognize repeated per-row database queries and replace them with joins, eager loading, batching, or redesigned access.

Mental model. Data architecture is about correctness under scale and concurrency. Query shape, ordering, transactions, indexes, retries, and duplicate execution matter as much as endpoint syntax. Design contracts that stay deterministic when multiple users and workers act at once.

Core rules

- Always define deterministic ordering for pagination.
- Correctness under concurrent updates must be designed, not assumed.
- Indexes should match real query patterns and be validated with query plans.
- Durable asynchronous work needs idempotency, retry policy, and dead-letter or failure handling.
- N+1 means one query to fetch a collection followed by one additional query per item.
- Detect it through query logs/traces and fix with joins, prefetching, batching, or denormalized read models where appropriate.

Request / execution flow

```
API input -> validation -> transaction/query plan -> storage -> deterministic result -> API response
```

Under the hood

For the n+1 query problem, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'The N+1 query problem' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: The N+1 query problem

```
EXPLAIN ANALYZE
SELECT id, name
FROM users
WHERE tenant_id = $1
ORDER BY id
LIMIT 50;
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Paginating without deterministic ordering.
- Ignoring transaction/concurrency behavior.
- Adding indexes without validating real query plans.

Production checklist

- Ordering deterministic?
- Index/query plan verified?
- Concurrency semantics documented?
- Limits prevent unbounded work?
- Retries/duplicates safe?

Questions a strong reviewer may ask

- How does this behave with 1M rows and concurrent writes?
- What is the stable ordering?
- Which index serves the query?

Hands-on lab

Create a small dataset for 'The N+1 query problem', inspect the query/transaction behavior, then scale the dataset or add concurrent writers to expose the edge case.

8.8 API contracts and versioning

Why this matters. Evolve request/response shapes without silently breaking clients and distinguish compatible from incompatible change.

Mental model. Data architecture is about correctness under scale and concurrency. Query shape, ordering, transactions, indexes, retries, and duplicate execution matter as much as endpoint syntax. Design contracts that stay deterministic when multiple users and workers act at once.

Core rules

- Always define deterministic ordering for pagination.
- Correctness under concurrent updates must be designed, not assumed.
- Indexes should match real query patterns and be validated with query plans.
- Durable asynchronous work needs idempotency, retry policy, and dead-letter or failure handling.

Request / execution flow

```
API input -> validation -> transaction/query plan -> storage -> deterministic result -> API response
```

Under the hood

For api contracts and versioning, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'API contracts and versioning' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: API contracts and versioning

```
EXPLAIN ANALYZE
SELECT id, name
FROM users
WHERE tenant_id = $1
ORDER BY id
LIMIT 50;
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Paginating without deterministic ordering.
- Ignoring transaction/concurrency behavior.
- Adding indexes without validating real query plans.

Production checklist

- Ordering deterministic?
- Index/query plan verified?
- Concurrency semantics documented?
- Limits prevent unbounded work?
- Retries/duplicates safe?

Questions a strong reviewer may ask

- How does this behave with 1M rows and concurrent writes?
- What is the stable ordering?
- Which index serves the query?

Hands-on lab

Create a small dataset for 'API contracts and versioning', inspect the query/transaction behavior, then scale the dataset or add concurrent writers to expose the edge case.

8.9 Validation and error envelopes

Why this matters. Provide stable machine-readable errors with field context, codes, and traceability without leaking internals.

Mental model. Data architecture is about correctness under scale and concurrency. Query shape, ordering, transactions, indexes, retries, and duplicate execution matter as much as endpoint syntax. Design contracts that stay deterministic when multiple users and workers act at once.

Core rules

- Always define deterministic ordering for pagination.
- Correctness under concurrent updates must be designed, not assumed.
- Indexes should match real query patterns and be validated with query plans.
- Durable asynchronous work needs idempotency, retry policy, and dead-letter or failure handling.

Request / execution flow

```
API input -> validation -> transaction/query plan -> storage -> deterministic result -> API response
```

Under the hood

For validation and error envelopes, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Validation and error envelopes' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Validation and error envelopes

```
EXPLAIN ANALYZE
SELECT id, name
FROM users
WHERE tenant_id = $1
ORDER BY id
LIMIT 50;
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Paginating without deterministic ordering.
- Ignoring transaction/concurrency behavior.
- Adding indexes without validating real query plans.

Production checklist

- Ordering deterministic?
- Index/query plan verified?
- Concurrency semantics documented?
- Limits prevent unbounded work?
- Retries/duplicates safe?

Questions a strong reviewer may ask

- How does this behave with 1M rows and concurrent writes?
- What is the stable ordering?
- Which index serves the query?

Hands-on lab

Create a small dataset for 'Validation and error envelopes', inspect the query/transaction behavior, then scale the dataset or add concurrent writers to expose the edge case.

8.10 Rate limiting

Why this matters. Protect capacity and abuse boundaries using identity-aware token buckets or similar algorithms.

Mental model. Data architecture is about correctness under scale and concurrency. Query shape, ordering, transactions, indexes, retries, and duplicate execution matter as much as endpoint syntax. Design contracts that stay deterministic when multiple users and workers act at once.

Core rules

- Always define deterministic ordering for pagination.
- Correctness under concurrent updates must be designed, not assumed.
- Indexes should match real query patterns and be validated with query plans.
- Durable asynchronous work needs idempotency, retry policy, and dead-letter or failure handling.
- Choose a meaningful key: IP, user, API key, tenant, endpoint, or combinations.
- Return clear retry information and protect the limiter itself from becoming a bottleneck.

Request / execution flow

```
API input -> validation -> transaction/query plan -> storage -> deterministic result -> API response
```

Under the hood

For rate limiting, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Rate limiting' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Rate limiting

```
EXPLAIN ANALYZE
SELECT id, name
FROM users
WHERE tenant_id = $1
ORDER BY id
LIMIT 50;
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Paginating without deterministic ordering.
- Ignoring transaction/concurrency behavior.
- Adding indexes without validating real query plans.

Production checklist

- Ordering deterministic?
- Index/query plan verified?
- Concurrency semantics documented?
- Limits prevent unbounded work?
- Retries/duplicates safe?

Questions a strong reviewer may ask

- How does this behave with 1M rows and concurrent writes?
- What is the stable ordering?
- Which index serves the query?

Hands-on lab

Create a small dataset for 'Rate limiting', inspect the query/transaction behavior, then scale the dataset or add concurrent writers to expose the edge case.

8.11 Queues and background workers

Why this matters. Move durable asynchronous work out of request latency and understand acknowledgement, retry, poison jobs, and idempotency.

Mental model. Data architecture is about correctness under scale and concurrency. Query shape, ordering, transactions, indexes, retries, and duplicate execution matter as much as endpoint syntax. Design contracts that stay deterministic when multiple users and workers act at once.

Core rules

- Always define deterministic ordering for pagination.
- Correctness under concurrent updates must be designed, not assumed.
- Indexes should match real query patterns and be validated with query plans.
- Durable asynchronous work needs idempotency, retry policy, and dead-letter or failure handling.

Request / execution flow

```
API input -> validation -> transaction/query plan -> storage -> deterministic result -> API response
```

Under the hood

For queues and background workers, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Queues and background workers' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Queues and background workers

```
EXPLAIN ANALYZE
SELECT id, name
FROM users
WHERE tenant_id = $1
ORDER BY id
LIMIT 50;
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Paginating without deterministic ordering.
- Ignoring transaction/concurrency behavior.
- Adding indexes without validating real query plans.

Production checklist

- Ordering deterministic?
- Index/query plan verified?
- Concurrency semantics documented?
- Limits prevent unbounded work?
- Retries/duplicates safe?

Questions a strong reviewer may ask

- How does this behave with 1M rows and concurrent writes?
- What is the stable ordering?
- Which index serves the query?

Hands-on lab

Create a small dataset for 'Queues and background workers', inspect the query/transaction behavior, then scale the dataset or add concurrent writers to expose the edge case.

8.12 Redis and application caching

Why this matters. Use TTLs, keys, invalidation, cache-aside patterns, and stampede protection deliberately.

Mental model. Data architecture is about correctness under scale and concurrency. Query shape, ordering, transactions, indexes, retries, and duplicate execution matter as much as endpoint syntax. Design contracts that stay deterministic when multiple users and workers act at once.

Core rules

- Always define deterministic ordering for pagination.
- Correctness under concurrent updates must be designed, not assumed.
- Indexes should match real query patterns and be validated with query plans.
- Durable asynchronous work needs idempotency, retry policy, and dead-letter or failure handling.

Request / execution flow

```
API input -> validation -> transaction/query plan -> storage -> deterministic result -> API response
```

Under the hood

For redis and application caching, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Redis and application caching' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Redis and application caching

```
EXPLAIN ANALYZE
SELECT id, name
FROM users
WHERE tenant_id = $1
ORDER BY id
LIMIT 50;
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Paginating without deterministic ordering.
- Ignoring transaction/concurrency behavior.
- Adding indexes without validating real query plans.

Production checklist

- Ordering deterministic?
- Index/query plan verified?
- Concurrency semantics documented?
- Limits prevent unbounded work?
- Retries/duplicates safe?

Questions a strong reviewer may ask

- How does this behave with 1M rows and concurrent writes?
- What is the stable ordering?
- Which index serves the query?

Hands-on lab

Create a small dataset for 'Redis and application caching', inspect the query/transaction behavior, then scale the dataset or add concurrent writers to expose the edge case.

8.13 Consistency and duplicate execution

Why this matters. Design operations under retries, concurrent workers, at-least-once delivery, and partial failure.

Mental model. Data architecture is about correctness under scale and concurrency. Query shape, ordering, transactions, indexes, retries, and duplicate execution matter as much as endpoint syntax. Design contracts that stay deterministic when multiple users and workers act at once.

Core rules

- Always define deterministic ordering for pagination.
- Correctness under concurrent updates must be designed, not assumed.
- Indexes should match real query patterns and be validated with query plans.
- Durable asynchronous work needs idempotency, retry policy, and dead-letter or failure handling.

Request / execution flow

```
API input -> validation -> transaction/query plan -> storage -> deterministic result -> API response
```

Under the hood

For consistency and duplicate execution, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Consistency and duplicate execution' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Consistency and duplicate execution

```
EXPLAIN ANALYZE
SELECT id, name
FROM users
WHERE tenant_id = $1
ORDER BY id
LIMIT 50;
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Paginating without deterministic ordering.
- Ignoring transaction/concurrency behavior.
- Adding indexes without validating real query plans.

Production checklist

- Ordering deterministic?
- Index/query plan verified?
- Concurrency semantics documented?
- Limits prevent unbounded work?
- Retries/duplicates safe?

Questions a strong reviewer may ask

- How does this behave with 1M rows and concurrent writes?
- What is the stable ordering?
- Which index serves the query?

Hands-on lab

Create a small dataset for 'Consistency and duplicate execution', inspect the query/transaction behavior, then scale the dataset or add concurrent writers to expose the edge case.

Part 9

IX. Production Infrastructure

Production adds network hops and process boundaries: CDN, load balancer, reverse proxy, containers, application instances, background workers, databases, and caches. Infrastructure can route and enrich requests, but it cannot infer your application-level authorization rules unless you configure that behavior explicitly.

Part flow: Browser -> CDN/LB -> reverse proxy -> app instance -> backend/database/cache -> response

9.1 Reverse proxies and Nginx

Why this matters. Understand path routing, TLS termination, buffering, timeouts, forwarded headers, and WebSocket upgrade configuration.

Mental model. Production adds network hops and process boundaries: CDN, load balancer, reverse proxy, containers, application instances, background workers, databases, and caches. Infrastructure can route and enrich requests, but it cannot infer your application-level authorization rules unless you configure that behavior explicitly.

Core rules

- Know which hop terminates TLS and which hop creates the next connection.
- Forwarded headers are trustworthy only when they come from trusted infrastructure and your app is configured accordingly.
- Stateless request handlers scale more simply because any instance can serve the next request.
- Health checks and graceful shutdown are part of correctness during deploys.
- Reverse proxies terminate client connections and create upstream connections; configure timeouts and forwarding deliberately.
- WebSockets require upgrade headers and idle timeout settings that differ from short HTTP requests.

Request / execution flow

```
Browser -> CDN/LB -> reverse proxy -> app instance -> backend/database/cache -> response
```

Under the hood

For reverse proxies and nginx, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Reverse proxies and Nginx' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Reverse proxies and Nginx

```
location /api/ {
    proxy_pass http://fastapi:8000/;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Request-ID $request_id;
}

location /socket.io/ {
    proxy_pass http://app;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Trusting forwarded headers from arbitrary clients.
- Using production timeouts that kill long-lived WebSockets.
- Running stateful sessions in one instance then scaling without shared state.

Production checklist

- TLS termination known?
- Forwarded headers trusted only from infrastructure?
- Timeouts suit HTTP and WebSockets?
- Readiness/graceful shutdown configured?

Questions a strong reviewer may ask

- Which component terminates this connection?
- How is health determined?
- What breaks when we add a second instance?

Hands-on lab

Sketch a Docker/Nginx version of 'Reverse proxies and Nginx', then use curl and logs to prove which hop receives and forwards each header.

9.2 Load balancers

Why this matters. Understand health checks, distribution, connection behavior, sticky sessions, and why stateless applications scale more easily.

Mental model. Production adds network hops and process boundaries: CDN, load balancer, reverse proxy, containers, application instances, background workers, databases, and caches. Infrastructure can route and enrich requests, but it cannot infer your application-level authorization rules unless you configure that behavior explicitly.

Core rules

- Know which hop terminates TLS and which hop creates the next connection.
- Forwarded headers are trustworthy only when they come from trusted infrastructure and your app is configured accordingly.
- Stateless request handlers scale more simply because any instance can serve the next request.
- Health checks and graceful shutdown are part of correctness during deploys.
- Health checks decide which instances receive traffic; readiness should fail while an instance cannot safely serve requests.
- Sticky sessions may hide stateful architecture problems; prefer external shared state when possible.

Request / execution flow

```
Browser -> CDN/LB -> reverse proxy -> app instance -> backend/database/cache -> response
```

Under the hood

For load balancers, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Load balancers' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Load balancers

```
Browser -> Load Balancer -> Reverse Proxy -> Next.js/FastAPI -> Database
| request id and forwarded metadata should be traceable |
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Trusting forwarded headers from arbitrary clients.
- Using production timeouts that kill long-lived WebSockets.
- Running stateful sessions in one instance then scaling without shared state.

Production checklist

- TLS termination known?
- Forwarded headers trusted only from infrastructure?
- Timeouts suit HTTP and WebSockets?
- Readiness/graceful shutdown configured?

Questions a strong reviewer may ask

- Which component terminates this connection?
- How is health determined?
- What breaks when we add a second instance?

Hands-on lab

Sketch a Docker/Nginx version of 'Load balancers', then use curl and logs to prove which hop receives and forwards each header.

9.3 API gateways

Why this matters. Understand centralized routing, authentication, rate limits, transformations, and the risk of hiding business policy in infrastructure.

Mental model. Production adds network hops and process boundaries: CDN, load balancer, reverse proxy, containers, application instances, background workers, databases, and caches. Infrastructure can route and enrich requests, but it cannot infer your application-level authorization rules unless you configure that behavior explicitly.

Core rules

- Know which hop terminates TLS and which hop creates the next connection.
- Forwarded headers are trustworthy only when they come from trusted infrastructure and your app is configured accordingly.
- Stateless request handlers scale more simply because any instance can serve the next request.
- Health checks and graceful shutdown are part of correctness during deploys.

Request / execution flow

```
Browser -> CDN/LB -> reverse proxy -> app instance -> backend/database/cache -> response
```

Under the hood

For api gateways, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'API gateways' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: API gateways

```
Browser -> Load Balancer -> Reverse Proxy -> Next.js/FastAPI -> Database
| request id and forwarded metadata should be traceable |
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Trusting forwarded headers from arbitrary clients.
- Using production timeouts that kill long-lived WebSockets.
- Running stateful sessions in one instance then scaling without shared state.

Production checklist

- TLS termination known?
- Forwarded headers trusted only from infrastructure?
- Timeouts suit HTTP and WebSockets?
- Readiness/graceful shutdown configured?

Questions a strong reviewer may ask

- Which component terminates this connection?
- How is health determined?
- What breaks when we add a second instance?

Hands-on lab

Sketch a Docker/Nginx version of 'API gateways', then use curl and logs to prove which hop receives and forwards each header.

9.4 Docker and containers

Why this matters. Understand images, layers, processes, networks, ports, volumes, environment, signals, and health checks.

Mental model. Production adds network hops and process boundaries: CDN, load balancer, reverse proxy, containers, application instances, background workers, databases, and caches. Infrastructure can route and enrich requests, but it cannot infer your application-level authorization rules unless you configure that behavior explicitly.

Core rules

- Know which hop terminates TLS and which hop creates the next connection.
- Forwarded headers are trustworthy only when they come from trusted infrastructure and your app is configured accordingly.
- Stateless request handlers scale more simply because any instance can serve the next request.
- Health checks and graceful shutdown are part of correctness during deploys.
- A container is a process isolation/package boundary, not a virtual machine.
- One container's localhost refers to itself; use service networking/DNS names to reach another container.

Request / execution flow

```
Browser -> CDN/LB -> reverse proxy -> app instance -> backend/database/cache -> response
```

Under the hood

For docker and containers, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Docker and containers' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Docker and containers

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:22-alpine
WORKDIR /app
COPY --from=build /app ./
CMD ["npm", "start"]
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Trusting forwarded headers from arbitrary clients.
- Using production timeouts that kill long-lived WebSockets.
- Running stateful sessions in one instance then scaling without shared state.

Production checklist

- TLS termination known?
- Forwarded headers trusted only from infrastructure?
- Timeouts suit HTTP and WebSockets?
- Readiness/graceful shutdown configured?

Questions a strong reviewer may ask

- Which component terminates this connection?
- How is health determined?
- What breaks when we add a second instance?

Hands-on lab

Sketch a Docker/Nginx version of 'Docker and containers', then use curl and logs to prove which hop receives and forwards each header.

9.5 Environment configuration and secrets

Why this matters. Keep environment-specific values outside source code, rotate secrets, and distinguish public client config from private server config.

Mental model. Production adds network hops and process boundaries: CDN, load balancer, reverse proxy, containers, application instances, background workers, databases, and caches. Infrastructure can route and enrich requests, but it cannot infer your application-level authorization rules unless you configure that behavior explicitly.

Core rules

- Know which hop terminates TLS and which hop creates the next connection.
- Forwarded headers are trustworthy only when they come from trusted infrastructure and your app is configured accordingly.
- Stateless request handlers scale more simply because any instance can serve the next request.
- Health checks and graceful shutdown are part of correctness during deploys.

Request / execution flow

```
Browser -> CDN/LB -> reverse proxy -> app instance -> backend/database/cache -> response
```

Under the hood

For environment configuration and secrets, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Environment configuration and secrets' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Environment configuration and secrets

```
Browser -> Load Balancer -> Reverse Proxy -> Next.js/FastAPI -> Database
| request id and forwarded metadata should be traceable |
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Trusting forwarded headers from arbitrary clients.
- Using production timeouts that kill long-lived WebSockets.
- Running stateful sessions in one instance then scaling without shared state.

Production checklist

- TLS termination known?
- Forwarded headers trusted only from infrastructure?
- Timeouts suit HTTP and WebSockets?
- Readiness/graceful shutdown configured?

Questions a strong reviewer may ask

- Which component terminates this connection?
- How is health determined?
- What breaks when we add a second instance?

Hands-on lab

Sketch a Docker/Nginx version of 'Environment configuration and secrets', then use curl and logs to prove which hop receives and forwards each header.

9.6 Health checks and readiness

Why this matters. Distinguish liveness, readiness, dependency health, graceful shutdown, and rollout safety.

Mental model. Production adds network hops and process boundaries: CDN, load balancer, reverse proxy, containers, application instances, background workers, databases, and caches. Infrastructure can route and enrich requests, but it cannot infer your application-level authorization rules unless you configure that behavior explicitly.

Core rules

- Know which hop terminates TLS and which hop creates the next connection.
- Forwarded headers are trustworthy only when they come from trusted infrastructure and your app is configured accordingly.
- Stateless request handlers scale more simply because any instance can serve the next request.
- Health checks and graceful shutdown are part of correctness during deploys.

Request / execution flow

```
Browser -> CDN/LB -> reverse proxy -> app instance -> backend/database/cache -> response
```

Under the hood

For health checks and readiness, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Health checks and readiness' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Health checks and readiness

```
Browser -> Load Balancer -> Reverse Proxy -> Next.js/FastAPI -> Database
| request id and forwarded metadata should be traceable |
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Trusting forwarded headers from arbitrary clients.
- Using production timeouts that kill long-lived WebSockets.
- Running stateful sessions in one instance then scaling without shared state.

Production checklist

- TLS termination known?
- Forwarded headers trusted only from infrastructure?
- Timeouts suit HTTP and WebSockets?
- Readiness/graceful shutdown configured?

Questions a strong reviewer may ask

- Which component terminates this connection?
- How is health determined?
- What breaks when we add a second instance?

Hands-on lab

Sketch a Docker/Nginx version of 'Health checks and readiness', then use curl and logs to prove which hop receives and forwards each header.

9.7 Deployments and rollbacks

Why this matters. Understand immutable builds, migrations, canary/rolling strategies, backward compatibility, and fast rollback.

Mental model. Production adds network hops and process boundaries: CDN, load balancer, reverse proxy, containers, application instances, background workers, databases, and caches. Infrastructure can route and enrich requests, but it cannot infer your application-level authorization rules unless you configure that behavior explicitly.

Core rules

- Know which hop terminates TLS and which hop creates the next connection.
- Forwarded headers are trustworthy only when they come from trusted infrastructure and your app is configured accordingly.
- Stateless request handlers scale more simply because any instance can serve the next request.
- Health checks and graceful shutdown are part of correctness during deploys.

Request / execution flow

```
Browser -> CDN/LB -> reverse proxy -> app instance -> backend/database/cache -> response
```

Under the hood

For deployments and rollbacks, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Deployments and rollbacks' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Deployments and rollbacks

```
Browser -> Load Balancer -> Reverse Proxy -> Next.js/FastAPI -> Database
| request id and forwarded metadata should be traceable |
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Trusting forwarded headers from arbitrary clients.
- Using production timeouts that kill long-lived WebSockets.
- Running stateful sessions in one instance then scaling without shared state.

Production checklist

- TLS termination known?
- Forwarded headers trusted only from infrastructure?
- Timeouts suit HTTP and WebSockets?
- Readiness/graceful shutdown configured?

Questions a strong reviewer may ask

- Which component terminates this connection?
- How is health determined?
- What breaks when we add a second instance?

Hands-on lab

Sketch a Docker/Nginx version of 'Deployments and rollbacks', then use curl and logs to prove which hop receives and forwards each header.

9.8 CDNs and static assets

Why this matters. Cache content near users, understand cache keys and invalidation, and separate static delivery from application requests.

Mental model. Production adds network hops and process boundaries: CDN, load balancer, reverse proxy, containers, application instances, background workers, databases, and caches. Infrastructure can route and enrich requests, but it cannot infer your application-level authorization rules unless you configure that behavior explicitly.

Core rules

- Know which hop terminates TLS and which hop creates the next connection.
- Forwarded headers are trustworthy only when they come from trusted infrastructure and your app is configured accordingly.
- Stateless request handlers scale more simply because any instance can serve the next request.
- Health checks and graceful shutdown are part of correctness during deploys.
- DNS maps names to records such as A/AAAA/CNAME; resolvers cache answers according to TTL.
- A DNS failure can occur before any TCP/TLS/HTTP activity, so application logs may show nothing.

Request / execution flow

```
Browser -> CDN/LB -> reverse proxy -> app instance -> backend/database/cache -> response
```

Under the hood

For cdns and static assets, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'CDNs and static assets' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: CDNs and static assets

```
Browser -> Load Balancer -> Reverse Proxy -> Next.js/FastAPI -> Database
| request id and forwarded metadata should be traceable |
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Trusting forwarded headers from arbitrary clients.
- Using production timeouts that kill long-lived WebSockets.
- Running stateful sessions in one instance then scaling without shared state.

Production checklist

- TLS termination known?
- Forwarded headers trusted only from infrastructure?
- Timeouts suit HTTP and WebSockets?
- Readiness/graceful shutdown configured?

Questions a strong reviewer may ask

- Which component terminates this connection?
- How is health determined?
- What breaks when we add a second instance?

Hands-on lab

Sketch a Docker/Nginx version of 'CDNs and static assets', then use curl and logs to prove which hop receives and forwards each header.

9.9 Horizontal scaling and statelessness

Why this matters. Run multiple application instances safely by moving shared state to deliberate external systems.

Mental model. Production adds network hops and process boundaries: CDN, load balancer, reverse proxy, containers, application instances, background workers, databases, and caches. Infrastructure can route and enrich requests, but it cannot infer your application-level authorization rules unless you configure that behavior explicitly.

Core rules

- Know which hop terminates TLS and which hop creates the next connection.
- Forwarded headers are trustworthy only when they come from trusted infrastructure and your app is configured accordingly.
- Stateless request handlers scale more simply because any instance can serve the next request.
- Health checks and graceful shutdown are part of correctness during deploys.

Request / execution flow

```
Browser -> CDN/LB -> reverse proxy -> app instance -> backend/database/cache -> response
```

Under the hood

For horizontal scaling and statelessness, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Horizontal scaling and statelessness' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Horizontal scaling and statelessness

```
Browser -> Load Balancer -> Reverse Proxy -> Next.js/FastAPI -> Database
| request id and forwarded metadata should be traceable |
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Trusting forwarded headers from arbitrary clients.
- Using production timeouts that kill long-lived WebSockets.
- Running stateful sessions in one instance then scaling without shared state.

Production checklist

- TLS termination known?
- Forwarded headers trusted only from infrastructure?
- Timeouts suit HTTP and WebSockets?
- Readiness/graceful shutdown configured?

Questions a strong reviewer may ask

- Which component terminates this connection?
- How is health determined?
- What breaks when we add a second instance?

Hands-on lab

Sketch a Docker/Nginx version of 'Horizontal scaling and statelessness', then use curl and logs to prove which hop receives and forwards each header.

Part 10

X. Security for Production Web Applications

Security is the enforcement of trust boundaries. Browser code and request headers are attacker-controlled unless independently verified. Authentication establishes identity; authorization decides allowed actions. Multi-tenant context, object ownership, input handling, cookies, and secrets must be protected at trusted server boundaries.

Part flow: Untrusted input -> authentication -> authorization -> validated operation -> protected resource
-> audited response

10.1 Threat modeling

Why this matters. Identify assets, trust boundaries, attackers, entry points, and abuse cases before selecting controls.

Mental model. Security is the enforcement of trust boundaries. Browser code and request headers are attacker-controlled unless independently verified. Authentication establishes identity; authorization decides allowed actions. Multi-tenant context, object ownership, input handling, cookies, and secrets must be protected at trusted server boundaries.

Core rules

- Assume browser input, query strings, JSON bodies, and custom headers are attacker controlled.
- Enforce authorization on the backend for every protected action and object, not only by hiding UI controls.
- Use parameterized queries and framework escaping rather than string concatenation.
- Reduce credential exposure with Secure/HttpOnly cookies, short token lifetimes where appropriate, and secret rotation.

Request / execution flow

```
Untrusted input -> authentication -> authorization -> validated operation -> protected resource ->
audited response
```

Under the hood

For threat modeling, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Threat modeling' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Threat modeling

```
Untrusted request -> authenticate -> authorize -> validate -> execute -> audit
Never treat a browser-supplied role or tenant header as proof by itself.
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Enforcing permissions only in the frontend.
- Logging secrets or tokens.
- Concatenating untrusted strings into queries or HTML.

Production checklist

- Trust boundary explicit?
- Authorization server-side?
- Secrets excluded from client/logs?
- Untrusted input validated/escaped/parameterized?

Questions a strong reviewer may ask

- What can the attacker control?
- Where is the permission enforced?
- What sensitive data could leak through logs or client bundles?

Hands-on lab

Threat-model 'Threat modeling': list attacker-controlled inputs, protected assets, the enforcement point, and one negative test that proves access is denied.

10.2 Broken access control

Why this matters. Enforce authorization server-side for every protected object and action, including tenant boundaries.

Mental model. Security is the enforcement of trust boundaries. Browser code and request headers are attacker-controlled unless independently verified. Authentication establishes identity; authorization decides allowed actions. Multi-tenant context, object ownership, input handling, cookies, and secrets must be protected at trusted server boundaries.

Core rules

- Assume browser input, query strings, JSON bodies, and custom headers are attacker controlled.
- Enforce authorization on the backend for every protected action and object, not only by hiding UI controls.
- Use parameterized queries and framework escaping rather than string concatenation.
- Reduce credential exposure with Secure/HttpOnly cookies, short token lifetimes where appropriate, and secret rotation.

Request / execution flow

```
Untrusted input -> authentication -> authorization -> validated operation -> protected resource ->
audited response
```

Under the hood

For broken access control, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Broken access control' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Broken access control

```
Untrusted request -> authenticate -> authorize -> validate -> execute -> audit
Never treat a browser-supplied role or tenant header as proof by itself.
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Enforcing permissions only in the frontend.
- Logging secrets or tokens.
- Concatenating untrusted strings into queries or HTML.

Production checklist

- Trust boundary explicit?
- Authorization server-side?
- Secrets excluded from client/logs?
- Untrusted input validated/escaped/parameterized?

Questions a strong reviewer may ask

- What can the attacker control?
- Where is the permission enforced?
- What sensitive data could leak through logs or client bundles?

Hands-on lab

Threat-model 'Broken access control': list attacker-controlled inputs, protected assets, the enforcement point, and one negative test that proves access is denied.

10.3 XSS

Why this matters. Understand injection into HTML/JS contexts, escaping, dangerous HTML APIs, and why HttpOnly cookies reduce token theft impact.

Mental model. Security is the enforcement of trust boundaries. Browser code and request headers are attacker-controlled unless independently verified. Authentication establishes identity; authorization decides allowed actions. Multi-tenant context, object ownership, input handling, cookies, and secrets must be protected at trusted server boundaries.

Core rules

- Assume browser input, query strings, JSON bodies, and custom headers are attacker controlled.
- Enforce authorization on the backend for every protected action and object, not only by hiding UI controls.
- Use parameterized queries and framework escaping rather than string concatenation.
- Reduce credential exposure with Secure/HttpOnly cookies, short token lifetimes where appropriate, and secret rotation.
- React escapes text interpolations by default, but dangerous HTML APIs and unsafe URL/script construction can bypass protections.
- CSP is defense-in-depth, not permission to render untrusted HTML unsafely.

Request / execution flow

```
Untrusted input -> authentication -> authorization -> validated operation -> protected resource ->
audited response
```

Under the hood

For xss, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'XSS' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: XSS

```
Untrusted request -> authenticate -> authorize -> validate -> execute -> audit  
Never treat a browser-supplied role or tenant header as proof by itself.
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Enforcing permissions only in the frontend.
- Logging secrets or tokens.
- Concatenating untrusted strings into queries or HTML.

Production checklist

- Trust boundary explicit?
- Authorization server-side?
- Secrets excluded from client/logs?
- Untrusted input validated/escaped/parameterized?

Questions a strong reviewer may ask

- What can the attacker control?
- Where is the permission enforced?
- What sensitive data could leak through logs or client bundles?

Hands-on lab

Threat-model 'XSS': list attacker-controlled inputs, protected assets, the enforcement point, and one negative test that proves access is denied.

10.4 Content Security Policy

Why this matters. Reduce script injection impact with explicit script/style/source policies and nonces where appropriate.

Mental model. Security is the enforcement of trust boundaries. Browser code and request headers are attacker-controlled unless independently verified. Authentication establishes identity; authorization decides allowed actions. Multi-tenant context, object ownership, input handling, cookies, and secrets must be protected at trusted server boundaries.

Core rules

- Assume browser input, query strings, JSON bodies, and custom headers are attacker controlled.
- Enforce authorization on the backend for every protected action and object, not only by hiding UI controls.
- Use parameterized queries and framework escaping rather than string concatenation.
- Reduce credential exposure with Secure/HttpOnly cookies, short token lifetimes where appropriate, and secret rotation.

Request / execution flow

```
Untrusted input -> authentication -> authorization -> validated operation -> protected resource ->
audited response
```

Under the hood

For content security policy, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Content Security Policy' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Content Security Policy

```
Untrusted request -> authenticate -> authorize -> validate -> execute -> audit
Never treat a browser-supplied role or tenant header as proof by itself.
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Enforcing permissions only in the frontend.
- Logging secrets or tokens.
- Concatenating untrusted strings into queries or HTML.

Production checklist

- Trust boundary explicit?
- Authorization server-side?
- Secrets excluded from client/logs?
- Untrusted input validated/escaped/parameterized?

Questions a strong reviewer may ask

- What can the attacker control?
- Where is the permission enforced?
- What sensitive data could leak through logs or client bundles?

Hands-on lab

Threat-model 'Content Security Policy': list attacker-controlled inputs, protected assets, the enforcement point, and one negative test that proves access is denied.

10.5 CSRF

Why this matters. Understand why browser-managed credentials create cross-site request risk and how SameSite plus CSRF tokens mitigate it.

Mental model. Security is the enforcement of trust boundaries. Browser code and request headers are attacker-controlled unless independently verified. Authentication establishes identity; authorization decides allowed actions. Multi-tenant context, object ownership, input handling, cookies, and secrets must be protected at trusted server boundaries.

Core rules

- Assume browser input, query strings, JSON bodies, and custom headers are attacker controlled.
- Enforce authorization on the backend for every protected action and object, not only by hiding UI controls.
- Use parameterized queries and framework escaping rather than string concatenation.
- Reduce credential exposure with Secure/HttpOnly cookies, short token lifetimes where appropriate, and secret rotation.
- CSRF relies on the browser attaching ambient credentials to a forged cross-site request.
- SameSite cookies, CSRF tokens, and origin checks are common layers; bearer tokens stored outside automatic cookie attachment have different risk tradeoffs.

Request / execution flow

```
Untrusted input -> authentication -> authorization -> validated operation -> protected resource ->
audited response
```

Under the hood

For csrf, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'CSRF' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: CSRF

```
Untrusted request -> authenticate -> authorize -> validate -> execute -> audit  
Never treat a browser-supplied role or tenant header as proof by itself.
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Enforcing permissions only in the frontend.
- Logging secrets or tokens.
- Concatenating untrusted strings into queries or HTML.

Production checklist

- Trust boundary explicit?
- Authorization server-side?
- Secrets excluded from client/logs?
- Untrusted input validated/escaped/parameterized?

Questions a strong reviewer may ask

- What can the attacker control?
- Where is the permission enforced?
- What sensitive data could leak through logs or client bundles?

Hands-on lab

Threat-model 'CSRF': list attacker-controlled inputs, protected assets, the enforcement point, and one negative test that proves access is denied.

10.6 SQL and NoSQL injection

Why this matters. Use parameterization and validated operators rather than concatenating user-controlled query syntax.

Mental model. Security is the enforcement of trust boundaries. Browser code and request headers are attacker-controlled unless independently verified. Authentication establishes identity; authorization decides allowed actions. Multi-tenant context, object ownership, input handling, cookies, and secrets must be protected at trusted server boundaries.

Core rules

- Assume browser input, query strings, JSON bodies, and custom headers are attacker controlled.
- Enforce authorization on the backend for every protected action and object, not only by hiding UI controls.
- Use parameterized queries and framework escaping rather than string concatenation.
- Reduce credential exposure with Secure/HttpOnly cookies, short token lifetimes where appropriate, and secret rotation.

Request / execution flow

```
Untrusted input -> authentication -> authorization -> validated operation -> protected resource ->
audited response
```

Under the hood

For sql and nosql injection, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'SQL and NoSQL injection' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: SQL and NoSQL injection

```
Untrusted request -> authenticate -> authorize -> validate -> execute -> audit
Never treat a browser-supplied role or tenant header as proof by itself.
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Enforcing permissions only in the frontend.
- Logging secrets or tokens.
- Concatenating untrusted strings into queries or HTML.

Production checklist

- Trust boundary explicit?
- Authorization server-side?
- Secrets excluded from client/logs?
- Untrusted input validated/escaped/parameterized?

Questions a strong reviewer may ask

- What can the attacker control?
- Where is the permission enforced?
- What sensitive data could leak through logs or client bundles?

Hands-on lab

Threat-model 'SQL and NoSQL injection': list attacker-controlled inputs, protected assets, the enforcement point, and one negative test that proves access is denied.

10.7 Authentication failures

Why this matters. Protect credential verification, token lifetime, session invalidation, brute-force limits, and secure recovery flows.

Mental model. Security is the enforcement of trust boundaries. Browser code and request headers are attacker-controlled unless independently verified. Authentication establishes identity; authorization decides allowed actions. Multi-tenant context, object ownership, input handling, cookies, and secrets must be protected at trusted server boundaries.

Core rules

- Assume browser input, query strings, JSON bodies, and custom headers are attacker controlled.
- Enforce authorization on the backend for every protected action and object, not only by hiding UI controls.
- Use parameterized queries and framework escaping rather than string concatenation.
- Reduce credential exposure with Secure/HttpOnly cookies, short token lifetimes where appropriate, and secret rotation.

Request / execution flow

```
Untrusted input -> authentication -> authorization -> validated operation -> protected resource ->
audited response
```

Under the hood

For authentication failures, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Authentication failures' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Authentication failures

```
Untrusted request -> authenticate -> authorize -> validate -> execute -> audit  
Never treat a browser-supplied role or tenant header as proof by itself.
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Enforcing permissions only in the frontend.
- Logging secrets or tokens.
- Concatenating untrusted strings into queries or HTML.

Production checklist

- Trust boundary explicit?
- Authorization server-side?
- Secrets excluded from client/logs?
- Untrusted input validated/escaped/parameterized?

Questions a strong reviewer may ask

- What can the attacker control?
- Where is the permission enforced?
- What sensitive data could leak through logs or client bundles?

Hands-on lab

Threat-model 'Authentication failures': list attacker-controlled inputs, protected assets, the enforcement point, and one negative test that proves access is denied.

10.8 Multi-tenant isolation

Why this matters. Treat tenant context as a security boundary and enforce it in backend authorization and data access, not only in UI headers.

Mental model. Security is the enforcement of trust boundaries. Browser code and request headers are attacker-controlled unless independently verified. Authentication establishes identity; authorization decides allowed actions. Multi-tenant context, object ownership, input handling, cookies, and secrets must be protected at trusted server boundaries.

Core rules

- Assume browser input, query strings, JSON bodies, and custom headers are attacker controlled.
- Enforce authorization on the backend for every protected action and object, not only by hiding UI controls.
- Use parameterized queries and framework escaping rather than string concatenation.
- Reduce credential exposure with Secure/HttpOnly cookies, short token lifetimes where appropriate, and secret rotation.

Request / execution flow

```
Untrusted input -> authentication -> authorization -> validated operation -> protected resource ->
audited response
```

Under the hood

For multi-tenant isolation, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Multi-tenant isolation' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Multi-tenant isolation

```
Untrusted request -> authenticate -> authorize -> validate -> execute -> audit  
Never treat a browser-supplied role or tenant header as proof by itself.
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Enforcing permissions only in the frontend.
- Logging secrets or tokens.
- Concatenating untrusted strings into queries or HTML.

Production checklist

- Trust boundary explicit?
- Authorization server-side?
- Secrets excluded from client/logs?
- Untrusted input validated/escaped/parameterized?

Questions a strong reviewer may ask

- What can the attacker control?
- Where is the permission enforced?
- What sensitive data could leak through logs or client bundles?

Hands-on lab

Threat-model 'Multi-tenant isolation': list attacker-controlled inputs, protected assets, the enforcement point, and one negative test that proves access is denied.

10.9 Security headers and HTTPS

Why this matters. Understand HSTS, frame controls, content sniffing protections,referrer policy, and secure transport.

Mental model. Security is the enforcement of trust boundaries. Browser code and request headers are attacker-controlled unless independently verified. Authentication establishes identity; authorization decides allowed actions. Multi-tenant context, object ownership, input handling, cookies, and secrets must be protected at trusted server boundaries.

Core rules

- Assume browser input, query strings, JSON bodies, and custom headers are attacker controlled.
- Enforce authorization on the backend for every protected action and object, not only by hiding UI controls.
- Use parameterized queries and framework escaping rather than string concatenation.
- Reduce credential exposure with Secure/HttpOnly cookies, short token lifetimes where appropriate, and secret rotation.

Request / execution flow

```
Untrusted input -> authentication -> authorization -> validated operation -> protected resource ->
audited response
```

Under the hood

For security headers and https, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Security headers and HTTPS' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Security headers and HTTPS

```
Untrusted request -> authenticate -> authorize -> validate -> execute -> audit  
Never treat a browser-supplied role or tenant header as proof by itself.
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Enforcing permissions only in the frontend.
- Logging secrets or tokens.
- Concatenating untrusted strings into queries or HTML.

Production checklist

- Trust boundary explicit?
- Authorization server-side?
- Secrets excluded from client/logs?
- Untrusted input validated/escaped/parameterized?

Questions a strong reviewer may ask

- What can the attacker control?
- Where is the permission enforced?
- What sensitive data could leak through logs or client bundles?

Hands-on lab

Threat-model 'Security headers and HTTPS': list attacker-controlled inputs, protected assets, the enforcement point, and one negative test that proves access is denied.

Part 11

XI. Observability, Testing, and Engineering Quality

High-standard production engineering requires evidence. Logs explain events, metrics summarize system behavior, traces connect distributed work, and tests prevent known regressions. Observability and testability are architecture properties, not decorations added after the feature is finished.

Part flow: User action -> request_id -> logs/metrics/spans/tests -> evidence -> diagnosis -> fix

11.1 Structured logging

Why this matters. Log events with stable fields, levels, context, and privacy discipline so production behavior is searchable.

Mental model. High-standard production engineering requires evidence. Logs explain events, metrics summarize system behavior, traces connect distributed work, and tests prevent known regressions. Observability and testability are architecture properties, not decorations added after the feature is finished.

Core rules

- Log structured fields such as `request_id`, `route`, `status`, `duration`, `user/tenant` identifiers when policy allows, and error code.
- Tests should prove behavior at the smallest useful boundary while critical journeys remain covered end to end.
- Measure production-like builds; development instrumentation can distort timing and lifecycle behavior.
- A reviewer should be able to trace data ownership, side effects, authorization, caching, and failure behavior from the code.
- Prefer JSON-like structured fields over string-only logs so production queries can group/filter reliably.
- Never log passwords, raw access tokens, private secrets, or unnecessary personal data.

Request / execution flow

```
User action -> request_id -> logs/metrics/spans/tests -> evidence -> diagnosis -> fix
```

Under the hood

For structured logging, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Structured logging' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Structured logging

```
{"level":"info","event":"request.complete","request_id":"8e3b","route":"/users","status":200,"duration_ms":34.2}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Measuring only development builds.
- Writing tests that mirror implementation instead of behavior.
- Logging prose without request IDs or stable fields.

Production checklist

- Request/trace IDs propagate?
- Useful logs/metrics/traces exist?
- Production-like performance measured?
- Critical journey covered by an integration/E2E test?

Questions a strong reviewer may ask

- How do we know this works in production?
- How do we trace one failure?
- Which test would fail if this regresses?

Hands-on lab

Instrument 'Structured logging' with a request ID, one metric, one structured log, and one automated test. Demonstrate how you would diagnose a failure.

11.2 Correlation and request IDs

Why this matters. Carry a request identifier through proxies, Next.js, FastAPI, workers, and logs to reconstruct one journey.

Mental model. High-standard production engineering requires evidence. Logs explain events, metrics summarize system behavior, traces connect distributed work, and tests prevent known regressions. Observability and testability are architecture properties, not decorations added after the feature is finished.

Core rules

- Log structured fields such as `request_id`, `route`, `status`, `duration`, `user/tenant` identifiers when policy allows, and error code.
- Tests should prove behavior at the smallest useful boundary while critical journeys remain covered end to end.
- Measure production-like builds; development instrumentation can distort timing and lifecycle behavior.
- A reviewer should be able to trace data ownership, side effects, authorization, caching, and failure behavior from the code.
- Generate or accept a trusted request ID at the edge and propagate it to downstream services and logs.
- Do not confuse a correlation ID with authentication; it is observability context, not proof of identity.

Request / execution flow

```
User action -> request_id -> logs/metrics/spans/tests -> evidence -> diagnosis -> fix
```

Under the hood

For correlation and request ids, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Correlation and request IDs' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Correlation and request IDs

```
request_id=8e3b route=/users status=200 duration_ms=34  
trace_id=7aa1 span=fastapi.users.list db_ms=11
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Measuring only development builds.
- Writing tests that mirror implementation instead of behavior.
- Logging prose without request IDs or stable fields.

Production checklist

- Request/trace IDs propagate?
- Useful logs/metrics/traces exist?
- Production-like performance measured?
- Critical journey covered by an integration/E2E test?

Questions a strong reviewer may ask

- How do we know this works in production?
- How do we trace one failure?
- Which test would fail if this regresses?

Hands-on lab

Instrument 'Correlation and request IDs' with a request ID, one metric, one structured log, and one automated test. Demonstrate how you would diagnose a failure.

11.3 Metrics

Why this matters. Measure request rate, errors, latency, saturation, queues, database pools, and business indicators instead of relying on anecdotes.

Mental model. High-standard production engineering requires evidence. Logs explain events, metrics summarize system behavior, traces connect distributed work, and tests prevent known regressions. Observability and testability are architecture properties, not decorations added after the feature is finished.

Core rules

- Log structured fields such as `request_id`, `route`, `status`, `duration`, `user/tenant` identifiers when policy allows, and error code.
- Tests should prove behavior at the smallest useful boundary while critical journeys remain covered end to end.
- Measure production-like builds; development instrumentation can distort timing and lifecycle behavior.
- A reviewer should be able to trace data ownership, side effects, authorization, caching, and failure behavior from the code.

Request / execution flow

```
User action -> request_id -> logs/metrics/spans/tests -> evidence -> diagnosis -> fix
```

Under the hood

For metrics, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Metrics' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Metrics

```
request_id=8e3b route=/users status=200 duration_ms=34  
trace_id=7aa1 span=fastapi.users.list db_ms=11
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Measuring only development builds.
- Writing tests that mirror implementation instead of behavior.
- Logging prose without request IDs or stable fields.

Production checklist

- Request/trace IDs propagate?
- Useful logs/metrics/traces exist?
- Production-like performance measured?
- Critical journey covered by an integration/E2E test?

Questions a strong reviewer may ask

- How do we know this works in production?
- How do we trace one failure?
- Which test would fail if this regresses?

Hands-on lab

Instrument 'Metrics' with a request ID, one metric, one structured log, and one automated test. Demonstrate how you would diagnose a failure.

11.4 Distributed tracing and OpenTelemetry

Why this matters. Represent a request as spans across services and use trace context to locate where latency or failure occurred.

Mental model. High-standard production engineering requires evidence. Logs explain events, metrics summarize system behavior, traces connect distributed work, and tests prevent known regressions. Observability and testability are architecture properties, not decorations added after the feature is finished.

Core rules

- Log structured fields such as `request_id`, `route`, `status`, `duration`, `user/tenant` identifiers when policy allows, and error code.
- Tests should prove behavior at the smallest useful boundary while critical journeys remain covered end to end.
- Measure production-like builds; development instrumentation can distort timing and lifecycle behavior.
- A reviewer should be able to trace data ownership, side effects, authorization, caching, and failure behavior from the code.
- A trace is composed of spans with parent/child relationships and timing.
- Propagate trace context across HTTP and queue boundaries to connect frontend/backend work.

Request / execution flow

```
User action -> request_id -> logs/metrics/spans/tests -> evidence -> diagnosis -> fix
```

Under the hood

For distributed tracing and opentelemetry, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Distributed tracing and OpenTelemetry' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Distributed tracing and OpenTelemetry

```
request_id=8e3b route=/users status=200 duration_ms=34
trace_id=7aa1 span=fastapi.users.list db_ms=11
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Measuring only development builds.
- Writing tests that mirror implementation instead of behavior.
- Logging prose without request IDs or stable fields.

Production checklist

- Request/trace IDs propagate?
- Useful logs/metrics/traces exist?
- Production-like performance measured?
- Critical journey covered by an integration/E2E test?

Questions a strong reviewer may ask

- How do we know this works in production?
- How do we trace one failure?
- Which test would fail if this regresses?

Hands-on lab

Instrument 'Distributed tracing and OpenTelemetry' with a request ID, one metric, one structured log, and one automated test. Demonstrate how you would diagnose a failure.

11.5 Frontend error monitoring

Why this matters. Capture uncaught errors, rejected promises, route failures, release information, and user-visible impact without collecting sensitive data.

Mental model. High-standard production engineering requires evidence. Logs explain events, metrics summarize system behavior, traces connect distributed work, and tests prevent known regressions. Observability and testability are architecture properties, not decorations added after the feature is finished.

Core rules

- Log structured fields such as `request_id`, `route`, `status`, `duration`, `user/tenant` identifiers when policy allows, and error code.
- Tests should prove behavior at the smallest useful boundary while critical journeys remain covered end to end.
- Measure production-like builds; development instrumentation can distort timing and lifecycle behavior.
- A reviewer should be able to trace data ownership, side effects, authorization, caching, and failure behavior from the code.

Request / execution flow

```
User action -> request_id -> logs/metrics/spans/tests -> evidence -> diagnosis -> fix
```

Under the hood

For frontend error monitoring, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Frontend error monitoring' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Frontend error monitoring

```
request_id=8e3b route=/users status=200 duration_ms=34  
trace_id=7aa1 span=fastapi.users.list db_ms=11
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Measuring only development builds.
- Writing tests that mirror implementation instead of behavior.
- Logging prose without request IDs or stable fields.

Production checklist

- Request/trace IDs propagate?
- Useful logs/metrics/traces exist?
- Production-like performance measured?
- Critical journey covered by an integration/E2E test?

Questions a strong reviewer may ask

- How do we know this works in production?
- How do we trace one failure?
- Which test would fail if this regresses?

Hands-on lab

Instrument 'Frontend error monitoring' with a request ID, one metric, one structured log, and one automated test. Demonstrate how you would diagnose a failure.

11.6 Unit, integration, component, and E2E tests

Why this matters. Choose the smallest test that proves the behavior while preserving enough integration to catch wiring failures.

Mental model. High-standard production engineering requires evidence. Logs explain events, metrics summarize system behavior, traces connect distributed work, and tests prevent known regressions. Observability and testability are architecture properties, not decorations added after the feature is finished.

Core rules

- Log structured fields such as `request_id`, `route`, `status`, `duration`, `user/tenant` identifiers when policy allows, and error code.
- Tests should prove behavior at the smallest useful boundary while critical journeys remain covered end to end.
- Measure production-like builds; development instrumentation can distort timing and lifecycle behavior.
- A reviewer should be able to trace data ownership, side effects, authorization, caching, and failure behavior from the code.

Request / execution flow

```
User action -> request_id -> logs/metrics/spans/tests -> evidence -> diagnosis -> fix
```

Under the hood

For unit, integration, component, and e2e tests, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Unit, integration, component, and E2E tests' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Unit, integration, component, and E2E tests

```
request_id=8e3b route=/users status=200 duration_ms=34
trace_id=7aa1 span=fastapi.users.list db_ms=11
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Measuring only development builds.
- Writing tests that mirror implementation instead of behavior.
- Logging prose without request IDs or stable fields.

Production checklist

- Request/trace IDs propagate?
- Useful logs/metrics/traces exist?
- Production-like performance measured?
- Critical journey covered by an integration/E2E test?

Questions a strong reviewer may ask

- How do we know this works in production?
- How do we trace one failure?
- Which test would fail if this regresses?

Hands-on lab

Instrument 'Unit, integration, component, and E2E tests' with a request ID, one metric, one structured log, and one automated test. Demonstrate how you would diagnose a failure.

11.7 React Testing Library

Why this matters. Test user-observable behavior rather than component implementation details and manage async UI reliably.

Mental model. High-standard production engineering requires evidence. Logs explain events, metrics summarize system behavior, traces connect distributed work, and tests prevent known regressions. Observability and testability are architecture properties, not decorations added after the feature is finished.

Core rules

- Log structured fields such as `request_id`, `route`, `status`, `duration`, `user/tenant` identifiers when policy allows, and error code.
- Tests should prove behavior at the smallest useful boundary while critical journeys remain covered end to end.
- Measure production-like builds; development instrumentation can distort timing and lifecycle behavior.
- A reviewer should be able to trace data ownership, side effects, authorization, caching, and failure behavior from the code.

Request / execution flow

```
User action -> request_id -> logs/metrics/spans/tests -> evidence -> diagnosis -> fix
```

Under the hood

For react testing library, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'React Testing Library' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: React Testing Library

```
request_id=8e3b route=/users status=200 duration_ms=34  
trace_id=7aa1 span=fastapi.users.list db_ms=11
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Measuring only development builds.
- Writing tests that mirror implementation instead of behavior.
- Logging prose without request IDs or stable fields.

Production checklist

- Request/trace IDs propagate?
- Useful logs/metrics/traces exist?
- Production-like performance measured?
- Critical journey covered by an integration/E2E test?

Questions a strong reviewer may ask

- How do we know this works in production?
- How do we trace one failure?
- Which test would fail if this regresses?

Hands-on lab

Instrument 'React Testing Library' with a request ID, one metric, one structured log, and one automated test. Demonstrate how you would diagnose a failure.

11.8 Vitest

Why this matters. Use fast unit/component tests, mocks, fake timers, and environment setup without confusing mocks with real integration.

Mental model. High-standard production engineering requires evidence. Logs explain events, metrics summarize system behavior, traces connect distributed work, and tests prevent known regressions. Observability and testability are architecture properties, not decorations added after the feature is finished.

Core rules

- Log structured fields such as `request_id`, `route`, `status`, `duration`, `user/tenant` identifiers when policy allows, and error code.
- Tests should prove behavior at the smallest useful boundary while critical journeys remain covered end to end.
- Measure production-like builds; development instrumentation can distort timing and lifecycle behavior.
- A reviewer should be able to trace data ownership, side effects, authorization, caching, and failure behavior from the code.

Request / execution flow

```
User action -> request_id -> logs/metrics/spans/tests -> evidence -> diagnosis -> fix
```

Under the hood

For vitest, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Vitest' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Vitest

```
request_id=8e3b route=/users status=200 duration_ms=34  
trace_id=7aa1 span=fastapi.users.list db_ms=11
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Measuring only development builds.
- Writing tests that mirror implementation instead of behavior.
- Logging prose without request IDs or stable fields.

Production checklist

- Request/trace IDs propagate?
- Useful logs/metrics/traces exist?
- Production-like performance measured?
- Critical journey covered by an integration/E2E test?

Questions a strong reviewer may ask

- How do we know this works in production?
- How do we trace one failure?
- Which test would fail if this regresses?

Hands-on lab

Instrument 'Vitest' with a request ID, one metric, one structured log, and one automated test. Demonstrate how you would diagnose a failure.

11.9 Playwright

Why this matters. Test full browser journeys, network behavior, authentication state, screenshots, and production-like routing.

Mental model. High-standard production engineering requires evidence. Logs explain events, metrics summarize system behavior, traces connect distributed work, and tests prevent known regressions. Observability and testability are architecture properties, not decorations added after the feature is finished.

Core rules

- Log structured fields such as `request_id`, `route`, `status`, `duration`, `user/tenant` identifiers when policy allows, and error code.
- Tests should prove behavior at the smallest useful boundary while critical journeys remain covered end to end.
- Measure production-like builds; development instrumentation can distort timing and lifecycle behavior.
- A reviewer should be able to trace data ownership, side effects, authorization, caching, and failure behavior from the code.

Request / execution flow

```
User action -> request_id -> logs/metrics/spans/tests -> evidence -> diagnosis -> fix
```

Under the hood

For playwright, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Playwright' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Playwright

```
request_id=8e3b route=/users status=200 duration_ms=34  
trace_id=7aa1 span=fastapi.users.list db_ms=11
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Measuring only development builds.
- Writing tests that mirror implementation instead of behavior.
- Logging prose without request IDs or stable fields.

Production checklist

- Request/trace IDs propagate?
- Useful logs/metrics/traces exist?
- Production-like performance measured?
- Critical journey covered by an integration/E2E test?

Questions a strong reviewer may ask

- How do we know this works in production?
- How do we trace one failure?
- Which test would fail if this regresses?

Hands-on lab

Instrument 'Playwright' with a request ID, one metric, one structured log, and one automated test. Demonstrate how you would diagnose a failure.

11.10 API contract and load testing

Why this matters. Verify response schemas and performance under realistic concurrency before users discover capacity limits.

Mental model. High-standard production engineering requires evidence. Logs explain events, metrics summarize system behavior, traces connect distributed work, and tests prevent known regressions. Observability and testability are architecture properties, not decorations added after the feature is finished.

Core rules

- Log structured fields such as `request_id`, `route`, `status`, `duration`, `user/tenant` identifiers when policy allows, and error code.
- Tests should prove behavior at the smallest useful boundary while critical journeys remain covered end to end.
- Measure production-like builds; development instrumentation can distort timing and lifecycle behavior.
- A reviewer should be able to trace data ownership, side effects, authorization, caching, and failure behavior from the code.

Request / execution flow

```
User action -> request_id -> logs/metrics/spans/tests -> evidence -> diagnosis -> fix
```

Under the hood

For api contract and load testing, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'API contract and load testing' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: API contract and load testing

```
request_id=8e3b route=/users status=200 duration_ms=34  
trace_id=7aa1 span=fastapi.users.list db_ms=11
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Measuring only development builds.
- Writing tests that mirror implementation instead of behavior.
- Logging prose without request IDs or stable fields.

Production checklist

- Request/trace IDs propagate?
- Useful logs/metrics/traces exist?
- Production-like performance measured?
- Critical journey covered by an integration/E2E test?

Questions a strong reviewer may ask

- How do we know this works in production?
- How do we trace one failure?
- Which test would fail if this regresses?

Hands-on lab

Instrument 'API contract and load testing' with a request ID, one metric, one structured log, and one automated test. Demonstrate how you would diagnose a failure.

11.11 Code review for web systems

Why this matters. Review execution location, side effects, request paths, auth, caching, error handling, observability, and performance rather than only syntax.

Mental model. High-standard production engineering requires evidence. Logs explain events, metrics summarize system behavior, traces connect distributed work, and tests prevent known regressions. Observability and testability are architecture properties, not decorations added after the feature is finished.

Core rules

- Log structured fields such as `request_id`, `route`, `status`, `duration`, `user/tenant` identifiers when policy allows, and error code.
- Tests should prove behavior at the smallest useful boundary while critical journeys remain covered end to end.
- Measure production-like builds; development instrumentation can distort timing and lifecycle behavior.
- A reviewer should be able to trace data ownership, side effects, authorization, caching, and failure behavior from the code.

Request / execution flow

```
User action -> request_id -> logs/metrics/spans/tests -> evidence -> diagnosis -> fix
```

Under the hood

For code review for web systems, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Code review for web systems' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Code review for web systems

```
request_id=8e3b route=/users status=200 duration_ms=34  
trace_id=7aa1 span=fastapi.users.list db_ms=11
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Measuring only development builds.
- Writing tests that mirror implementation instead of behavior.
- Logging prose without request IDs or stable fields.

Production checklist

- Request/trace IDs propagate?
- Useful logs/metrics/traces exist?
- Production-like performance measured?
- Critical journey covered by an integration/E2E test?

Questions a strong reviewer may ask

- How do we know this works in production?
- How do we trace one failure?
- Which test would fail if this regresses?

Hands-on lab

Instrument 'Code review for web systems' with a request ID, one metric, one structured log, and one automated test. Demonstrate how you would diagnose a failure.

Part 12

XII. Failure Case Studies and Debugging Playbooks

A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Part flow: Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix -> regression test

12.1 Case study: Axios POST keeps firing

Why this matters. Trace repeated requests back to render-time side effects, Effect loops, unstable dependencies, remounting, timers, retries, or duplicate event wiring.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.
- Axios rejects promises for many non-2xx statuses by default, unlike fetch, but repeated calls still originate from application control flow or configured retry logic.
- Use an Axios instance for baseURL/common headers and interceptors sparingly so request behavior remains inspectable.
- Search for every call site and use the Network panel Initiator/stack trace to identify who invoked the request.
- Common causes include request-in-render, Effects without stable dependencies, remounts due to keys, timers/polling, duplicate listeners, form submit plus click, and explicit retry interceptors.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix -> regression test
```

Under the hood

For case study: axios post keeps firing, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: Axios POST keeps firing' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: Axios POST keeps firing

```
// BUG: request runs every render
function Page() {
  axios.post('/api/missing-endpoint', payload);
  return <div>...</div>;
}

// Better: interaction-owned side effect
async function handleSubmit() {
  await axios.post('/api/users', payload);
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.
- `axios.post` is called in the component body.
- Effect dependency changes every render.
- Form submit and button click both call the mutation.
- A retry interceptor or polling timer is active.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: Axios POST keeps firing' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

12.2 Case study: opening an MUI Dialog re-renders the page

Why this matters. Separate expected parent state rendering from expensive subtree rendering, portal behavior, layout/paint work, and actual network side effects.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.
- Opening a controlled Dialog usually calls `setOpen(true)`, which re-renders the owner; that alone is not a bug.
- MUI modal content may mount/unmount and uses portal/focus management; expensive content can be profiled and optionally kept mounted when justified.
- `setOpen(true)` causes the component that owns `open` to re-render; descendants may also render by default.
- Profile whether the rerender is expensive before optimizing. Move dialog state lower, split expensive subtrees, memoize only when measured, or keep expensive modal content mounted when appropriate.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix -> regression test
```

Under the hood

For case study: opening an mui dialog re-renders the page, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: opening an MUI Dialog re-renders the page' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: opening an MUI Dialog re-renders the page

```
function DialogController() {  
  const [open, setOpen] = useState(false);  
  return <>  
    <Button onClick={() => setOpen(true)}>New user</Button>  
    <Dialog open={open} onClose={() => setOpen(false)}>  
      <UserForm />  
    </Dialog>  
  </>;  
}
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.
- Treating an expected owner re-render as a bug without profiling.
- Keeping expensive table state in the same owner as dialog state.
- Measuring dev Strict Mode logs instead of production commit cost.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: opening an MUI Dialog re-renders the page' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

12.3 Case study: screenshot or viewport change triggers work

Why this matters. Trace resize/media-query/focus/visibility observers and distinguish React rendering from browser repainting.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.
- DevTools screenshot tooling may resize the viewport or alter focus/visibility depending on workflow.
- Hooks such as useMediaQuery, resize observers, or focus/visibility listeners can update state; browser repaint by itself is not a React render.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix -> regression test
```

Under the hood

For case study: screenshot or viewport change triggers work, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: screenshot or viewport change triggers work' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: screenshot or viewport change triggers work

1. Reproduce
2. Capture Network initiator + timing
3. Capture React Profiler
4. Correlate server logs
5. Change one hypothesis at a time

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: screenshot or viewport change triggers work' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

12.4 Case study: Server Component request misses headers

Why this matters. Understand why the outbound Next.js-to-FastAPI request is new and must receive explicit trusted context.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.
- Read trusted incoming context with server APIs, then explicitly construct downstream headers.
- Do not forward raw Authorization/Cookie headers blindly if your architecture expects a server-derived identity/tenant context.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix -> regression test
```

Under the hood

For case study: server component request misses headers, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: Server Component request misses headers' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: Server Component request misses headers

```
const incoming = await headers();
const trustedTenant = incoming.get('x-trusted-tenant');

await fetch(`${process.env.API_URL}/users`, {
  headers: {
    ...(trustedTenant ? {'x-tenant-id': trustedTenant} : {}),
    'x-request-id': incoming.get('x-request-id') ?? crypto.randomUUID(),
  },
});
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: Server Component request misses headers' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

12.5 Case study: client fetch works through proxy but server fetch does not

Why this matters. Map the exact request paths and prove which request enters proxy.ts and which leaves the Next.js server directly.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix -> regression test
```

Under the hood

For case study: client fetch works through proxy but server fetch does not, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: client fetch works through proxy but server fetch does not' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: client fetch works through proxy but server fetch does not

1. Reproduce
2. Capture Network initiator + timing
3. Capture React Profiler
4. Correlate server logs
5. Change one hypothesis at a time

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: client fetch works through proxy but server fetch does not' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

12.6 Case study: CORS fails only in the browser

Why this matters. Recognize CORS as browser enforcement and compare browser-to-API with server-to-server requests.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.
- CORS does not stop a request from reaching the server in all cases; it primarily controls whether browser JavaScript may access the response.
- A preflight is an OPTIONS request that asks whether the actual cross-origin request is allowed.
- Credentials require explicit compatible origin configuration; wildcard behavior is restricted.
- Compare the browser request's Origin/preflight with a server-to-server curl call.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix ->
regression test
```

Under the hood

For case study: cors fails only in the browser, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: CORS fails only in the browser' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: CORS fails only in the browser

1. Reproduce
2. Capture Network initiator + timing
3. Capture React Profiler
4. Correlate server logs
5. Change one hypothesis at a time

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: CORS fails only in the browser' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

12.7 Case study: cookie is not sent

Why this matters. Inspect Domain, Path, Secure, SameSite, credentials mode, cross-origin rules, and whether the request is browser or server initiated.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix ->
regression test
```

Under the hood

For case study: cookie is not sent, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: cookie is not sent' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: cookie is not sent

1. Reproduce
2. Capture Network initiator + timing
3. Capture React Profiler
4. Correlate server logs
5. Change one hypothesis at a time

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.
- Secure cookie on HTTP.
- SameSite blocks cross-site context.
- credentials option omitted.
- Domain/Path do not match.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: cookie is not sent' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

12.8 Case study: 401 versus 403

Why this matters. Separate missing/invalid identity from authenticated-but-not-authorized access and keep frontend handling aligned with backend semantics.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix -> regression test
```

Under the hood

For case study: 401 versus 403, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: 401 versus 403' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: 401 versus 403

1. Reproduce
2. Capture Network initiator + timing
3. Capture React Profiler
4. Correlate server logs
5. Change one hypothesis at a time

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: 401 versus 403' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

12.9 Case study: 404, 405, and 422

Why this matters. Distinguish missing route, wrong HTTP method, and validation failure using the Network panel and backend route table.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix -> regression test
```

Under the hood

For case study: 404, 405, and 422, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: 404, 405, and 422' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: 404, 405, and 422

1. Reproduce
2. Capture Network initiator + timing
3. Capture React Profiler
4. Correlate server logs
5. Change one hypothesis at a time

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: 404, 405, and 422' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

12.10 Case study: hydration mismatch

Why this matters. Trace server/client output differences, browser-only values, time/random data, invalid HTML, and third-party mutations.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.
- Hydration attaches client behavior to server-generated HTML; server and client initial output must match.
- Random values, timestamps, browser-only conditionals, locale differences, or invalid HTML nesting can cause mismatches.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix ->
regression test
```

Under the hood

For case study: hydration mismatch, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: hydration mismatch' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: hydration mismatch

1. Reproduce
2. Capture Network initiator + timing
3. Capture React Profiler
4. Correlate server logs
5. Change one hypothesis at a time

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: hydration mismatch' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

12.11 Case study: stale data after mutation

Why this matters. Trace browser caches, Next.js caches, query caches, server caches, and missing invalidation/revalidation.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix ->
regression test
```

Under the hood

For case study: stale data after mutation, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: stale data after mutation' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: stale data after mutation

1. Reproduce
2. Capture Network initiator + timing
3. Capture React Profiler
4. Correlate server logs
5. Change one hypothesis at a time

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: stale data after mutation' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

12.12 Case study: slow page caused by a request waterfall

Why this matters. Use Network timing and server traces to convert independent sequential work into parallel work or server-side aggregation.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix ->
regression test
```

Under the hood

For case study: slow page caused by a request waterfall, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: slow page caused by a request waterfall' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: slow page caused by a request waterfall

1. Reproduce
2. Capture Network initiator + timing
3. Capture React Profiler
4. Correlate server logs
5. Change one hypothesis at a time

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: slow page caused by a request waterfall' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

12.13 Case study: WebSocket disconnects in production

Why this matters. Check upgrade headers, proxy timeouts, heartbeat policy, load balancer behavior, reconnect logic, and shared connection state.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix ->
regression test
```

Under the hood

For case study: websocket disconnects in production, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: WebSocket disconnects in production' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: WebSocket disconnects in production

1. Reproduce
2. Capture Network initiator + timing
3. Capture React Profiler
4. Correlate server logs
5. Change one hypothesis at a time

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.
- Proxy upgrade headers missing.
- Idle timeout shorter than heartbeat interval.
- Reconnect loop has no backoff.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: WebSocket disconnects in production' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

12.14 Case study: form submits twice

Why this matters. Check native submit plus click handlers, Strict Mode misconceptions, duplicate listeners, disabled states, retries, and idempotency.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.
- A form onSubmit and a button onClick can both call the same mutation if wired carelessly.
- Disable or lock the submit path while in flight and use server-side idempotency for operations where duplicates are costly.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix -> regression test
```

Under the hood

For case study: form submits twice, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: form submits twice' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: form submits twice

```
// Avoid calling the same mutation from both onClick and onSubmit.  
<form onSubmit={handleSubmit}>  
  <button type="submit" disabled={saving}>  
    {saving ? 'Saving...' : 'Save'}  
  </button>  
</form>
```

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: form submits twice' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

12.15 Case study: memory leak from listeners, timers, or subscriptions

Why this matters. Match every acquired resource with cleanup and prove lifecycle behavior by mounting and unmounting repeatedly.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix ->
regression test
```

Under the hood

For case study: memory leak from listeners, timers, or subscriptions, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: memory leak from listeners, timers, or subscriptions' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: memory leak from listeners, timers, or subscriptions

1. Reproduce
2. Capture Network initiator + timing
3. Capture React Profiler
4. Correlate server logs
5. Change one hypothesis at a time

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: memory leak from listeners, timers, or subscriptions' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

12.16 Case study: works in development, fails in production

Why this matters. Compare environment variables, origins, proxy/load-balancer paths, caching, build-time assumptions, HTTPS, minification, and concurrency.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix ->
regression test
```

Under the hood

For case study: works in development, fails in production, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: works in development, fails in production' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: works in development, fails in production

1. Reproduce
2. Capture Network initiator + timing
3. Capture React Profiler
4. Correlate server logs
5. Change one hypothesis at a time

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: works in development, fails in production' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

12.17 Case study: tracing one request from browser to database

Why this matters. Use browser Network, request IDs, Next.js logs, FastAPI logs, database timing, and response headers to reconstruct the entire path.

Mental model. A debugging playbook converts a vague symptom into observable questions. Reproduce reliably, inspect the browser and server evidence, identify the exact trigger, draw the request/render path, then remove hypotheses one by one. Fix the cause at the correct boundary rather than silencing the symptom.

Core rules

- Reproduce the problem with the smallest reliable sequence of actions.
- Inspect Network initiator/timing, React Profiler, and server logs before modifying code.
- Draw the exact request and render path, including proxies and server-to-server calls.
- Add a regression test or instrumentation guard after fixing the root cause.

Request / execution flow

```
Symptom -> reproduce -> observe -> identify trigger -> locate boundary -> prove cause -> fix ->
regression test
```

Under the hood

For case study: tracing one request from browser to database, the most important engineering skill is to identify ownership. The code that triggers the operation, the process that executes it, and the component that observes the result may all be different. Do not collapse them into one mental box. Inspect the real boundary and decide what data, credentials, lifetime, and failure semantics belong there.

In a high-standard production review, 'Case study: tracing one request from browser to database' is rarely evaluated in isolation. Reviewers care about how it behaves on retries, under concurrency, during navigation/unmount, behind proxies, with authorization, and when dependencies are slow or unavailable. Make those behaviors explicit in code and tests.

Concrete example: Case study: tracing one request from browser to database

1. Reproduce
2. Capture Network initiator + timing
3. Capture React Profiler
4. Correlate server logs
5. Change one hypothesis at a time

Step-by-step reasoning

1. Identify the trigger and write it down before looking at implementation details.
2. Identify where the shown code executes and what state/credentials are available in that environment.
3. Mark every side effect and every boundary crossing. A new outbound HTTP call is a new request.
4. Use the appropriate inspection tool: browser Network/Performance, React Profiler, server logs, traces, database query logs, or proxy logs.
5. Test one failure mode deliberately: invalid input, missing auth, timeout, cancellation, duplicate trigger, or stale cache.

Common bugs

- Fixing the visible symptom without proving the trigger.
- Turning off Strict Mode, caching, or security controls as a workaround.
- Failing to add a regression check after the fix.

Production checklist

- Symptom reproduced?
- Initiator and request path identified?
- Root cause proven with one controlled change?
- Regression test or monitor added?

Questions a strong reviewer may ask

- What evidence proves this is the trigger?
- Which tool shows the initiator?
- Why will the fix remain correct in production?

Hands-on lab

Recreate 'Case study: tracing one request from browser to database' intentionally in a tiny app, capture the evidence, then fix the root cause and add a regression test.

Part A

Quick decision tables

Where should this code run?

Need	Default place to consider
Read secret/server credential	Server Component, Route Handler, Server Action, or backend service
Interactive browser state/event	Client Component
Initial data needed for server-rendered page	Server Component or backend called from server
Browser-safe API facade/aggregation	Route Handler / BFF
Mutation from a form with server ownership	Server Action or browser API call, with authorization at trusted backend boundary
Persistent bidirectional realtime messages	WebSocket/Socket.IO when justified
One-way event stream	SSE when suitable
Durable background work	Queue + worker, not in-process BackgroundTasks alone

Common status interpretation

Code	Meaning to investigate
400	Malformed or semantically invalid request by application convention
401	Missing/invalid authentication
403	Authenticated but not authorized
404	Resource/route not found (or intentionally hidden)
405	Route exists but method is not allowed
409	Conflict with current resource state
422	Validation failure, especially common in FastAPI
429	Rate limited
500	Unexpected application failure
502	Upstream/proxy received invalid/failing upstream response
503	Service unavailable/not ready/overloaded
504	Gateway timeout waiting for upstream

Part B

Production review checklist

- Execution location is explicit for every important function.
- Component render paths are pure; network mutations occur in event/action paths.
- Effects synchronize external systems and clean up acquired resources.
- Every browser/server/backend request path is documented and observable.
- Authentication and authorization are enforced at trusted server boundaries.
- Tenant context cannot be spoofed by changing a browser header.
- Pagination and list queries are bounded, ordered, and indexed.
- Retryable operations are idempotent or protected by idempotency keys.
- Caching and invalidation are deliberate across browser, CDN, Next.js, Redis, and database layers.
- WebSocket/SSE connections have authentication, heartbeat/timeout, reconnect, and cleanup policy.
- Production builds are profiled; client bundle and server latency are measured.
- Logs, metrics, traces, and request IDs are sufficient to reconstruct critical failures.
- Critical user journeys have integration/E2E coverage and negative authorization tests.
- Deployments have readiness, graceful shutdown, migration compatibility, and rollback strategy.

Part C

Primary references

The concepts in this handbook are grounded in stable web fundamentals and current official framework guidance. Framework APIs evolve; verify exact syntax for your pinned project version.

- MDN Web Docs: HTTP, Fetch, CORS, Web APIs, browser performance - <https://developer.mozilla.org/>
- React documentation: rendering, Effects, Strict Mode, performance - <https://react.dev/>
- Next.js documentation: App Router, Server/Client Components, Route Handlers, Server Actions, Proxy, caching - <https://nextjs.org/docs>
- FastAPI documentation: routing, dependencies, middleware, CORS, async, WebSockets - <https://fastapi.tiangolo.com/>
- Material UI documentation: Dialog, Modal, bundle/performance guidance - <https://mui.com/material-ui/>
- Axios documentation: requests, instances, interceptors, cancellation - <https://axios-http.com/>
- OWASP: Web application security guidance and Top 10 - <https://owasp.org/>
- OpenTelemetry documentation - <https://opentelemetry.io/docs/>

Part D

Glossary

ASGI Python asynchronous server/application interface used by frameworks such as FastAPI/Starlette.

BFF Backend for Frontend: a server layer shaped around one frontend's needs.

Commit React phase that applies calculated changes to the host DOM.

CORS Browser-enforced cross-origin response access policy controlled by HTTP headers.

Hydration Client React attaching interactivity to server-generated HTML.

Idempotent Repeating the same operation has the same intended effect as performing it once.

Origin Scheme + host + port, used heavily by browser security policy.

Proxy A component that receives a request and forwards/transforms/reroutes it toward another destination.

RSC React Server Components model/payload used by frameworks such as Next.js App Router.

Side effect Work that interacts with state outside the pure computation: network, storage, timers, DOM mutation, subscriptions, logging, navigation, etc.

WebSocket Persistent full-duplex message connection established via an HTTP upgrade handshake.

Part E

Final mental model

When you get stuck, do not ask first, “Which React/Next function should I use?” Ask:

1. What event caused work to begin?
2. Which machine/process is executing this line?
3. Is this calculation pure, or does it change the outside world?
4. If there is a request, who created it and which hops will see it?
5. Which credentials/context are actually on that request?
6. What state changes when it completes?
7. What will React render, what will React commit, and what will the browser repaint?
8. What evidence in Network/Profiler/logs/traces proves each step?

If you can answer those questions, the framework stops feeling like a collection of arbitrary rules and becomes a set of tools placed on top of understandable systems.